



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE ESTUDIOS SUPERIORES ARAGON

“CALIDAD EN LA ADMINISTRACIÓN DE REQUERIMIENTOS PARA INGENIERÍA INVERSA”

TESIS

**que para obtener el título de
INGENIERO EN COMPUTACIÓN**

PRESENTA:

MARGARITA ARELLANO JUÁREZ

ASESOR:

DRA. GABRIELA ORDAZ VILLEGAS

MÉXICO, 2013





Universidad Nacional
Autónoma de México

Dirección General de Bibliotecas de la UNAM

Biblioteca Central



UNAM – Dirección General de Bibliotecas

Tesis Digitales

Restricciones de uso

DERECHOS RESERVADOS ©

PROHIBIDA SU REPRODUCCIÓN TOTAL O PARCIAL

Todo el material contenido en esta tesis esta protegido por la Ley Federal del Derecho de Autor (LFDA) de los Estados Unidos Mexicanos (México).

El uso de imágenes, fragmentos de videos, y demás material que sea objeto de protección de los derechos de autor, será exclusivamente para fines educativos e informativos y deberá citar la fuente donde la obtuvo mencionando el autor o autores. Cualquier uso distinto como el lucro, reproducción, edición o modificación, será perseguido y sancionado por el respectivo titular de los Derechos de Autor.

ÍNDICE

INTRODUCCIÓN	1
CAPÍTULO 1. CICLOS DE VIDA DE LOS SISTEMAS ORIENTADOS A OBJETOS	4
1.1. INTRODUCCIÓN A LOS CICLOS DE VIDA	5
1.1.1. Ciclo de Vida Lineal	8
1.1.2. Ciclo de Vida en Cascada Simple	9
1.1.3. Ciclo de Vida en “V”	11
1.1.4. Ciclo de Vida Sashimi	12
1.1.5. Ciclo de Vida en Cascada Con Sub Proyectos	13
1.1.6. Ciclo de Vida Iterativo	14
1.1.7. Ciclo de Vida por Prototipos	15
1.1.8. Ciclo de Vida Evolutivo	16
1.1.9. Ciclo de Vida Incremental	17
1.1.10. Ciclo de Vida en Espiral	18
1.1.11. Ciclo de Vida Orientado a Objetos	18
1.2. ETAPAS DEL CICLO DE VIDA ORIENTADO A OBJETOS	25
1.2.1. Especificación de los Requerimientos	25
1.2.2. Análisis	26
1.2.3. Diseño	27
1.2.4. Desarrollo	28
1.2.5. Pruebas	29
1.2.6. Implementación	30
1.2.7. Mantenimiento	30
1.2.8. Síntesis	31
REFERENCIAS BIBLIOGRÁFICAS	32
REFERENCIAS ELECTRÓNICAS	32
CAPÍTULO 2. CALIDAD EN LOS SISTEMAS DE INFORMACIÓN	34
2.1. INTRODUCCIÓN A LA CALIDAD DE LOS SISTEMAS DE INFORMACIÓN	35
2.2. CALIDAD EN LOS REQUERIMIENTOS	38
2.2.1. Características de los Requerimientos (ISO, 20091)	39
2.2.2. Clasificación de los Requerimientos	40
2.2.3. Lineamientos y Criterios de Calidad para los Requerimientos	42

2.2.3.1. Características de calidad de los requerimientos de IEEE ESTÁNDAR 830-1998	42
2.2.3.2. Características de calidad de los requerimientos según la norma ISO 9126 ISO/IEC9126	46
2.3. ADMINISTRACIÓN DE REQUERIMIENTOS	49
2.3.1. Etapas de la Administración de Requerimientos	50
2.4. ARTEFACTOS PARA EVALUAR LA CALIDAD DE LOS REQUERIMIENTOS	55
2.4.1. Validación de Requerimientos	64
2.5. MODIFICACIONES A LOS REQUERIMIENTOS	65
2.5.1. Análisis de la Modificación	66
2.5.2. Análisis del Cambio en Costeo	66
2.5.3. Autorización del Acuerdo e Implementación del Cambio	66
2.5.4. Problemas por Ausencia de Administración de Cambios	67
REFERENCIAS BIBLIOGRÁFICAS	68
REFERENCIAS ELECTRÓNICAS	68
CAPÍTULO 3. INGENIERÍA INVERSA	69
3.1. INTRODUCCIÓN A LA INGENIERÍA INVERSA	70
3.2. INGENIERÍA INVERSA Y DISEÑO TRADICIONAL	71
3.3. PROCESO DE LA INGENIERÍA INVERSA	72
3.3.1. Análisis de la Funcionalidad	74
3.3.2. Lectura de Código Fuente	75
3.3.3. Restructuración del Código Fuente	75
3.3.4. Depuración del Código Fuente	76
3.3.5. Abstraer el Sistema	76
3.3.6. Documentación de Sistema (Requerimientos Obtenidos)	77
3.4. INGENIERÍA INVERSA Y REINGENIERÍA	78
3.4.1. Ingeniería Inversa como Etapa de Obtención de Requerimientos de la Reingeniería	79
3.4.2. Refinamiento y Simplificación de Requerimientos	79
3.4.3. Especificación Final de los Requerimientos	80
3.5. HERRAMIENTAS PARA LA INGENIERÍA INVERSA	80
3.5.1. Las Herramientas CASE	80
3.5.2. Los Depuradores	81
3.5.3. Las Herramientas de Inyección de Fallos	83
3.5.4. Los Depuradores	84
3.5.5. Los Compiladores Inversos o Decompiladores	86
REFERENCIAS BIBLIOGRÁFICAS	87

REFERENCIAS ELECTRÓNICAS	87
CAPITULO 4. REQUERIMIENTOS PARA INGENIERIA INVERSA	89
4.1. INTRODUCCIÓN A REQUERIMIENTOS PARA INGENIERÍA INVERSA	90
4.2. ABSTRACCIÓN DEL SISTEMA	93
4.2.1. Funcionalidad en los Niveles de Abstracción	94
4.3. MODELADO DE UN NUEVO SISTEMA	96
4.3.1. Obtención de Requerimientos	99
4.3.2. Evaluación de Calidad para Requerimientos	101
4.4. ANÁLISIS Y DOCUMENTACIÓN DE REQUERIMIENTOS	102
4.4.1. Requerimientos Funcionales	103
4.4.2. Requerimientos No Funcionales	104
4.4.3. Requerimientos Técnicos	104
4.5. VALIDACIÓN DE LOS REQUERIMIENTOS	105
4.5.1. Check List para Analista	105
4.5.2. Check List para Usuario	109
4.5.3. Check List para Diseño	111
4.6. RE ESTRUCTURACIÓN DEL NUEVO SISTEMA	115
REFERENCIAS BIBLIOGRÁFICAS	117
REFERENCIAS ELECTRÓNICAS	117
CAPITULO 5. SEGUIMIENTO DE LOS REQUERIMIENTOS	118
5.1. SEGUIMIENTO DE LOS REQUERIMIENTOS	119
5.2. CARACTERÍSTICAS DE LA TRAZABILIDAD DE LOS REQUERIMIENTOS	120
5.3. MATRIZ DE TRAZABILIDAD	123
REFERENCIAS BIBLIOGRÁFICAS	125
CONCLUSIÓN	126
ANEXO 1. DIAGRAMAS UML	131
ANEXO 2. ARTEFACTOS PARA EVALUACIÓN	138
GLOSARIO	143

INTRODUCCIÓN

PROBLEMÁTICA DE LOS REQUERIMIENTOS

Todo sistema surge a partir de un problema o una necesidad, incluso de una mejora. Este problema tiene características particulares que lo definen, a estas características se les conoce como requerimientos, que son descripciones específicas y detalladas de los elementos que componen un problema y pueden ser parte de la solución. Una necesidad, tiene muchas características y cada una de ellas debe estar cubierta para que el sistema sea útil. Es necesario tener un orden al construirlo, a este orden lo conocemos como ciclo de vida de un sistema.

Desde hace algunos años se han venido estudiando los requerimientos para el desarrollo de un sistema. Este estudio surge de la gran cantidad de proyectos fracasados o defectuosos. La mayoría de estos sistemas que no cubren los requisitos tienen defectos desde la etapa del análisis, dado que el costo que se paga por ello es muy alto se justifica la necesidad de un estudio minucioso de los requerimientos.

La principal etapa de cualquier proyecto en sistemas es el análisis, ya que en este se especifican los requerimientos. Estos requerimientos moldean el sistema a construir y nos permiten conocer exactamente las necesidades que debe cubrir y en algunos casos como se debe hacer. Aun en la actualidad la etapa del análisis es subestimada para la construcción de un nuevo sistema y en algunos casos es omitida para reducir los tiempos de desarrollo del mismo, sin medir el riesgo de calidad que esto ocasionaría.

Se diría que el análisis comienza desde la entrevista con el usuario para levantamiento de requerimientos, pero esto no siempre es cierto. En la mayoría de los casos el análisis se inicia desde la estimación de un proyecto, ya que es necesario conocer el impacto del sistema, necesidades, tiempos de desarrollo y pruebas.

Existen muchas maneras de realizar el análisis de un sistema pero la generalmente se apega al estándar UML, esto se debe a la fácil representación de su sintaxis y semántica. A pesar de ser la primera etapa de un sistema, el análisis se encuentra activo hasta la última etapa del ciclo de vida, esto es, se utilizan desde el levantamiento de los requerimientos, su análisis, especificación, validación y seguimiento. Cada parte del análisis debe ser lo más clara posible para que la siguiente etapa del ciclo sea más fluida.

De ahí que el objetivo de este trabajo fuera resaltar y sugerir una serie de artefactos para una planeación y ejecución efectiva de la administración de requerimientos que pueden ser obtenidos desde una ingeniería inversa, basándose en criterios específicos de calidad. No es lo mismo obtener los requerimientos de un usuario que abstraerlos de un sistema ya existente, documentarlos y obtener una validación de los mismos.

Para este trabajo se estudiarán los ciclos de vida orientados a objetos y las disciplinas de Ingeniería de Requerimientos, Ingeniería Inversa y también se estudiarán los criterios de calidad para los sistemas de información y para los requerimientos. Todo esto tiene por objetivo obtener un método para administrar los requerimientos, de un sistema ya existente, cuidando la calidad de los mismos y proponer algunos artefactos que faciliten la validación y seguimiento de cada requerimiento.

Este estudio se basó en la Ingeniería de requerimientos e Ingeniería inversa usando UML. Debemos resaltar que nuestro objetivo principal serán los requerimientos, por lo que sólo se cubrirá a detalle la etapa del análisis; con la finalidad de abstraer los requerimientos desde un sistema ya existente mediante una ingeniería inversa y se marcará un seguimiento de los mismos en las siguientes etapas del ciclo de vida, sin intervenir o adentrar en otras etapas del ciclo de vida.

CAPITULO 1. CICLOS DE VIDA DE LOS SISTEMAS ORIENTADOS A OBJETOS

Existen diferentes formas de elaborar un sistema, sin embargo se deben tener bien identificadas las etapas por las que debe pasar y un orden de flujo.

En este capítulo se mencionarán las etapas fundamentales para un ciclo de vida de sistemas de software, definidos en el Rational Unified Process (RUP) y se describirán en un marco general las principales características de distintos modelos de desarrollo para sistemas de software. Al final se hablara del enfoque que se puede dar a cada uno ya sea de forma estructurada u orientada a objetos basándose en las etapas que lo componen.

1.1. INTRODUCCIÓN A LOS CICLOS DE VIDA

Desde un punto de vista de calidad, todo proceso debe tener un método de desarrollo ordenado y explícito para su desarrollo o ejecución, para los sistemas de información esto no es excepción.

Alrededor de la década de los setenta, los sistemas experimentaron un gran crecimiento, tanto en el desarrollo como en complejidad. Los sistemas habían sido desarrollados, hasta entonces, por pasos abstractos que se encontraban dentro de la cabeza de cada desarrollador, esto es, no existía un análisis formal de la complejidad del sistema a construir. Entonces el **software** desarrollado era para un uso específico y sólo eran modificaciones de acuerdo a sus necesidades; trabajando bajo la técnica corregir-modificar (Cantone, 2008).

El crecimiento de las empresas e industrias trajo una serie de modificaciones constantes en los sistemas de software que usaban hasta el momento para adaptarse a sus nuevas necesidades. Este crecimiento en las empresas originaba muchas modificaciones y en algunas situaciones la reconstrucción total del sistema mismo. Al ser detectados estos constantes cambios se inició una competencia entre las empresas desarrolladoras de software para satisfacer las necesidades del cliente con la menor cantidad de errores y buscando una flexibilidad del sistema para futuras adaptaciones.

En esta época el desarrollo del software no tenía metodologías bien definidas y las formas de desarrollo tenían muchos problemas. Algunos de ellos eran desarrollar un software sin defectos, de un fácil mantenimiento y que pudiera ser verificable. Si había un cambio o crecimiento en el negocio de la empresa, era muy complicado actualizar el sistema para cubrir esa necesidad, de ahí que era necesario reestructurar por completo el sistema.

Es bien sabido por los programadores que codificar un software es un proceso complejo ya que se debe abstraer la necesidad. Una vez que un programador terminaba el código, debía ser precisamente él quien diera el mantenimiento, de lo contrario el siguiente programador tenía que “entender” lo desarrollado por otro y adaptarlo a la nueva necesidad del usuario.

Todos estos conflictos en el desarrollo del software fueron conocidos como “crisis del software”. Las principales características de la problemática fueron:

⇒ Software no fiable

- ⇒ No se satisfacían las necesidades del cliente
- ⇒ Mantenimiento permanente además de complicado y estático
- ⇒ Entrega del producto con retrasos
- ⇒ Costos superiores a los estimados

Es precisamente a partir de aquí que se introduce el término “ingeniería de software”, a pesar de ser adjudicado a F.L. Bauer en la primera conferencia de desarrollo de software (OTAN 1968), había sido utilizado y estudiado previamente por Edsger Dijkstra.

A partir del planteamiento de esta problemática, las empresas proveedoras de software se enfocaron en la satisfacción total del cliente mediante la búsqueda y desarrollo de metodologías que solventaran estos puntos y con ello se inicia una competencia por la búsqueda de calidad en el desarrollo del software. Esta competencia entre las empresas desembocó en metodologías que guiaban paso a paso el desarrollo de un sistema de información. Es entonces cuando surge el concepto “ciclo de vida de los sistemas”.

La ISO (International Organization for Standardization) define en la norma 12207 al ciclo de vida de software como “un marco de referencia que contiene las actividades y las tareas involucradas en el desarrollo, explotación y mantenimiento de un producto software abarcando desde la definición hasta la finalización de su uso” (Weitzenfeld, 2005).

Actualmente, los desarrollos de software se originan en el hecho de que es muy costoso rectificar los **errores** que se detectan tarde, dentro de las fases de implementación o pruebas. El uso de un ciclo de vida permite que los errores se detecten lo antes posible y permite reducir los costos de la rectificación, dependiendo de la etapa en que son localizados.

De acuerdo al supuesto planteado por Pressman (1993) un error descubierto antes o durante el diseño, cuesta 1 unidad monetaria del costo total del proyecto, siguiendo este mismo costo, el mismo error descubierto en la etapa de desarrollo cuesta corregirlo 6.5 unidades, en la etapa de pruebas, tiene un costo de 15 unidades y corregirlo una vez entregado, cuesta entre 60 y 100 unidades. Lo que nos daría una gráfica parecida a la siguiente.

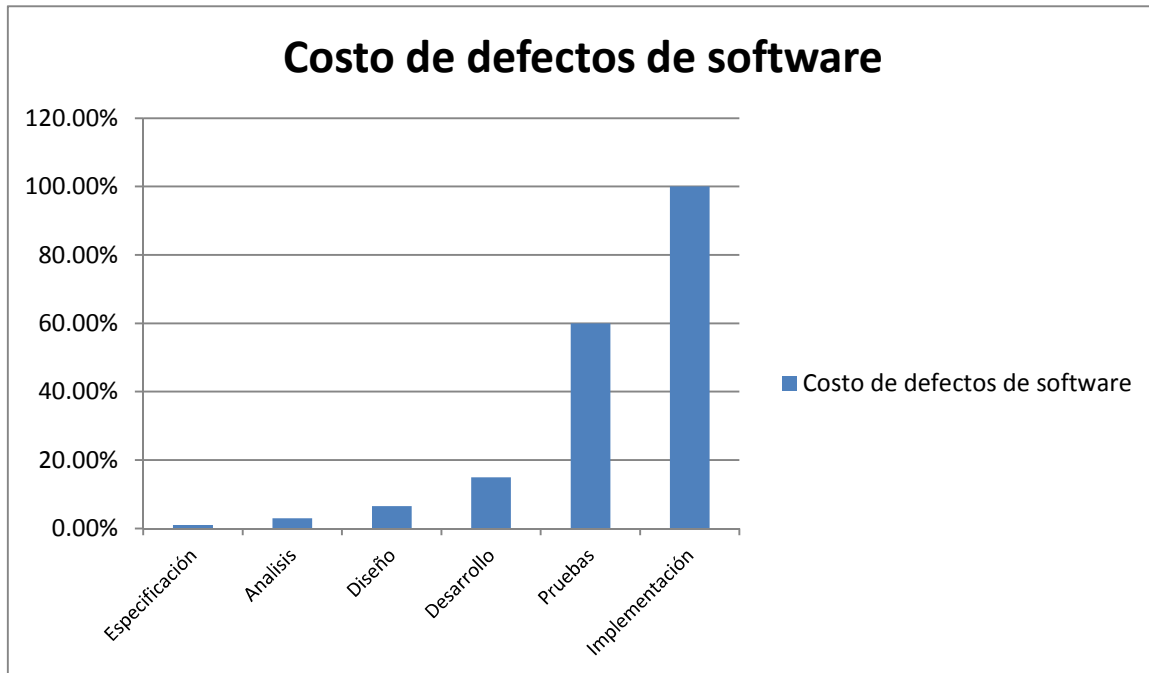


Figura 1. Estimación de costo de defectos

El término ciclo de vida del software describe el desarrollo de software desde la fase inicial hasta la final. El propósito de este ciclo es definir las distintas fases intermedias que se requieren para **validar** el desarrollo de la aplicación, es decir, para garantizar que el software cumpla con los requisitos para la aplicación y verificación de los procedimientos de desarrollo con el objetivo de asegurar que los métodos utilizados son apropiados (Cantone, 2008).

En un marco general y basándose en distintos modelos, un ciclo de vida clásico consta de las siguientes etapas:

- ⇒ **Especificación.**- Se obtienen las necesidades puntuales del cliente.
- ⇒ **Análisis.**- De las necesidades especificadas por el cliente se procede a unas identificaciones de problemas, oportunidades y objetivos; se analizan las necesidades del sistema mismo a desarrollar y se delimitan los requerimientos y el alcance del sistema.
- ⇒ **Diseño.**- En esta etapa se diseñan procedimientos precisos con la información obtenida de la etapa de análisis con el fin de que los datos que se introducen al sistema sean los correctos. Esta etapa también incluye el diseño de las bases de datos según lo requerido y las salidas que el cliente obtendrá de estas, según sus necesidades de información.
- ⇒ **Desarrollo.**- Es aquí donde la etapa del análisis y el diseño desbordan sus resultados para facilitar el trabajo de los programadores. Todo lo anterior sirve

de guía para que los programadores conozcan las necesidades de manera abstracta y un método de desarrollo dependiendo del diseño y la arquitectura.

- ⇒ **Pruebas.**- La finalidad de una etapa de pruebas es depurar todos los posibles defectos del sistema antes de llegar a manos del cliente, validar que las necesidades originales del sistema se estén cubriendo. Se verifica que el sistema este funcionando correctamente y que cumpla con los requerimientos del usuario.
- ⇒ **Implementación.**- Esta etapa consiste en realizar la transición de la forma de trabajo actual a la nueva forma de trabajo, siempre tratando de mostrar las ventajas y facilidades que proporciona este nuevo sistema. Esta etapa puede constituir en una capacitación para el cliente o el nuevo usuario del sistema.
- ⇒ **Mantenimiento.**- Es la última etapa del ciclo de vida de los sistemas, consiste en realizar todas las actividades necesarias a fin de mantener el sistema trabajando adecuadamente y se anticipa a los posibles cambios que puedan surgir un con un crecimiento del negocio, respetando los niveles de calidad establecidos.

Existen distintos modelos el ciclo de vida o lo que es lo mismo distintas pautas a seguir en el desarrollo de los sistemas de información, tanto si se tratará de un proyecto sencillo en una organización pequeña como en organización grandes o para desarrollo de proyectos complejos es necesario establecer un modelo según las necesidades del sistema a desarrollar. A continuación se mencionarán algunos modelos en aspectos generales.

1.1.1. Ciclo de Vida Lineal

Este modelo se considera el más sencillo y **propuesto por Pressman** (1993), consiste en ordenar las etapas en forma secuencial, es decir, cada etapa se realiza sólo una vez en forma ordenada, no permite retroalimentación entre etapas a modo que la división de tareas permite prever los tiempos para cada una de ellas y obtener una estimación total en el tiempo de desarrollo.

Las etapas que conforman este modelo son 6, análisis, diseño, desarrollo, pruebas, implementación y mantenimiento. El flujo que siguen estas etapas se muestra en la Figura 2, donde el resultado obtenido de una etapa corresponde a la entrada de información para la siguiente, según el orden mostrado.

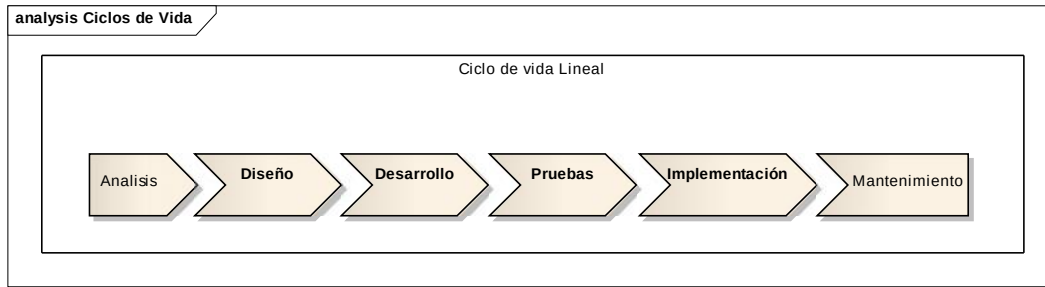


Figura 2. Diagrama del ciclo de vida lineal

Este modelo es muy usado con todos los requerimientos bien establecidos, desde la primera etapa. Desafortunadamente si alguna etapa de este ciclo es mal calculada, la afectación será directamente a las siguientes y cada una con un incremento ligeramente mayor al anterior. Otra de sus desventajas es: no es apto para desarrollos que superen los requerimientos de retroalimentación ya que es muy costoso retomar una etapa anterior al detectar una falla, y que los tiempos fueron previamente calculados.

Se destaca como ventaja la sencillez de su administración, tanto económica como temporal, ya que se acomoda a proyectos internos de una empresa para sistemas pequeños que realicen altas, bajas, cambios y consultas sobre un conjunto de datos (Fontela, 2011).

1.1.2. Ciclo de Vida en Cascada Simple

Este modelo fue propuesto por Winston Royce en 1970 en su obra "Managing the Development of Large Software Systems" (Cantone, 2008).

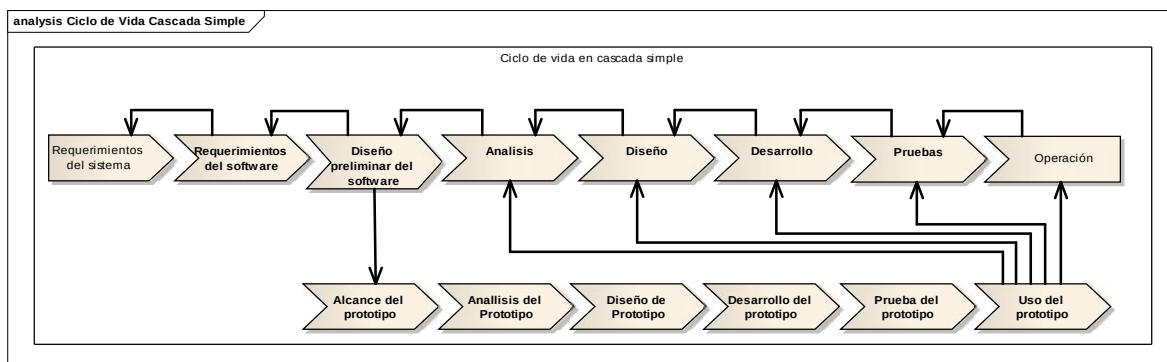


Figura 3. Diagrama del ciclo de vida en cascada simple con sub ciclo para prototipo

Este modelo, permite realizar retroalimentaciones hacia etapas anteriores, por ejemplo, si se encuentra algún defecto del análisis en la etapa del diseño, es posible hacer la retroalimentación y regresar a corregir ese error. Las etapas que forman este modelo son: Requerimientos del sistema (levantamiento de requerimientos del sistema), requerimientos del software (obtención de requerimientos del software), diseño preliminar del software (diseño de prototipos), análisis, diseño, desarrollo, pruebas y operación. Ver Figura 3.

En el caso de usar prototipos, éste se vuelve necesario como guía para cada una de las siguientes etapas. El prototipo es un buen apoyo para validar con el cliente si han sido entendidas de forma clara sus necesidades. Los prototipos para este modelo también cuentan con un sub ciclo, lo que permite minimizar los posibles defectos y clarificar las necesidades a modo de etapas lineales, es decir, para los prototipos, la salida de una etapa es la entrada de otra, sin permitir una retroalimentación. Las etapas del sub ciclo del prototipo son: Alcance del prototipo, análisis del prototipo, diseño del prototipo, desarrollo del prototipo, pruebas del prototipo y uso del prototipo.

Este ciclo de vida es adecuado para proyectos en los que se cuenta con todos los requerimientos desde el primer momento, es decir, para aquellos proyectos que cuenten con una funcionalidad conocida como las reingenierías ó un proyecto similar a uno ya existente.

Algunas ventajas que presenta este modelo es la planificación sencilla, la calidad del producto será mayor ya que permite una validación etapa por etapa. Si se tienen todos los requerimientos claramente especificados desde el comienzo permite trabajar con mayor precisión, por lo tanto, se puede trabajar con poco personal, desafortunadamente muy pocas veces se cuenta con esta condición.

A pesar de que es un modelo muy bien estructurado es poco el uso en proyectos actuales ya que el cliente pocas veces mantiene las especificaciones iniciales. Si se comete un error en la fase de análisis, no lo descubrimos hasta la entrega, por lo que se generó un gasto inútil de recursos y no permite una visión del sistema si no hasta que se llega al final del ciclo, por ello el cliente verá el resultado sólo hasta llegar a la última etapa del ciclo. No existen indicadores fiables del progreso del trabajo y es comparativamente más lento que otros y el coste es mayor (Cantone, 2008).

1.1.3. Ciclo de Vida “V”

Diseñado por Alan Davis a principios del siglo XX (Cantone, 2008). Tiene las mismas etapas que el ciclo de vida en cascada simple, con la diferencia de tener dos sub-etapas de retroalimentación, la primera entre el análisis y el mantenimiento y la segunda entre el diseño y las pruebas.

El lado izquierdo de la V representa la descomposición de las necesidades, y la creación de las especificaciones del sistema. El lado derecho de la V representa la integración de las piezas y su verificación. V significa «verificación y validación» (Forsberg, 2005). Ver Figura 4.

Este ciclo de vida se puede utilizar en aplicaciones simples. Requiere de una confiabilidad muy alta por lo tanto no se permiten errores. El error en una de sus etapas, puede tener muchos más defectos en sus niveles descendentes y no olvidemos que puede ser difícil que el cliente exponga claramente todos los requisitos. El cliente debe tener paciencia pues obtendrá el producto al final de todo el ciclo de vida. Las pruebas pueden ser caras debido a que se realizan en cada etapa, estas requieren tiempo y recursos. En ocasiones no son suficientemente efectivas y el producto final obtenido puede que no cumpla con todos los requisitos del usuario.

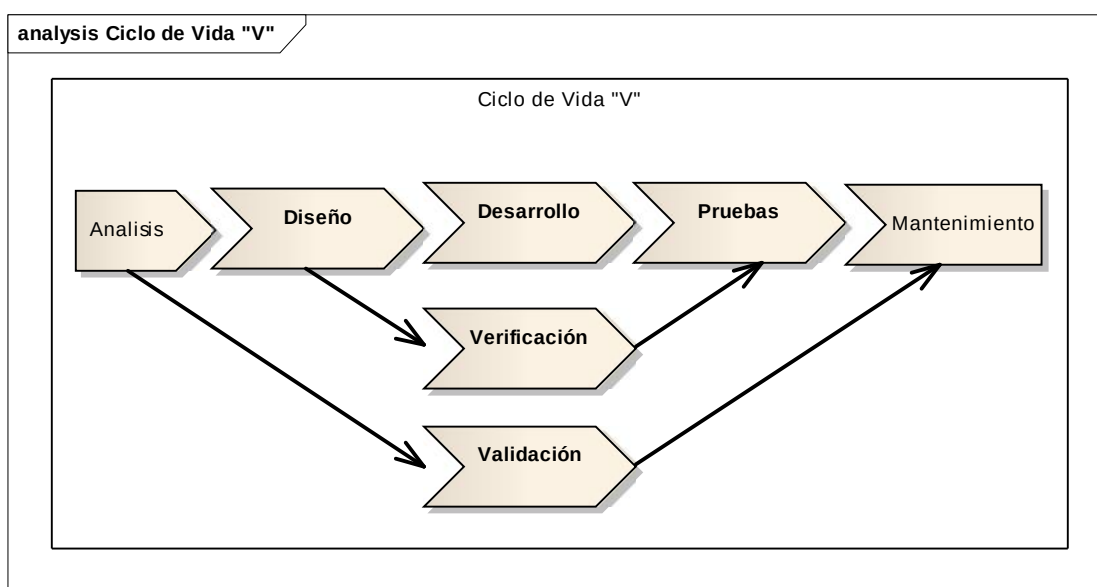


Figura 4. Diagrama del ciclo de vida “V”

La corriente de especificación (la parte izquierda) consiste principalmente de:

- ⇒ Conceptos de operaciones: qué debe hacer el sistema a grandes rasgos.
- ⇒ Requisitos del sistema y arquitectura del mismo.
- ⇒ Diseño detallado.

La corriente de pruebas (la parte derecha) por su parte, suele consistir de:

- ⇒ Integración de las distintas partes, test y verificación de las mismas.
- ⇒ Verificación y validación del sistema en conjunto.
- ⇒ Mantenimiento del sistema.

La corriente de desarrollo puede consistir en personalización, configuración o codificación.

1.1.4. Ciclo de vida Sashimi

El modelo Sashimi fue creado originalmente por Peter DeGrace (Borja, 2011). Su nombre se deriva del modelo japonés para presentar el pescado crudo en rodajas. La diferencia con otros modelos consiste básicamente en que sashimi permite que las etapas del ciclo de vida sean solapadas, es decir, aumentan la eficiencia mediante la retroalimentación de las etapas (Cantone, 2008).

Las etapas de este ciclo son: Obtención de requerimientos, análisis, diseño arquitectónico, diseño detallado, desarrollo, pruebas, implementación y mantenimiento. A pesar que estas etapas se encuentran solapadas, permiten una retroalimentación hacia etapas anteriores, para poder corregir defectos. Ver Figura 5.

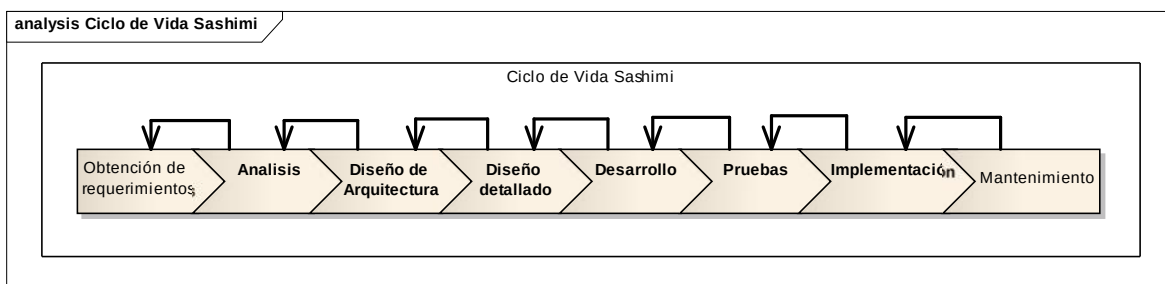


Figura 5. Diagrama del ciclo de vida Sashimi

Sashimi tiene como ventaja una ganancia de calidad en el producto final, además tiene la ventaja de no requerir una documentación detallada para cada etapa. La documentación es más sencilla ya que es compartida en las etapas.

Este modelo es muy similar al modelo en cascada puro, así que también depende que sus objetivos o requerimientos estén bien definidos antes de continuar con el flujo. Entre sus desventajas se encuentran con que es muy difícil administrar el comienzo y el fin de cada etapa; si existe algún problema, genera demasiadas inconsistencias en el proyecto para corregir el error debido a que las etapas están entrelazadas.

1.1.5. Ciclo de Vida en Cascada con Sub Proyectos

Es una variación del ciclo de vida en cascada. Se diferencian porque una vez que se ha llegado la etapa del diseño (arquitectura), el sistema se divide en varios sub sistemas independientes entre si, con la finalidad de simplificar la complejidad del sistema. Una vez dividido, es mas sencillo su desarrollo y posteriormente requiere una etapa de integración donde cada uno de los sub-sistemas se conectan entre si para obtener un sistema total.

Este modelo permite realizar pruebas por sub-sistemas y detectar defectos de una manera más sencilla. Sin embargo, los desarrolladores deben tener bien definidos los estándares de comunicación entre los sub-sistemas. Un descuido en este aspecto parece que el sistema total esta mal cuando sólo es un error de comunicación, otro problema es la pues tiene que ser mucho más detallada, aunque su principal ventaja es que el desarrollo resulta más ágil.

En la Figura 6 se muestra el flujo de las etapas en este modelo. Se inicia con un análisis general del sistema, un diseño general; es aquí donde se divide el sistema en sub sistemas, posterior a esto se inicia un nuevo sub ciclo para cada sub sistema. Este sub ciclo tiene un análisis de sub sistema, diseño de sub sistema, desarrollo y pruebas del sub sistema. Una vez concluidas las pruebas por sub sistemas continúa el flujo del ciclo de vida del sistema original, con una etapa de integración, donde se une el desarrollo realizado y los siguientes pasos son pruebas del sistema ya integrado y mantenimiento, respectivamente.

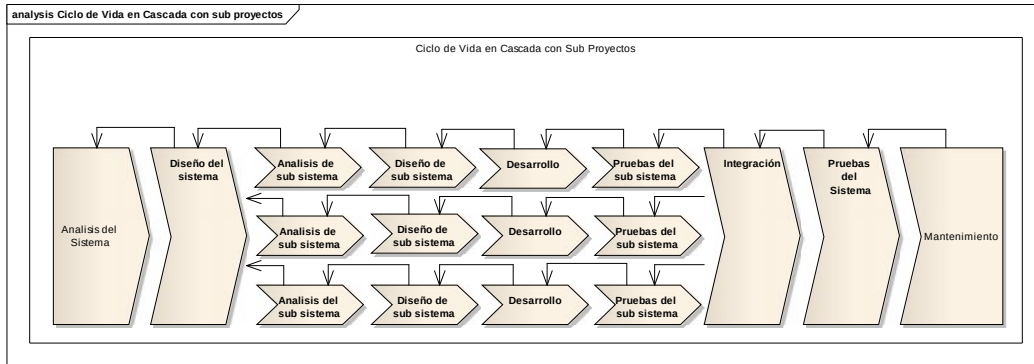


Figura 6. Diagrama del ciclo de vida en cascada con sub proyectos

1.1.6. Ciclo de Vida Iterativo

Este ciclo tiene como principal ventaja el reducir el riesgo entre la obtención de requisitos y el producto final, por malos entendidos en el levantamiento de requerimientos. Como su nombre lo indica, itera los ciclos de vida en cascada con las mismas etapas de dicho modelo y permite una entrega al cliente cada vez mas completa en cada versión. Por cada iteración entregada, el cliente evalúa, detecta y propone alguna mejora, esto hasta que las necesidades del cliente queden cubiertas en su totalidad. Ver Figura 7.

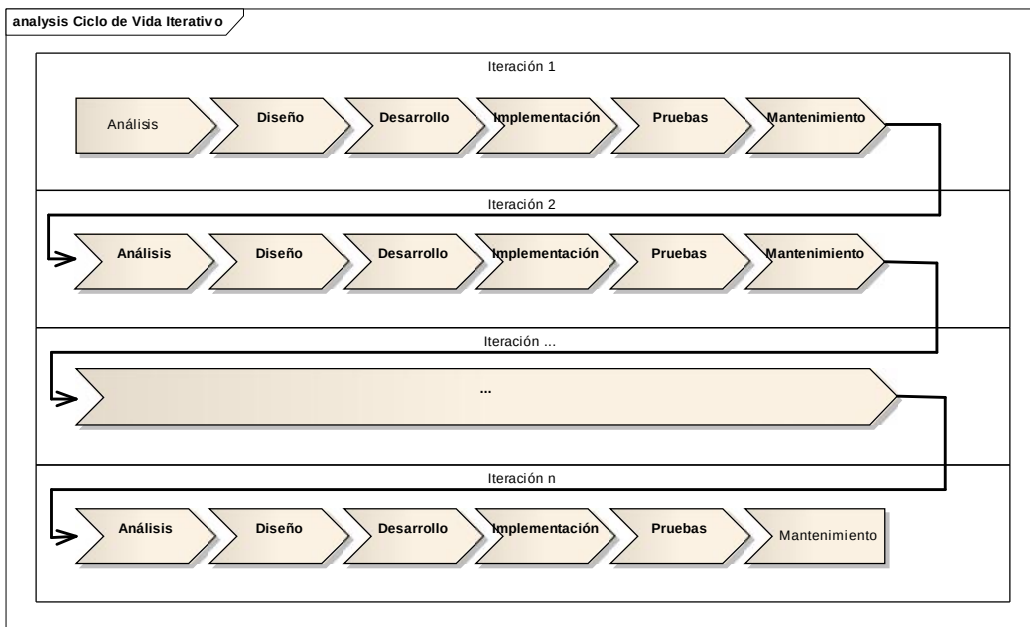


Figura 71. Diagrama del ciclo de vida iterativo (iteraciones del modelo en cascada)

Se utiliza principalmente en proyectos en los que los requerimientos aun no están definidos o son confusos por parte del cliente, este ciclo permite mejoras en cada una de las iteraciones. Para este ciclo, la dimensión del proyecto no importa ya que las iteraciones hacen flexible el desarrollo del sistema.

1.1.7. Ciclo de Vida por Prototipos

Los prototipos son muy importantes para que el cliente pueda visualizar los requerimientos solicitados el sistema y vea reflejada su necesidad en algo más concreto. En algunas empresas, para realizar un convenio de los requerimientos es solicitado un prototipo, con la finalidad de ver como serán atendidas sus necesidades.

Es muy común que la obtención de los requerimientos pueda ser complicada, ya sea porque el cliente desconozca lo que quiere, tiene dificultad para expresar sus necesidades o por falta de experiencia para obtener la información por parte del consultor. Es aquí donde este ciclo es el más útil. Suele recurrirse a un apoyo gráfico donde se expone al cliente las funcionalidades que puede proporcionar el producto final.

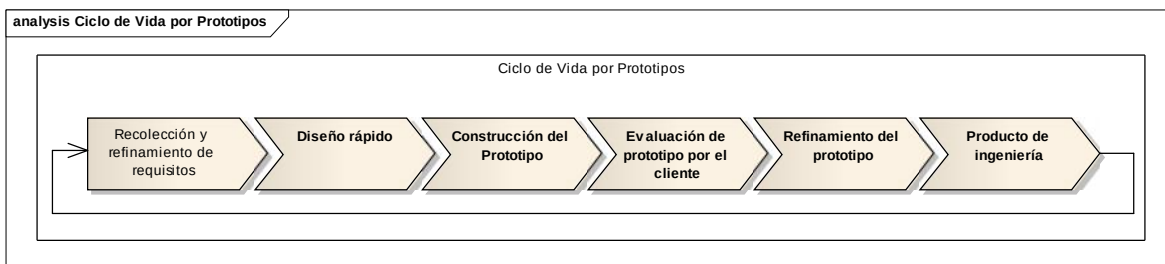


Figura 8. Diagrama del ciclo de vida por prototipo

A diferencia de otros modelos, el modelo por prototipos tiene las siguientes etapas: Recolección y refinamiento de requisitos, diseño rápido, construcción del prototipo, evaluación del prototipo por el cliente, refinamiento del prototipo y producto de ingeniería. Son etapas secuenciales que pueden iterarse para el desarrollo de un sistema completo. Ver Figura 8.

Cantone (2008) refiere las ventajas de este ciclo, basadas en que es el único apto para desarrollos en los que no se conoce a priori sus especificaciones ó la tecnología a utilizar, como contrapartida por este desconocimiento se tiene la desventaja de ser altamente costoso y difícil para la administración temporal.

Este estilo fue promovido en 1985 por DuPont Co. (inventores del nylon y la lycra) mediante una metodología propia conocida como producción iterativa de prototipos. James Martin realizó un libro posterior definiendo una metodología similar e introduciendo el término RAD (Rapid application development, por sus siglas en inglés) (Cantone, 2008).

Este ciclo se utiliza de forma más común para sistemas innovadores, dentro o fuera de la empresa o en investigaciones de tecnologías. Su principal ventaja es que sirve como apoyo para la etapa de desarrollo del sistema.

1.1.8. Ciclo de Vida Evolutivo

Afortunadamente para el cliente en este ciclo de vida los requerimientos pueden cambiar en cualquier momento ya que consta de iteraciones constantes de ciclos, este ciclo es aplicado cuando el proyecto es muy grande y no se tienen todos los requisitos especificados desde el comienzo de proyecto. En este caso es posible que conforme se obtenga la evaluación de los clientes, se obtengan nuevos requisitos.

Este método se lleva a cabo mediante el requisito, desarrollo y evaluación. De esta manera un resultado nos lleva a una siguiente versión hasta llegar a una satisfacción del cliente, versión a versión. Ver Figura 9.

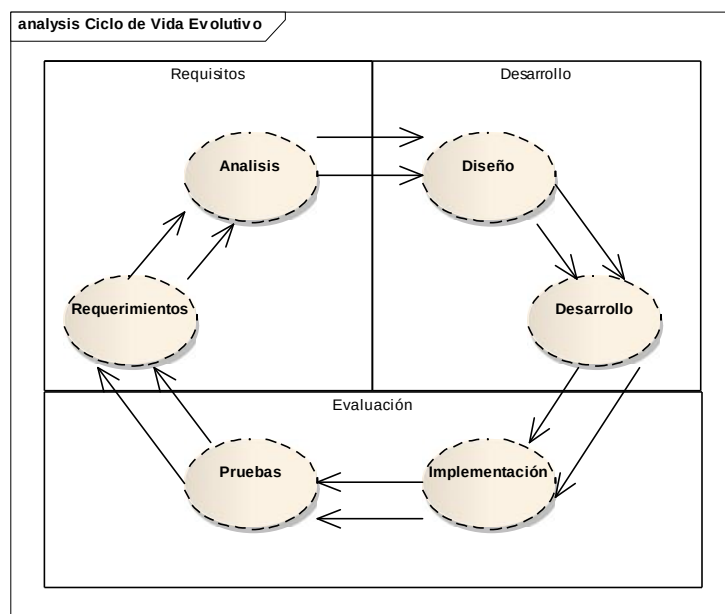


Figura 9. Diagrama del ciclo de vida Evolutivo

1.1.9. Ciclo de Vida Incremental

Este modelo se adjudica a Lehman en 1984 (Guerra & Mousalli, 2009). El objetivo de este modelo es el desarrollo modular del sistema, de tal forma que se incrementa poco a poco la funcionalidad que existe en el sistema. Consiste en tener un ciclo de vida por cada funcionalidad. Esto permite entregas al cliente por versiones y agregar funcionalidad al sistema en cada versión, de tal forma que en cada entrega el sistema se va completando. Ver Figura 10.

Algunas de las ventajas de este ciclo es que al dividir en módulos se reduce la probabilidad de defectos, al hacer un desarrollo independiente de las funcionalidades, se pueden permutar los requerimientos del cliente, los defectos encontrados en el sistema se pueden depurar en la siguiente iteración y nos permite satisfacer al cliente si necesita entregas rápidas aunque sean parciales aun cuando no se dispone de los requerimientos de todas las funcionalidades al inicio del proyecto (Witzenfeld, 2005).

Construir un sistema pequeño siempre es menos riesgoso que construir un sistema grande. Como el desarrollo es independiente a las funcionalidades es más fácil aclarar los requerimientos del usuario. Si se detecta un error grave, sólo se puede depurar en la siguiente iteración. No es necesario disponer de los requerimientos de todas las funcionalidades en el comienzo del proyecto y además facilita la labor del desarrollo con la conocida filosofía “divide y vencerás” (Cantone, 2008).

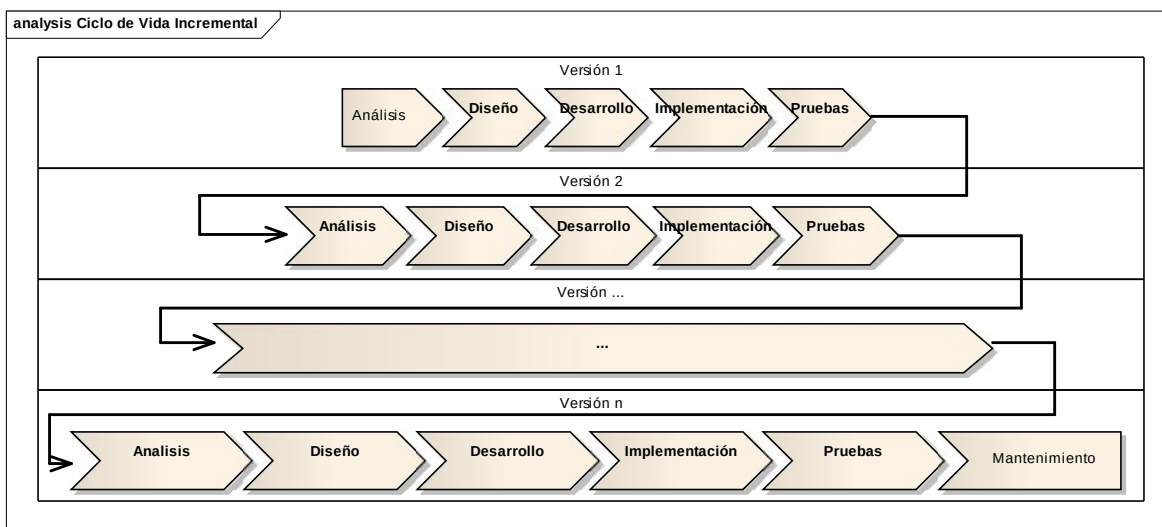


Figura 10. Diagrama del Ciclo de vida incremental

1.1.10. Ciclo de Vida en Espiral

Este modelo es muy parecido al ciclo de vida por prototipos, algunos autores lo consideran una variación de él. Diseñado a finales de la década de los 80's por Barry Boehm (Weitzenfeld, 2005), se basa en una serie de ciclos repetitivos para ir ganando madurez en el producto final, mediante el desarrollo o evolución de los prototipos.¹⁹ y tiene la característica de ser incremental, pero se toman más en cuenta los riesgos por desconocer los requerimientos.

El ciclo de vida en espiral permite obtener prototipos que cumplan paulatinamente con lo requerido por el cliente. A diferencia de otros, este modelo consta de cuatro etapas para su desarrollo.

- ⇒ **Planificación.**- Obtención de requerimientos iniciales y posteriormente por iteración.
- ⇒ **Análisis.**- En esta etapa se analiza el riesgo de acuerdo a los requerimientos obtenidos y se evalúa si se continúa con el desarrollo.
- ⇒ **Implementación.**- Se desarrolla un prototipo y si se tiene comentarios se incluyen en el.
- ⇒ **Evaluación.**- El cliente evalúa el prototipo y si tiene comentarios se incluyen en la siguiente iteración; si se da por bueno el prototipo se termina el proyecto.

Aunque este modelo tiene la ventaja de realizar un desarrollo basándose en requerimientos no definidos, el costo del proyecto puede llegar a ser demasiado por cada vez que se recorre la espiral ó existe una nueva iteración.²⁰

1.1.11. Ciclo de Vida Orientado a Objetos

Este modelo fue presentado formalmente en la década de los 90's por Piattini (Guerra & Mousalli, 2009), sin embargo, los pioneros de este modelo fueron pioneros de los ciclos de vida que son Rumbaugh, Booch y Jacobson. Se basa en la filosofía de los objetos, donde cada requerimiento es considerado como un objeto cuyas propiedades son atributos y su comportamiento son métodos. Este método usa fichas de clase-responsabilidad-colaboración para facilitar la especificación de casos de uso (Henao, Valencia & Ortiz, 2010).

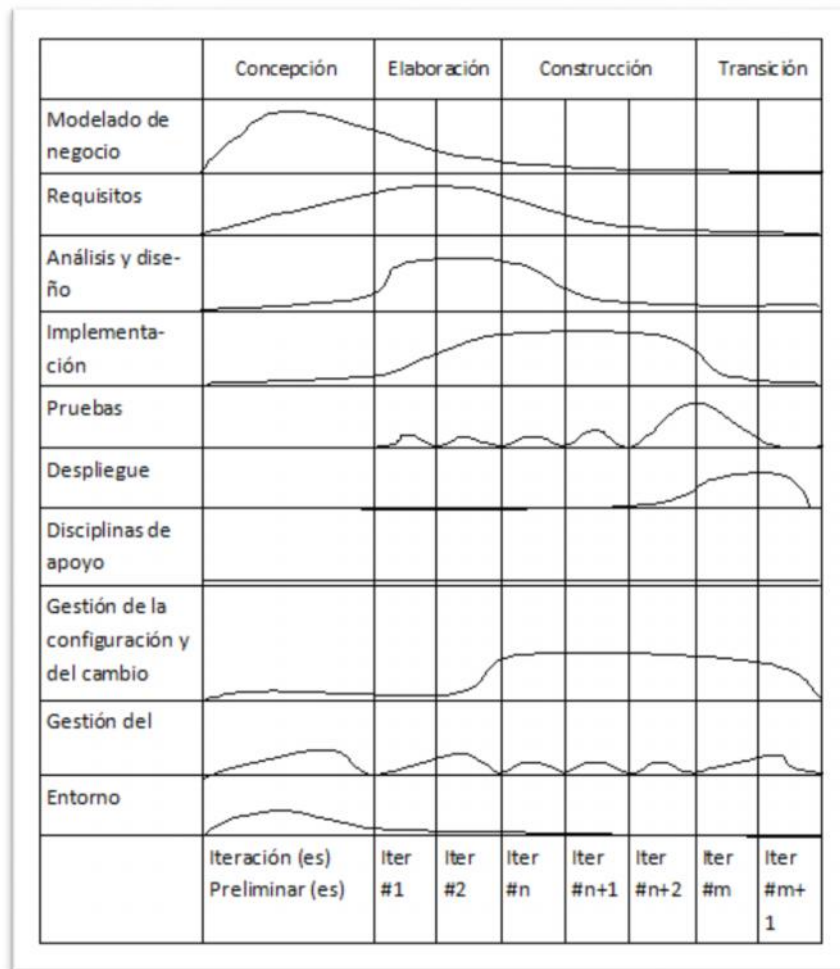


Figura 11. Ciclo de vida del desarrollo de software propuesto por Booch, Rumbaugh y Jacobson

En la Figura 10 (Booch, Rumbaugh & Jacobson, 2005), en la parte superior se enuncian las fases, las cuales son:

- ⇒ Concepción: Establece la visión del sistema y delimita el alcance del proyecto. Esto incluye la obtención del negocio, requisitos de alto nivel y plan inicial de proyecto. El Plan inicial incluye: criterios de éxito, evaluación del riesgo, estimaciones de recursos y un plan de fases que muestre la planificación de los hitos principales.
- ⇒ Elaboración: Analiza el dominio del problema, establece una base arquitectónica, desarrolla el plan del proyecto y elimina los elementos de mas alto riesgo. Las decisiones arquitectónicas deben tomarse con una comprensión total del sistema. Implica describir la mayoría de los requisitos del sistema. Para demostrar la arquitectura, se deben ejecutar los casos de uso más significativos.

- ⇒ Construcción. Aquí se desarrolla de forma iterativa e incremental un producto completo que esta preparado para pasar a la comunidad de usuarios. Esto implica describir los requisitos restantes y los criterios de aceptación, refinando el diseño y completando la implementación para las pruebas de software.
- ⇒ Se involucra al arquitecto del sistema, el gestor del proyecto y a los jefes del equipo de construcción, así como a todo el personal de desarrollo y pruebas.
- ⇒ Al final de esta fase, se decide si, el software, lugares de instalación y usuarios están preparados para instalar la primera versión funcional del sistema.
- ⇒ Transición. Durante esta fase, el software se despliega en la comunidad de usuarios. Hay que tener en cuenta que hemos estado involucrados con los usuarios a través de todo el proyecto mediante demostraciones y validaciones preliminares. Una vez que el sistema esta en manos de los usuarios finales, a menudo aparecen cuestiones que requieren un desarrollo adicional para ajustar el sistema, corregir algunos problemas no detectados o finalizar algunas características que habían sido propuestas. Esta fase comienza normalmente con la versión beta del producto. Para esta fase se encuentran el gestor del proyecto, los probadores, el personal de ventas y comercialización. Sin descuidar las fronteras de la negociación.

Las iteraciones que se muestran en la parte inferior de la misma Figura, son un ciclo completo de desarrollo que produce una versión (interna o externa) de un producto ejecutable, que constituye un subconjunto del producto final en desarrollo, que ira incrementando de iteración en iteración hasta convertirse en el sistema final. Cada iteración pasa por diferentes disciplinas aunque con diferente énfasis en cada una de ellas, dependiendo de la fase.

Las disciplinas se muestran en la columna izquierda, tienen un conjunto de artefactos y actividades relacionados. Las disciplinas de este ciclo son:

- ⇒ Modelado del negocio: Describe la estructura y la dinámica de la organización del cliente.
- ⇒ Requisitos. Extrae los requisitos utilizando diferentes métodos.
- ⇒ Análisis y diseño. Describe las diferentes vistas arquitectónicas.
- ⇒ Implementación. Tiene en cuenta el desarrollo de software, las pruebas unitarias y la integración.

- ⇒ Pruebas. Describe los scripts, la ejecución de pruebas y las métricas para rastreo de defectos.
- ⇒ Despliegue. Incluye la facturación de los materiales, notas de edición, formación y otros aspectos relacionados con la entrega de la aplicación.
- ⇒ Gestión de conFiguraciones. Controla los cambios y mantiene la integridad de los artefactos de un proyecto y de las actividades de gestión.
- ⇒ Gestión de proyecto. Describe varias estrategias de trabajo en un proceso iterativo.
- ⇒ Entorno. Cubre la infraestructura necesaria para desarrollar un sistema.

Un artefacto es algún documento, informe o ejecutable que se produce, manipula o consume. Algunos artefactos se muestran en la siguiente tabla son:

Tabla 1. Artefactos utilizados para el ciclo de vida del software

Artefactos Proceso Unificado de Rational		
Modelos del Proceso unificado de Rational	Modelo de Casos de uso del negocio.	Establece una abstracción de la organización.
	Modelo de análisis del negocio.	Establece el contexto del sistema.
	Modelo de casos de uso	Establece los requisitos funcionales.
	Modelo de Análisis (opcional).	Establece un diseño conceptual.
	Modelo de diseño	Establece el vocabulario del problema y su solución.
	Modelo de datos (opcional).	Establece la representación de los datos para las bases de datos y otros repositorios.
	Modelo de despliegue.	Establece la topología hardware sobre la cual se ejecutará el sistema, además de los mecanismos de sincronización y concurrencia.
Artefactos técnicos del Proceso Unificado de Rational	Conjunto de requisitos.	Describe qué debe hacer el sistema.
	Conjunto de análisis y diseño.	Describe como se va a construir el sistema.
	Conjunto de pruebas.	Describe el método por el que se validará y se verificará el sistema.
	Conjunto de implementación.	Describe el ensamblado de los componentes software desarrollado.
	Conjunto de despliegue.	Proporciona todos los datos para la configuración entregable.

Una actividad describe las tareas (pasos de concepción, realización y revisión) que llevan a cabo los trabajadores para crear o modificar los artefactos, junto con las técnicas y guías para ejecutar las tareas, incluyendo posiblemente el uso de herramientas para ayudar a automatizar algunas de ellas.

Este modelo tiene sus fundamentos en la abstracción de los requerimientos, obtener las características esenciales de los requerimientos para tratarlos como objetos. Normalmente en este modelo se utiliza las fichas CRC (clase – responsabilidades – colaboración) para facilitar la especificación de los casos de uso.

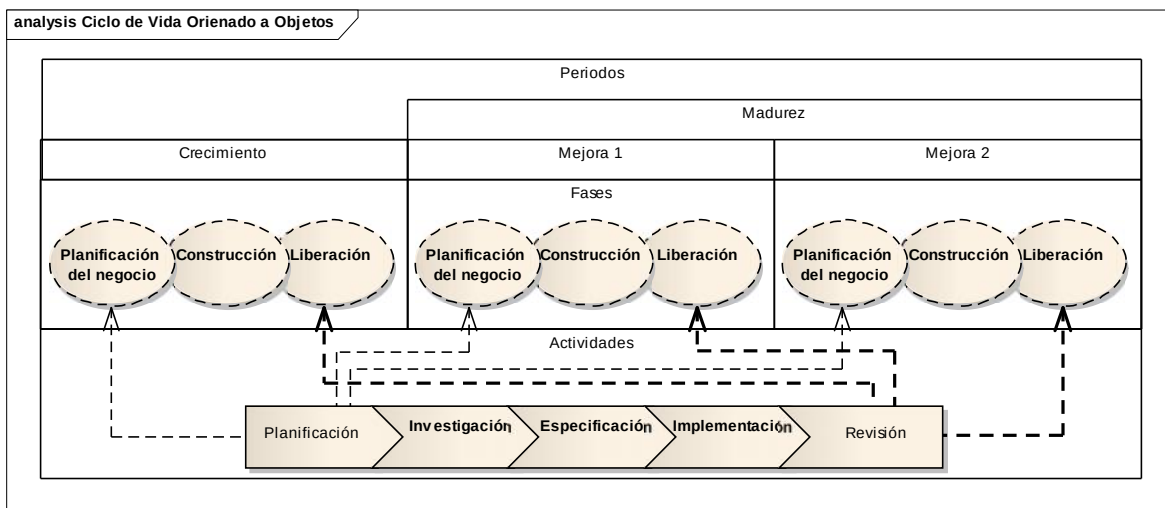


Figura 11. Diagrama del ciclo de vida orientado a objetos

Existen algunos Modelo de Desarrollo de Sistemas orientados a objetos y son mencionados brevemente, en la tabla 2. Modelos para el ciclo de vida de desarrollo de software orientado a objetos.

CAPÍTULO 1. CICLOS DE VIDA DE LOS SISTEMAS ORIENTADOS A OBJETOS

Tabla 2. Modelos para el ciclo de vida de desarrollo de Software Orientado a objetos

Modelo	Autor	Etapas	Características
Modelo de Agrupamiento	Bertrand Meyer 1990	⇒ Especificación ⇒ Diseño ⇒ Implementación ⇒ Verificación/Validación ⇒ Generalización	Evita el efecto todo-nada propio del modelo en cascada. Tiene un elemento secuencial y uno concurrente. Existen diferentes sub ciclos de vida, que pueden solaparse en el tiempo. Con etapas de especificación, Diseño, Realización, Validación, Generalización. Se aplica al clúster no al sistema completo.
Modelo Fuente	Henderson-Sellers y Edwards en 1990	⇒ Estudio de viabilidad y requisitos ⇒ Análisis ⇒ Diseño conceptual ⇒ Componentes ⇒ Codificación ⇒ Pruebas unitarias ⇒ Pruebas de sistema ⇒ Utilización ⇒ Mantenimiento	Representa de forma grafica, un alto grado de iteración y solapamiento que hace posible la tecnología de objetos. Además, se propone un modelo de ciclo de vida para cada clase o módulo. Cada clase puede estar en una fase diferente del ciclo de vida durante el desarrollo del sistema.
Modelo Remolino	James Rumbaugh 1992	⇒ Especificación ⇒ Diseño ⇒ Realización ⇒ Validación y Generalización	Las metodologías de desarrollo no ofrecen una visión real del ciclo de vida en el desarrollo orientado al objeto. El ciclo de vida de un desarrollo orientado al objeto es desordenado, involucrando múltiples iteraciones interrelacionadas.
Modelo Pinball	Amplmer 1994	⇒ Encontrar atributos ⇒ Encontrar clases ⇒ Encontrar métodos ⇒ Encontrar relación ⇒ Definir agregación ⇒ Definir su sistemas ⇒ Definir colaboración ⇒ Definir herencia ⇒ Programar ⇒ Probar	Se ejecuta de forma iterativa a encontrar clases, atributos, métodos e interrelaciones y define colaboraciones, herencias, agregación y subsistemas. Los pasos toman diferente orden, pero siempre siendo simultáneos El desarrollo de este modelo puede ser de dos maneras, una basándose en tecnologías y métodos aprobados y la segunda con mayor riesgo, pero mas ventajas.

Algunas diferencias entre un ciclo de vida estructurado y uno orientado a objetos se muestran en la tabla 3. Básicamente, la diferencia entre estos ciclos de vida es el enfoque con el que se llevan a cabo las etapas del análisis, diseño y desarrollo. Mientras los ciclos de vida estructurados se centran en el proyecto, el orientado a objetos se enfoca en el producto (Niето, 2009).

Tabla 3. Comparación entre el ciclo de vida estructurado y el orientado a objetos (Weitzenfeld, 2005)

Etapa \ Modelo	Ciclo de Vida Estructurado	Ciclo de Vida Orientado a Objetos
Especificación de requerimientos	⇒ Obtener las necesidades del sistema en términos de funciones, estableciendo datos de entrada y salida. ⇒ Las condiciones de transformación de los datos.	⇒ Modelado de negocio ⇒ Identifica a los usuarios del sistema ⇒ Identificar deficiencias actuales en el ambiente del usuario ⇒ Establecer objetivos y alcances para el nuevo sistema ⇒ Determinar la factibilidad de un nuevo sistema y escenarios aceptables ⇒ Diseñar plan de proyecto
Análisis	⇒ Transformar las entradas principales, políticas de usuario y esquema del proyecto en especificación estructurada con: ⇒ Diagramas de flujo de datos ⇒ Diagramas de entidad-relación ⇒ Diagramas de transición de estados ⇒ Diccionario de datos. ⇒ Obtener especificaciones de los procesos	⇒ Especificación de casos de uso ⇒ Diseño de modelo de casos de uso ⇒ Diagramas de transición de estados ⇒ Diagramas de colaboración ⇒ Diagramas de secuencia ⇒ Diagramas de Subsistemas
Diseño	⇒ Distribuye módulos de la especificación a los elementos correspondientes, tareas a cada procesador ⇒ Diseña una estructura jerárquica de las especificaciones con los interfaces entre los módulos para la implantación de acuerdo a una heurística definida.	⇒ Se diseña la arquitectura del sistema. Una estructura de desarrollo modular o en capas ⇒ Esta arquitectura frecuentemente es diseñada para un desarrollo orientado a objetos ⇒ Se diseñan los diagramas de clases ⇒ Diagramas de organización ⇒ Diagramas de secuencia
Desarrollo	⇒ Consiste en la codificación e integración en una estructura cada vez mas completa de un sistema final, mediante programación estructurada e implantación descendente. ⇒ Los problemas complejos se van simplificando en problemas menores con la estrategia TOP-DOWN	⇒ La implementación por lo general se lleva a cabo en un lenguaje orientado a objetos y con interfaces entre clases y/o módulos, de acuerdo a los subsistemas definidos en el análisis

Haciendo una comparación entre el ciclo de vida estructurado y el orientado a objetos, encontramos que el estructurado tiene una etapa específica para generar

los test de prueba una vez realizado el desarrollo, mientras en un ciclo de vida orientado a objetos, el test de pruebas se puede generar a la par del desarrollo, una vez obtenidos y especificados los requerimientos.

La adaptación del usuario a un nuevo sistema es muy importante, ya que será este quién finalmente trabajará con el software desarrollado. En el modelo estructurado, los procedimientos se documentan para un manual de usuario y en la etapa de instalación es común que los usuarios reciban este manual o a veces un entrenamiento. En los ciclos orientados a objetos la documentación se lleva a cabo por la etapa de pruebas y dentro de la implementación se encuentra la instalación y puesta en ejecución del sistema y la adaptación de los usuarios al nuevo sistema.

El modelo orientado a objetos contempla una etapa de mantenimiento que prevé las mejoras y posibles modificaciones al software, lo que da un grado de satisfacción más para el cliente, mientras que en el modelo estructurado, la última etapa del ciclo es la instalación del mismo.

1.2. ETAPAS DEL CICLO DE VIDA ORIENTADO A OBJETOS

1.2.1. Especificación de los Requerimientos.

Las **necesidades** por las que el cliente requiere un sistema tienen ciertas características específicas, y estas características son los **requerimientos** que se utilizan para el desarrollo del mismo.

En muchos ciclos de vida esta etapa no se considera relevante ya que se dan por entendido al realizar la estimación del proyecto. Se sigue subestimando el análisis de los requerimientos, sin darse cuenta que es precisamente aquí donde nace el sistema y que un error en esta etapa tiene costosas consecuencias en el proyecto.

En los últimos años se ha indicado la importancia de éstos y la necesidad de su claridad. De echo ya existe una disciplina conocida como “ingeniería de requerimientos”, cuya importancia es dar prioridades a los mismos y mantener los márgenes de calidad para evitar defectos en los mismos. Este aspecto profundizará un poco más en el siguiente capítulo, ya que el tamaño de esta disciplina ha dado pie a otras investigaciones y estudios.

El objetivo de este paso es obtener las necesidades del cliente y sus características lo más detallado posible, con la finalidad de facilitar las siguientes etapas del ciclo. Para cumplir con este paso, normalmente se lleva a cabo mediante entrevistas con el cliente y se puede explorar como esta funcionando actualmente el sistema o el negocio para obtener los pasos y los actores del sistema. La importancia de cada elemento dentro del **proceso**.

Para un mejor entendimiento de los requerimientos se diseña un modelo de requisitos ó en algunos casos conocido como modelo de dominio, donde se describen los elementos del negocio que son claves para el sistema.

El modelo de requisitos es el primero en desarrollarse y tiene como objetivo delimitar el sistema y capturar la funcionalidad que ofrecerá desde la perspectiva del usuario, la mayoría de las veces sirve como base para formar todos los demás modelos en el desarrollo de software.

Este modelo puede trabajar como un contrato entre el desarrollador, el cliente y el usuario del sistema (Cantone, 2008) En general cualquier cambio en la funcionalidad del sistema es más fácil de hacer y con menores consecuencias a este nivel comparado con una etapa posterior.

Para poder diseñar este modelo es necesario hacer una descripción del problema en aspectos generales con el propósito de comprender la totalidad del problema y sus implicaciones, aquí se deben separa los requisitos verdaderos de las decisiones relacionadas con el diseño e implementación. A esto se le conoce como análisis de requerimientos.

Este tema se profundizará en el capítulo 5, donde se vera a detalle la etapa de obtención de requerimientos.

1.2.2. Análisis.

El primer paso es entender que debe hacer el sistema y documentar las necesidades.

Del entendimiento de las necesidades especificadas por el cliente, se procede a una identificación de problemas, oportunidades, objetivos y posibles soluciones. Una vez identificados estos elementos se delimitan los concernientes a los requerimientos y ahora se analizan las necesidades del sistema a desarrollar, estas necesidades pueden ser técnicas ó de información.

Lo más factible en esta etapa es que una vez identificados los problemas ó falta de información se regrese con el cliente para aclarar estos puntos, son la finalidad de evitar ambigüedades en las siguientes etapas del ciclo.

Los requerimientos se deben clasificar de acuerdo a su **naturaleza** y aislar sus características y elementos esenciales.

Ya despejadas las dudas y entendidos los requerimientos, se realiza una documentación de los mismos que permita un entendimiento tanto del cliente como de diseño y se modelan los requerimientos y el sistema. Esta documentación debe ser **validada** y **verificada** por el cliente con la finalidad de limitar el alcance del sistema y acordar que lo entendido por el analista, es lo deseado por el cliente.

Para un análisis orientado a objetos la documentación de la funcionalidad se lleva a cabo mediante **casos de uso** y para su comunicación y para su comunicación y entendimiento se identifican los **actores** y se diseñan los **modelos de casos de uso** y de **actividades**.

El modelo de casos de uso describe un sistema en términos de sus distintas formas de utilización, cada una de las cuales se conoce como casos de uso.

El modelo de casos de uso contiene y representa todas las funcionalidades del sistema y los actores afectan al sistema. Pero la descripción formal de esta etapa se llevará acabo en el capitulo 5, donde se describirá a detalle la etapa del análisis.

1.2.3. Diseño

Durante el diseño se extiende el modelo de requisitos de acuerdo a las especificaciones de rendimiento y los protocolos de interacción para los sistemas externos.

En el diseño se modela la aplicación que va a ser realizada, se formaliza en un marco más técnico. La información obtenida en el análisis y orienta a un lenguaje de programación. En este paso se define el patrón de desarrollo a utilizar. El sistema a construir debe estar adaptado al ambiente en el que será implementado y se toman en cuenta las características y riesgos propios de su funcionamiento. Aquí se limita a formar modelos de como va a funcionar internamente el sistema,

la comunicación entre los elementos que lo conforman... a grandes rasgos, se diseña la arquitectura del sistema.

En un diseño orientado a objetos se trabaja un diseño general del sistema y un diseño detallado.

En el diseño del sistema se busca optimizar la información derivada de los requisitos y el análisis. Se diseña una arquitectura que descompone en sub-sistemas el sistema general y los elementos de comunicación entre ellos. Debe quedar en claro que la etapa de diseño compone en marcos generales los elementos del sistema.

El diseño detallado se basa en conceptos de la teoría de objetos. Su finalidad es abstraer los requerimientos hasta concebir un elemento funcional que ayude a cubrir con el requisito y las características estándar que deben cumplir todos los elementos alrededor de este requisito. En el caso de que los sub-sistemas se comuniquen entre sí, es necesario estandarizar los elementos de comunicación.

Los modelos del diseño generalmente van orientados al ciclo de vida del sistema, pero eso implica que los **patrones de desarrollo** serán utilizados.

Los principales diagramas que se desarrollan en esta etapa del ciclo son los **diagramas de secuencia**, de **clases**, de **colaboración** y de **estados**.

1.2.4. Desarrollo

En esta etapa se codifican los elementos que conformaran el sistema. Contiene todos los elementos que darán solución al sistema, desde la vista del usuario hasta los procesos ocultos del mismo; los almacenamientos de información (bases de datos o repositorios) y las conexiones a los mismos.

En un desarrollo orientado a objetos se codificaran módulos alrededor de las clases se identifican los objetos, se modelan y nombran las clases de forma estándar y perfectamente claras para que cada elemento que lo requiera pueda auxiliarse con el uso de estas. Se da forma a las líneas de comunicación y se definen e implementan las conexiones entre clases y/o módulos.

Aquí se identifican y codifican los atributos y métodos de las clases, se definen a detalle los tipos de métodos de las clases y los parámetros de este, de acuerdo a las necesidades de la plataforma de implementación.

En el desarrollo existen muchos lenguajes y **métodos**, pero esto depende tanto del diseño como del desarrollador.

Los diagramas con los que se comunican los desarrolladores son principalmente los diagramas de distribución y los diagramas de componentes que pueden ser elaborados con apoyo del diseñador ó arquitecto.

1.2.5. Pruebas

Para que un sistema sea realmente de calidad, cada una de las etapas debe ser construida bajo estándares de calidad, sin embargo esta etapa es la que mide la calidad con la que ha sido construido el sistema. La etapa de pruebas tiene por objetivo identificar y reportar todos los defectos encontrados en el sistema. Estos defectos se obtienen mediante lo construido por desarrollo y lo entendido por análisis con el fin de que los resultados sean los esperados y especificados por los requerimientos.

Al igual que las otras etapas, las pruebas se pueden planear y ejecutar mediante modelos. El modelo de pruebas es el responsable de revisar la calidad del sistema, consiste en la validación del sistema ó prueba de especificación y verificación ó prueba de resultado. El modelo de prueba combina pruebas de unidad y de integración.

Con esto no queremos decir que deba ser el mismo, puede utilizarse un método de pruebas distinto al utilizado en el desarrollo.

No olvidemos que existen diferentes **pruebas**, desde las pruebas a los requisitos, al diseño y codificación hasta las pruebas de funcionalidad del módulo ó sistema desarrollado, de estrés.... Estas pruebas se realizan antes de la entrega al cliente, con la finalidad de depurar el sistema y corregir los errores. Detectar un error de los requerimientos en esta etapa del sistema es muy costoso, ya que el requerimiento ha pasado por distintas etapas del ciclo de vida y diferentes procesos para su desarrollo, sin embargo es mejor encontrarlo en esta etapa a permitir que el defecto llegue al cliente.

Es necesario tener en cuenta todos los requerimientos, tanto funcionales como no funcionales y técnicos, que se deben validar ya que si fueron especificados por el usuario deben cumplir con la condición para ser considerado un sistema de calidad.

1.2.6. Implementación

Esta etapa consiste en la transición de la forma de trabajo actual al nuevo sistema. Consiste en un proceso de instalación y diseño de un manual de usuario, planes de entrenamiento del usuario y planes de instalación, conversión y acondicionamiento. Es importante conocer el sistema que se va a instalar y la plataforma en la que se encontrará funcionando para hacer un acoplamiento de comunicación entre estos dos, así como los usuarios que interactuarán con el.

En ocasiones el adiestramiento de los usuarios también está incluido en esta etapa, ya que hay que considerar a quienes se va a adiestrar y los diferentes tipos de usuarios para los que fue desarrollado el sistema.

1.2.7. Mantenimiento

La etapa de mantenimiento de un sistema es la continuación del ciclo de vida, luego de haber completado, una primera versión de este. De cierta manera, el mantenimiento significa seguir un nuevo ciclo de actividades de desarrollo, pero a partir de un sistema ya existente.

Es importante recordar que los diagramas descritos en este trabajo no son todos, ya que dentro de UML existen alrededor de 20 diferentes diagramas, pero depende de las necesidades del proyecto.

Para facilitar el entendimiento de una metodología común entre el cliente y la compañía de software se han diseñado modelos de ciclo de vida y lenguajes de comunicación común para el entendimiento del proceso (Weitzenfeld, 2005). El lenguaje más utilizado para estos proyectos es UML (Unified Modeling Language), por ello nos apoyaremos en estos modelos para cada una de las etapas del ciclo de vida.

De esta manera se puede decir que los modelos son los métodos con un lenguaje en común que usa los desarrolladores de software para ordenar las técnicas en el proyecto y sirven como guía de administración para el proyecto.

1.3. SINTESIS

En este capítulo se ha mencionado en aspectos generales las etapas de un ciclo de vida estándar, de acuerdo a RUP. La finalidad corresponde a identificar los flujos y formas de trabajo de cada uno de los modelos de desarrollo así como las ventajas y desventajas de estos. El objetivo se encuentra hacia una teoría orientada a objetos ya que en la vida real es eso con lo que nos identificamos, objetos. Del mismo modo se ha mencionado las diferencias entre cada etapa de un enfoque estructurado a uno orientado a objetos.

Se debe recordar que el objetivo general de este estudio se orienta a una los requerimientos y como se ha mencionado en este capítulo, estos son, entro otras características, factor para la selección del mejor modelo de desarrollo.

REFERENCIAS BIBLIOGRAFICAS

- Dijkstra, E. W (1976). *A Discipline of Programming*. Prentice-Hall, Inc. United States of America.
- Presman, R. S. *Ingeniería de Software* (1993). Un enfoque práctico. Editorial McGraw-Hill. Tercera edición
- Weitzenfeld, A. (2005) *Ingeniería de software orientada a objetos con UML, java e internet*. Editorial Thomson.
- Fontela, C.(2011). *UML. Modelado de software para profesionales*. Editorial Alfaomega. 1ra edición.
- Grady Booch, James Rumbaugh & Ivar Jacobson (2005). *El lenguaje unificado de modelado. UML 2.0*. 2ª. Edición. Peason Adison Wesley. España

REFERENCIAS ELECTRONICAS

- Cantone, D. (2008). *Biblia del programador implementación y debugging: claves, técnicas y herramientas para construir código solido y confiable*. Mp ediciones. Recuperado de img.redusers.com/imagenes/libros/lpcu097/capitulogratis.pdf
- Forsberg, K., Mooz, H., Cotterman, H (2005). *Visualizing Project Management* (en inglés), 3ª edición, John Wiley and Sons, Nueva York, NY Recuperado de: http://es.wikipedia.org/wiki/M%C3%A9todo_en_V
- Guerra, M. A. & Mousalli K. G. (2009). *Metodología extendida para la creación de software educativo desde una visión integradora*. Por Zulma Cataldi Universidad de Buenos Aires. de: <http://www.slideshare.net/guestffe8df/metodologa-extendida-1731772>
- Muñoz Díaz, D. *Federación de Bases de datos mediante ontologías*: Antive. De: http://e-archivo.uc3m.es/bitstream/10016/5844/1/PFC_David_Munoz_Diaz.pdf
- ISO/IEC 9126, (2009). de <http://iso25000.com/index.php/iso-iec-9126.html>
- Borja, E. (2011). *Els Cicles de Vida del SW*. de: <http://ciclesvidasw.blogspot.mx/>

- Zuñiga S. S. (2004). *Ingeniería de Software I. Métodos – Técnicas – Herramientas*. Recuperado de www.ingesoftwareltda.com/IngSw1.doc
- Nieto Castellanos, Bernardo (2009). “Ciclo de vida de los sistemas”. de: http://www.slideshare.net/UNM/ciclo-de-vida-de-los-sistemas?src=related_normal&rel=5653807
- Henao Acosta, Carolina, Ortiz Villegas, Juan Pablo & Valencia Gil, Carolina.(2010). *Desarrollo de Software Orientado a Objetos*. de: <http://www.slideshare.net/juanpabloov18/desarrollo-de-software-orienta-a-objetos>
- Cetina Dzul R.C., Hau May A.S., Par Puc N.G. y Tabasco A. A. *Modelo Orientado a Objetos*. de http://www.slideshare.net/jose_rob/modelo-orientado-a-objetos
Recuperado de carolina.terna.net/ingsw2/Datos/Cascada-ModeloV.doc
- 201. de <http://ciclesvidasw.blogspot.mx/2011/11/ciclo-de-vida-tipo-sashimi.html>
- Boucchechter I.& Rojas. (2005), Richard. *Ciclos de Vida de Ingeniería de Software*. De: carolina.terna.net/ingsw2/Datos/Cascada-ModeloV.doc
- Tawy. (2010) de <http://ciclosdevida.blogcindario.com/2010/03/00001-ciclo-de-vida.html>
- recuperado de http://en.wikipedia.org/wiki/Winston_W._Royce
- Tabla 2. Recuperado de <http://www.buenastareas.com/ensayos/Modelos-De-Desarrollo-De-Sistemas-Orientados/1193373.html>

CAPITULO 2. CALIDAD EN LOS SISTEMAS DE INFORMACIÓN

Como se ha visto, existen muchas formas de desarrollar un sistema. A este capítulo corresponde describir aquellas normas que establecen una buena práctica para el desarrollo de los sistemas de información, con un enfoque hacia los requerimientos.

Las características de estos son tomadas como base de estándares establecidos tanto por ISO como por IEEE para tener un buen fundamento que facilite el desarrollo de sistemas de información.

2.1. INTRODUCCIÓN A LA CALIDAD DE LOS SISTEMAS DE INFORMACIÓN

En el ambiente de desarrollo actual, el objetivo es satisfacer al cliente mediante sistemas que cumplan sus necesidades tanto las actuales como las futuras, con el menor costo y tiempo de desarrollo. Para cumplir con dicho objetivo los sistemas deben de seguir con ciertos estándares, esto es, calidad, cada una de las etapas del proceso de desarrollo debe ser realizada con eficacia, esto es con calidad.

Existen varias definiciones de calidad, dependiendo de los autores, pero en el presente trabajo se enunciarán las dos más importantes de acuerdo a su institución, una de la ISO y otra de IEEE. Agregaremos una más del autor Pressman (1993), que describe la calidad para fines de esta investigación ya que su definición está orientada a la satisfacción de los requerimientos.

De acuerdo a las normas ISO 9000, "La calidad del software es el grado en que un conjunto de características inherentes del software que cumple con los requisitos del sistema"(Weitzenfeld, 2005, pp. 406.).

De acuerdo a la IEEE, "Calidad es el grado en que un sistema, componente o proceso cumple con los requerimientos especificados y las necesidades del cliente o usuario" (Weitzenfeld, 2005, pp. 713).

En resumen, ambas definiciones mencionan el nivel de importancia de cubrir las necesidades solicitadas por el cliente, o en algunos casos requeridos por el mismo sistema. La definición más apegada a los propósitos de esta investigación es enunciada por Pressman define la calidad como "la concordancia con los requisitos funcionales y de rendimiento explícitamente establecidos, con los estándares de desarrollo explícitamente documentados además de las características que se espera de todo software desarrollado profesionalmente"(Chiosso, 2008).

Si bien, al mencionar que son explícitamente establecidos, Pressman sólo hace referencia a los solicitados por el cliente, cuando en la actualidad, obtener los requisitos es un trabajo complejo. Es muy común en la actualidad que el consultor,

se da a la tarea de encontrar los requisitos implícitos de un sistema y convertirlos en explícitos, incluso, en ocasiones, al realizar el análisis, se encuentran necesidades que jamás estuvieron en la vista tanto del cliente como del consultor y estas mismas deben ser expuestas para ver si serán consideradas o no, dentro del nuevo sistema.

Las definiciones anteriores son muy interesantes dado que a pesar de estar claramente definido el término es común que los sistemas se realicen sin lineamientos, lo cual genera modificaciones contantes, que tienen costo en tiempo y recursos, al respecto Chiosso (2008) encontró ...

- ⇒ El 16% son completados con el alcance esperado, en el tiempo planificado y dentro del presupuesto asignado.
- ⇒ El 53% de los proyectos son completados con menor alcance, y/o sobrecosto y/o fuera de término.
- ⇒ El 31% de los proyectos son cancelados antes de terminar.

Del total de proyectos que se completan:

- ⇒ El 70% de los proyectos terminan fuera de plazo.
- ⇒ El 54% de los proyectos sufren sobrecostos.
- ⇒ El 66% de los proyectos no son considerados exitosos.
- ⇒ El 30% de los proyectos son cancelados antes de terminar.

Principales factores de éxito:

- ⇒ Involucramiento del usuario 24.5 %
- ⇒ Apoyo de la gerencia 22.8 %
- ⇒ Enunciado claro de los requerimientos 19.6 %
- ⇒ Planeamiento adecuado 18.2 %
- ⇒ Expectativas realistas 7.7 %
- ⇒ Recursos humanos competentes 7.2 %

Principales factores de fracaso:

- ⇒ Requerimientos incompletos 28.1 %
- ⇒ Falta de involucramiento del usuario 14.4 %
- ⇒ Falta de recursos 13.6 %
- ⇒ Expectativas no realistas 16.9 %
- ⇒ Falta de apoyo de la Gerencia 8.7 %
- ⇒ Requerimientos cambiantes 18.3 %

Del análisis realizado por Chiosso y Weitzenfeld se puede concluir que sus estudios coinciden respecto sobre factores que afectan la obtención de un producto de calidad son:

- ⇒ El cliente o usuario
- ⇒ El desarrollador o equipo de desarrollo
- ⇒ El proceso mismo
- ⇒ El producto final

El usuario es el responsable de definir los requerimientos o requisitos del producto final, mientras que el consultor debe canalizar las necesidades de la empresa, sugerir posibles mejoras a cada requerimiento y/o complementar la idea general del usuario para formalizar el requerimiento. El desarrollador – refiriéndolo como el equipo de desarrollo - es el responsable del proceso de producción de calidad, el producto es el resultado esperado y el proceso es un conjunto de etapas que debe ser desarrollada con ciertos niveles de calidad para cada una. Para verificar que los procesos de desarrollo sean de calidad, se emplean medidas bien definidas que permiten mejoras continuas de los productos, con la finalidad de evitar anomalías en la aplicación de alguna metodología.

Al hablar de calidad en los sistemas de información inevitablemente viene a la mente el concepto “Modelos de madurez del proceso”, sin embargo, el enfoque en este estudio será con carácter exclusivo de los requerimientos, sólo se enunciarán los modelos de madurez, pero sin entrar en el detalle. No debemos olvidar que “para que un producto sea de calidad, cada una de sus etapas deben ser desarrolladas con calidad”.

Los modelos de madurez del proceso evalúan la calidad y apoyan una mejora continua para los procesos. Existen certificaciones para las empresas que verifican que “digan lo que hacen”, “hagan lo que dicen” y “demuestren que lo que dicen así lo hacen” (Tabla 1). Estos modelos tienen la finalidad de guiar el proceso de desarrollo de software e ir depurando los defectos de un proyecto a otro. Debido a que estos modelos se encuentran bajo las normas de calidad antes mencionadas, tienen por objetivo cubrir las necesidades requeridas, lo que diferencia unos modelos de otros es como se realizará este proceso o que actividades se realizarán para garantizar el resultado.

Tabla 1. Modelos de madurez de los procesos (Weitzenfeld, 2005)

Modelos de madurez de los procesos (calidad del software)	
Año	Modelo
1987	ISO 9001
1991	ImproveIT V1.0 (TickIT) ISO 9000-3 SEI SW-CMM V1.0
1992	TickIT V2.0
1993	SEI SW-CMM V1.1
1994	ISO 9001 (relanzamiento)
1995	ISO 12207 ISO 15504 (SPICE) PSP
1996	IEEE/EIA 12207 TSP
1997	ISO 9000-3 (relanzamiento) SEI para revisiones de SW-CMM
1998	ISO 15504 (SPICE relanzamiento) TickIT V4.0
1999	PEMM V1.0
2000	ISO 9000:2000 SEI CMMI V1.02
2001	PMMM
2003	SEI CMMI V3.0

Recordemos que todo sistema nace de una necesidad ya sea explícita por el cliente o implícita por los usuarios, es por ello que un requerimiento debe ser obtenido bajo ciertos criterios que permitan una correcta administración de estos. El método de obtención de requerimientos puede variar de acuerdo a diferentes factores, incluso debido al modelo de desarrollo del sistema mismo.

2.2. CALIDAD EN LOS REQUERIMIENTOS

Debemos recordar en todo momento que la calidad máxima o mínima requerida en un software es determinada por el cliente, y comúnmente se documenta en un artefacto conocido como criterios de aceptación, sin embargo, es obligación del proveedor, de la solución a la necesidad establecida, verificar que el sistema desarrollado cubra todos los requerimientos solicitados.

Al obtener la información del cliente, es muy común que lo solicitado por él, no sea lo comprendido por el consultor, por esa razón es necesario tener bien claro que es un requerimiento y cuáles son sus características.

La norma ISO 9126 consolidó una norma de seis atributos para que un sistema se considere de calidad, este modelo nos sirve como guía estándar, independiente al modelo adoptado (ISO/IEC 9126, 2009). Los atributos de esta norma:

- ⇒ Funcionalidad
- ⇒ Fiabilidad
- ⇒ Usabilidad
- ⇒ Eficiencia
- ⇒ Mantenibilidad
- ⇒ Portabilidad

Estos atributos, deben encontrarse inmersos en el sistema, y poseen la característica de ser comprobables. Si bien un sistema tiene criterios de calidad, en un marco general, por qué no deberían los requerimientos – que son la razón de ser de un sistema – tener, también, características de calidad a fin de identificar posibles defectos e incluso riesgos desde una primera etapa.

2.2.1. Características de los requerimientos (ISO, 2009).

Afortunadamente para los analistas, ya no es necesario inventar el hilo negro, en su defecto sería refinarlo. La Organización Internacional para la Estandarización (ISO) ya ha definido estas características que deben poseer los requerimientos para ser considerados y posteriormente clasificados para su análisis e implementación en un sistema.

- ⇒ **Necesarios.** Los requerimientos deben ser sólo aquellos que resultan necesarios para la construcción y funcionamiento de forma correcta del sistema. En ocasiones, algunos requerimientos de los especificados por el usuario no son requeridos para el sistema, o por el contrario, algunos no son mencionados, cometiendo el error de darlos por entendido.

- ⇒ **Concisos.** El requerimiento debe dirigirse a una sola característica del sistema, no se debe tratar de englobar varias necesidades en un sólo requerimiento, ya que esto lo vuelve más confuso y resta simplicidad al trabajo de su análisis.
- ⇒ **Completo.** Los requerimientos deben estar claramente especificados, documentados, almacenados y aprobados por el usuario, para tener un respaldo de lo solicitado y acordado por ambas partes.
- ⇒ **Consistentes.** Los requerimientos especificados por el usuario no deben contradecir u oponerse a otros requerimientos previamente especificados y debe ser coherente con la documentación obtenida. Es necesario tener especial cuidado en esta característica cuando por parte del cliente se tienen dos usuarios o dos directivos que estén especificando las características del nuevo sistema, dado que sus puntos de vista pueden oponerse y colocar en conflicto los requerimientos y con esto el sistema mismo.
- ⇒ **No ambiguo.** Se deben expresar puntos objetivos de las necesidades del sistema, no ambigüedades, debe ser especificado con una claridad que permita ser identificado de una manera única por todos sus lectores.
- ⇒ **Verificables.** Un requerimiento es una característica del sistema. Al ser parte de una necesidad del sistema, tiene la propiedad de ser verificable, en algún punto del periodo de pruebas, este requerimiento debe ser comprobado.

2.2.2. Clasificación de los requerimientos

Ahora que ya se han descrito las características de los requerimientos bien valdría, la pena hacer referencia a la clasificación de los requerimientos de acuerdo a sus particularidades. La clasificación de los requerimientos nos permite ordenar prioridades de atención. Es muy común que la clasificación sólo abarque requerimientos funcionales y no funcionales, dejando a un lado aquellos que forman el entorno de la necesidad real. Resulta de gran importancia para comprender las necesidades originales del sistema.

Flores, Mora, Álvarez y Fernández (2006) refieren que los requerimientos se pueden clasificar en:

- ⇒ **Sociales.** Son aquellos requerimientos que comprenden el contexto social en donde funcionará el sistema y estudian a los usuarios que interactuarán con él.
- ⇒ **Operacionales.** Los requerimientos operacionales describen.

- ⇒ **Funcionales.** Como su nombre lo dice, describe la funcionalidad del sistema, una a una las tareas de que debe hacer el sistema, esta característica puede llegar a confundirse con la operacional si no se tiene específico cuidado. Los requerimientos funcionales son una descripción detallada de la funcionalidad del sistema para una tarea específica, su interacción con otras entidades y sus excepciones.
- ⇒ **No funcionales.** Los requerimientos no funcionales, por lo general son restricciones del sistema que no afectan su funcionalidad, pero si especifican el cómo debe funcionar y algunos atributos que debe tener, como son rendimiento, interfaces. Algunos conocen a los requerimientos funcionales como “cualidades del sistema”, que nada tiene que ver con el punto de cumplir o no la tarea del sistema. Afecta el cómo está cumpliendo su objetivo, pero no el hecho de que lo esté haciendo o no, algunos ejemplos de requerimientos no funcionales son:
 - Comportamiento. Como debe reaccionar el sistema al recibir determinada acción.
 - Diseño. Como debe comunicarse el sistema, ya sea con otros sistemas o con el usuario.
 - Implantación.
 - Pruebas. Como deben realizarse las pruebas para obtener los resultados esperados.
 - Calidad. Que criterios deben tomarse en cuenta de forma obligatoria para el sistema y cuales pueden quedar delegados.
 - Portabilidad. Que tan fácil debe ser implementar un sistema en un ambiente u otro. La información emitida, debe ser compatible con otros sistemas para permitir su manipulación.

Para el analista es necesario tener en claro las características con las que debe contar un buen requerimiento y una clasificación de los mismos, con el fin de facilitar su administración y estudio. Es común que las características sean los lineamientos de calidad para el análisis de los requerimientos (calidad en el análisis). Pudiera parecer que este es un paso innecesario, pero debemos recordar que, para que un sistema sea de calidad, debe existir en cada una de sus etapas. Por esta razón, el que se cubran dichas propiedades en los requerimientos, reduce el riesgo de defectos desde los cimientos de un sistema.

Al referirse a la administración de los requerimientos, nos referimos a un seguimiento de los mismos a lo largo de todo el ciclo de vida del sistema, desde que nacen hasta el mantenimiento de los mismos. Este seguimiento es para tener identificados los elementos que se involucran con cada requerimiento, en

cualquier aspecto, desde su justificación hasta su implementación, con el fin de prevenir futuras modificaciones al sistema.

2.2.3. Lineamientos y criterios de calidad para los requerimientos

Como ya se ha visto en puntos anteriores, los requerimientos son el pilar de un buen sistema. Debido a la natural complejidad de estos, se han desarrollado métodos para la “Administración de Requerimientos”. Estas metodologías tienen el objetivo de facilitar la gestión de los requerimientos y agilizar su entendimiento por parte de todos los involucrados en el proyecto.

2.2.3.1. Características de Calidad de los Requerimientos de IEEE STANDAR 830-1998.

De acuerdo a la norma IEEE STANDAR 830-1998, los requerimientos tienen características que deben ser consideradas para tener una buena especificación (Adrián, 2008). Estas características nos permitirán verificar los requerimientos y facilitar su administración. La especificación de las características con las que se desea un sistema, generalmente se realiza en un ambiente determinado. La especificación puede escribirse por uno o más representantes del proveedor y uno o más representantes del cliente o para un mejor entendimiento de las necesidades se recomienda que sean ambos.

Adrián (2008) menciona que las dos características que nos permiten la clasificación de los requerimientos son:

- ⇒ **Naturaleza.** La naturaleza de la especificación la tarea o actividad a ser resuelta o una característica especial del software, puede ser la funcionalidad (qué), las interfaces externas (cómo), la velocidad, disponibilidad, tiempo, los portabilidad, exactitud, mantenimiento, seguridad y restricciones de diseño que impusieron en una aplicación como idiomas y políticas.
- ⇒ **Ambiente.** El software que se desarrolle puede contener toda la funcionalidad, ser un programa que funcione por sí solo o ser parte de un sistema más grande. Si forma parte de otro, se deben considerar las interfaces entre el sistema y el software "modulador", requisitos y funciones.

Debemos tomar en cuenta que obtener un requerimiento, no es lo mismo que especificarlo. La Especificación de Requerimientos de Software (ERS), consiste en documentar de forma detallada o general un requerimiento a fin de tener todas las que permitan tener en claro los objetivos del desarrollo. Es necesario tener en

cuenta que la ERS es una recopilación de los requerimientos y puede contener algunas características de estos, pero no por ello quiere decir que sean las mismas (Adrián, 2008).

La especificación de requerimientos de software tiene un papel esencial en el desarrollo de los sistemas, por lo tanto la persona que defina la especificación no debe de ir más allá de los límites que son:

- ⇒ Limitar la especificación a aquellos requerimientos que se encuentren documentados en el alcance del proyecto.
- ⇒ Se deben evitar confusiones o ambigüedades en la especificación.
- ⇒ La especificación no debe describir planes o detalles de aplicación.
- ⇒ No debe imponer restricciones adicionales en el software, que vayan más allá de las establecidas por los requerimientos no funcionales.

Estas características se conocen como características de la ERS:

- ⇒ **Correcta.** Una ERS es correcta si y sólo si cada requisito declarado se encuentra en el software.
- ⇒ **Inequívoca.** Una ERS es inequívoca si cada requisito declarado tiene una sola interpretación.
- ⇒ **Completa.** Una ERS está completa si contiene los requisitos relacionados a la funcionalidad, el desarrollo, restricciones del diseño, los atributos e interfaces externas. Tiene todas las etiquetas llenas y referencias a todos los elementos. Si posee la definición de las respuestas del software a todas las entradas y situaciones que se puedan dar.
- ⇒ **Consistente.** Una ERS es internamente consistente si ningún subconjunto de requisitos individuales que pueda entrar en conflicto con él.
- ⇒ **Delinear importancia y/o estabilidad.** Una ERS debe delinear la importancia y estabilidad si cada requisito tiene un identificador para indicar importancia o estabilidad de ese requerimiento en particular.
- ⇒ **Comprobable.** Una ERS es comprobable si y sólo si, cada requisito declarado es comprobable, esto sucede si y sólo si, en un requisito existe un proceso rentable finito con que una persona o la máquina puede verificar que el producto reúne el requisito
- ⇒ **Modificable.** Una ERS es modificable si y sólo si, su estructura y estilo son tales que puede hacerse un cambio fácilmente.
- ⇒ **Identificable.** Una ERS es identificable si el origen de cada requerimiento es claro y facilidad las referencias de cada requerimiento en el desarrollo futuro o documentación del mismo.

⊕ REALIAZACION DE LA ERS ENTRE PROVEEDOR Y CLIENTE

El proceso para desarrollar un software comienza con el proveedor y con el acuerdo del cliente sobre lo que el sistema debe realizar. Por lo tanto, debe de realizarse una ERS conjuntamente entre cliente y proveedor. No realizarlo de esta forma puede causar conflictos en la etapa de entrega. Los conflictos que surgen por la falta de esta práctica son básicamente dos:

- ⇒ Los clientes no entienden el diseño del software y el proceso de desarrollo como para escribir solos una ERS.
- ⇒ Los proveedores no entiende bien el problema del cliente y el campo de acción como para especificar los requerimientos y el software final sea satisfactorio.

⊕ EVOLUCION DE LA ERS

Como se ha mencionado, a lo largo del estudio, los requerimientos tienden a cambiar o extenderse. Cada etapa, genera una entrega de una a otra, por lo tanto, al cambiar los requerimientos, cada entregable es susceptible de ser modificado, por lo tanto la ERS puede necesitar evolucionar así como las necesidades que dan origen al software.

Para llevar a cabo esta actualización se debe tener cuidado desde su especificación inicial hasta los puntos de modificación en tiempo y los requerimientos afectados. Se deben tener dos consideraciones en este proceso:

- ⇒ Se deben especificar los requerimientos completamente como se entienden en el momento, a pesar de que puedan ser cambiados en el futuro.
- ⇒ Un proceso de cambio formal debe identificarse, que se deje un informe de los cambios realizados.

Los cambios realizados a los documentos deben aprobarse por ambas partes a fin de mantener un sólo entendido. Dichos cambios aprobados deben conjuntarse en la ERS de manera que se tenga un lineamiento de las modificaciones y el permiso concedido a la revisión y remplazo de los requerimientos originales especificados en la ERS.

⊕ PROTOTIPOS

Generar prototipos tiene la ventaja de que el cliente puede ver plasmada su idea y dar una opinión o muestra una reacción que permite evaluar si va por buen camino o no. Un prototipo facilita al usuario la comprensión de cómo se producirá la

interacción hombre – máquina (Pressman, 1993). Además los prototipos reflejan aspectos importantes antes de finalizar los sistemas, así pueden surgir nuevas preguntas y situaciones ante el despliegue de prototipos. También acorta el tiempo de desarrollo, ya que suele sufrir menos cambios.

Sin embargo, es muy común que el usuario vea el prototipo como algo ya definido, y en ocasiones espera ver en su sistema final exactamente lo que vio en su prototipo, sin tener en consideración que un prototipo sólo muestra el flujo que tiene el sistema, no una visión final del mismo. Pressman menciona algunas características que pueden ser problemáticas por el uso de prototipos (Pressman, 1993), como son:

- ⇒ Lo que el cliente ve funcionando es una primera versión del sistema, por lo que se ha hecho de una manera muy ligera, descuidando aspectos como la calidad y mantenimiento. Una vez que se informa al usuario que se hará una reconstrucción del sistema, solicita que se apliquen “algunas mejoras” que son necesarias para que el prototipo mostrado sea funcional, rebasando así, los costos originales del proyecto.
- ⇒ Las características técnicas utilizadas para el prototipo, pueden no coincidir con las requeridas en la especificación original, pero al ser mostradas deben formar parte del nuevo sistema (software funcional en diferentes sistemas operativos o elementos de comunicación).

⊕ GENERANDO EL DISEÑO EN LA ERS

Un requerimiento especifica una función externa visible o un atributo del sistema. Un diseño describe un subcomponente particular de un sistema y su interacción con otros componentes.

La ERS debe especificar las funciones que se realizarán, los datos a usar, que resultados dará, en que situación y para quien serán estos resultados. Documentar y administrar una ERS, resulta una tarea muy laboriosa, pero es necesario tener un registro bien definido de los requerimientos solicitados por el cliente. No es lo mismo tener especificados los requerimientos del usuario a detalle, que tener una especificación general de los requerimientos, ya que por lo general, este documento puede ser la base de una negociación. ¿Cuántos requerimientos tiene el sistema? y ¿De qué complejidad?

Los prototipos dentro de la ERS pueden ser un arma de doble filo, por un lado son ventaja para el entendimiento y por el otro, si no se aclara “que es un prototipo” puede retener al técnico a diseñar tal cual está el prototipo. Si bien es cierto que

los prototipos pueden sugerir mejoras al sistema, se debe diferenciar entre las mejoras y los nuevos requerimientos.

2.2.3.2. Características de Calidad de los Requerimientos según la norma ISO 9126 ISO/IEC 9126

La norma ISO 9126 ISO/IEC 9126 fue englobada en el proyecto SQuaRE para el desarrollo de la norma ISO 25000 donde se establece un nuevo modelo para el análisis de los requerimientos. Dicho modelo se divide en tres elementos: “calidad interior”, “calidad exterior” y “calidad de uso”. Estos elementos tienen características que se dividen en sub características y a su vez tienen atributos (ISO/IEC 9126, 2009). Los atributos obtienen mediciones que sirven como métricas para evaluar el software. Las métricas dentro de esta norma se clasifican según su naturaleza.

- ⇒ **Básicas** (internas). Se obtienen de un análisis del código o una ejecución del software
- ⇒ **De agregación** (externas). Es un conjunto específico de métricas básicas que resulta en un valor total.
- ⇒ **Derivadas** (de uso). Son funciones matemáticas cuya entrada es el valor de otra métrica.

Esta norma “englobada” establece los atributos de la norma ISO 9126 como características comunes para los elementos de calidad interna y externa del software y define otras cuatro para el elemento de calidad de uso.

- ⇒ **Efectividad**. Facultad que el software proporciona al usuario para alcanzar sus objetivos con precisión y completitud.
- ⇒ **Productividad**. Facultad de utilizar sólo los recursos necesarios en proporción a la efectividad.
- ⇒ **Seguridad**. Cumplimiento del software para los niveles de riesgo permitidos, sean físicos o de datos (información).
- ⇒ **Satisfacción**. Cumplimiento de expectativas del usuario para una situación determinada.

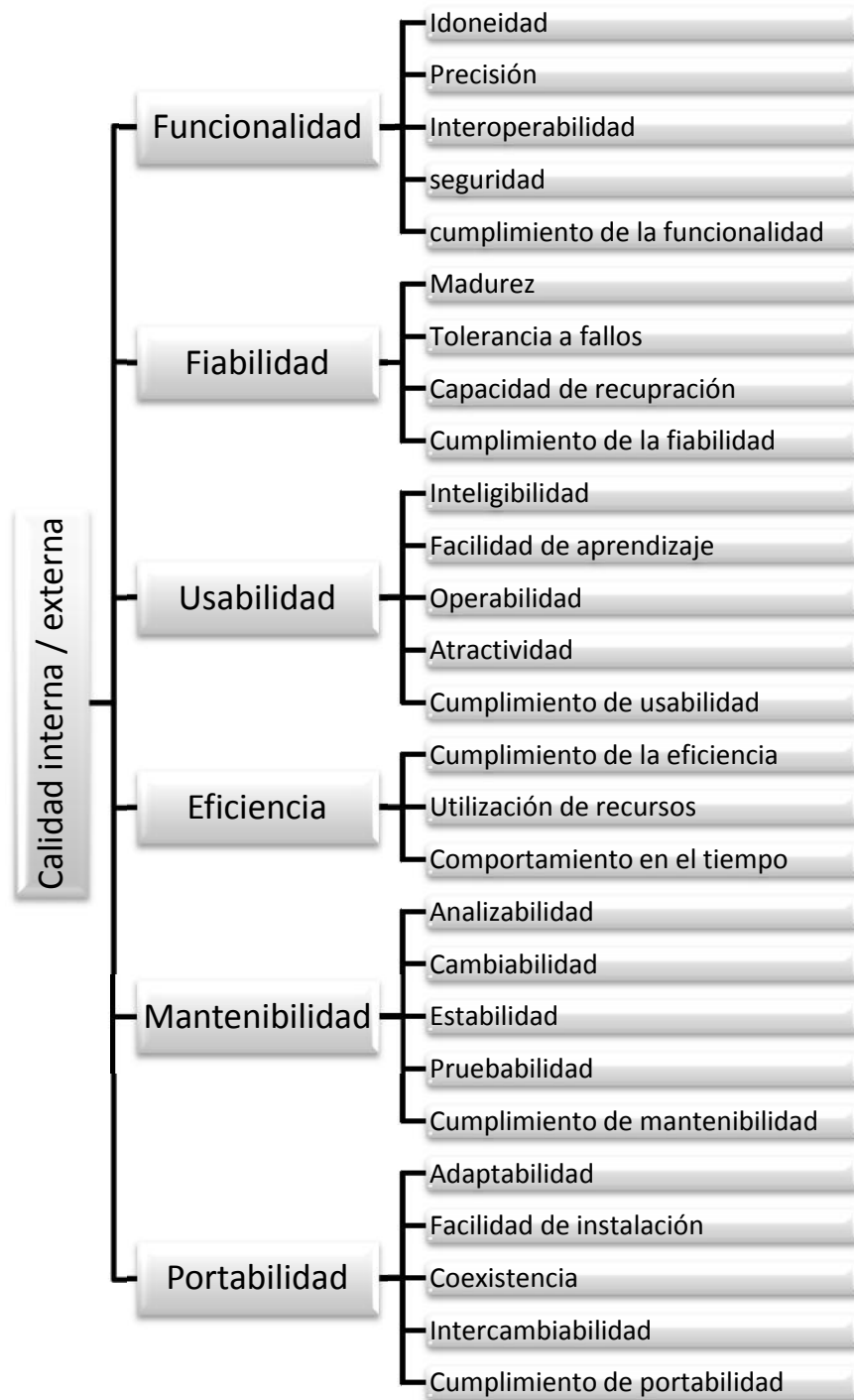


Figura 1. Calidad interna y externa del software



Figura 2. Calidad de uso

De acuerdo a lo definido por esta norma y lo conocido por las etapas del ciclo de vida – definidas en el capítulo anterior- se identificarán los elementos de la calidad correspondientes a los requerimientos en las etapas de levantamiento y especificación en la Tabla 2.

Tabla 2. Elementos de calidad en el ciclo de vida.

Etapas del Ciclo de vida	Objetivo	Etapas de calidad	Objetivo
Especificación de requerimientos	Obtener, entender y modelar las necesidades del usuario en un mapa general para un entendimiento de la necesidad del usuario.	De Uso	Métricas para tener una utilización efectiva por parte del usuario.
Análisis	De las necesidades especificadas por el usuario, se procede a una identificación y delimitación de requerimientos. Se especifica, qué debe hacer el sistema.	Calidad externa	Métricas utilizadas para medir el comportamiento del producto.
Diseño	En esta etapa se diseñan procedimientos precisos con la información obtenida de la etapa de análisis. Se especifica cómo debe trabajar el sistema.	Calidad interna	Métricas para medir las características del código fuente.

Una vez definido el mínimo detalle que debe poseer un requerimiento, de donde se obtiene, que tipos hay, que características debe poseer y cómo se debe

documentar, se resume que todas las necesidades expresadas por el cliente deben ser documentadas de forma específica en algún artefacto. Un requerimiento es la necesidad fundamental de un sistema, que puede expresarse en forma general e ir desglosándose conforme se analiza el problema.

Este requerimiento es, principalmente, establecido por el cliente, sin embargo, es posible que sea el analista quien identifique muchos otros que no son considerados en primera instancia. Para que un requerimiento sea considerado como tal, debe cubrir ciertas características establecidas en estándares internacionales. La correcta obtención de las necesidades dependen mucho de la cooperación y disponibilidad de los involucrados, sin embargo, existen otros factores que intervienen en el éxito de un proyecto y que pocas veces son considerados, aunque la mayoría de los autores menciona que el error más grave en un desarrollo de software se comete en la obtención y especificación de los requerimientos.

En el siguiente sub capítulo veremos que una vez entendidos los requerimientos y su naturaleza resulta más sencilla su gestión en un proyecto. Ya que un requerimiento puede llegar a ser como un catálogo, tener un registro, modificaciones, consultas e incluso hasta eliminación. De ahí el subtítulo mencionado.

2.3. ADMINISTRACIÓN DE REQUERIMIENTOS

Como es sabido, la administración por definición es el proceso de planificar, organizar, dirigir y controlar el uso de los recursos y las actividades de trabajo con el propósito de lograr los objetivos o metas de la organización de manera eficiente y eficaz (Thompson, 2008).

Dentro de los sistemas de software se conoce como la organización de registro, eliminación, modificación y consulta de alguna información. La administración de los requerimientos consta desde el análisis hasta la validación de los mismos y el manejo de las modificaciones. A simple vista, esto podría parecer un proceso muy sencillo, pero el esfuerzo real puede resultar más complejo; dependiendo del negocio al que estén dirigidos ya que comprende planificar, organizar, definir, clasificar, asignar, dar seguimiento, dirigir y controlar la información en torno a estos requerimientos(Standish Group).

Actualmente existen muchos modelos para la administración de requerimientos, dependiendo de la metodología de desarrollo de software o la administración del proyecto. Debido a la complejidad que pueden llegar a tomar los requerimientos se ha desarrollado un estudio de la aplicación de estos, conocida como Ingeniería de Requerimientos. Este estudio profundiza la administración de los mismos y a pesar de que las etapas pueden llegar a ser las mismas, su análisis resulta más completo, ya que no fue diseñado para una metodología específica, sino para un entendimiento y estudio general de los requerimientos.

Ya que esta investigación está orientada a objetos, el enfoque a las definiciones, tanto de etapas, como de ingeniería y administración de requerimientos será con un enfoque orientado a objetos.

Mena (2005) propone un orden para las actividades del proceso de obtención y análisis de requerimientos de la siguiente manera:

- ⇒ Comprensión del dominio
- ⇒ Recolección de requerimiento
- ⇒ Clasificación
- ⇒ Resolución de conflictos
- ⇒ Priorización
- ⇒ Verificación (completos, consistentes y acordes)

La administración de requerimientos es indispensable para el aseguramiento de la calidad de los productos, así como para el control y seguimiento de los proyectos.

2.3.1 Etapas de la Administración de Requerimientos.

Tomando la propuesta de Arias (2007) en su estudio de la ingeniería en requerimientos se describirán las etapas de la administración de requerimientos, agregando como primera etapa la comprensión del dominio sugerida por Mena (2005), con la finalidad de tener una estructura más sólida para el análisis.

Comprensión del dominio. Es muy recomendable que antes de tener contacto con el cliente se tenga un conocimiento y entendimiento general sobre el dominio del negocio, con la finalidad de llegar a entender el funcionamiento y las necesidades puntuales del mismo. Este conocimiento facilita muchísimo la transición de la información del usuario al diseñador y desarrollador. Aquí la habilidad del analista debe constar de una flexibilidad del lenguaje y un buen conocimiento del negocio, la tecnología y arquitectura del sistema, para evitar solicitud de requerimientos fuera de lo estimado y poder resolver las dudas de

ambas partes –cliente y desarrollo-. Se recomienda un estudio de la empresa y el ramo dentro del mercado.

Extracción. El objetivo principal de esta etapa consiste en obtener las necesidades del sistema, las características requeridas y en algunas ocasiones, como debe funcionar, con el cuidado de que los requerimientos cumplan con las características establecidas por los estándares de calidad (Arias, 2007).

Dependiendo del tipo de sistema a desarrollar son las técnicas utilizadas para obtener los requerimientos.

Las entrevistas y cuestionarios se utilizan para obtener información principalmente de los usuarios y es más común cuando se desarrollará un nuevo sistema. Esto no quiere decir que se utiliza sólo en esas ocasiones, es posible que si el sistema ya existe, esta técnica se use para encontrar deficiencias en él. Utilizar esta técnica para obtener información para un sistema existente sería un doble gasto, ya que nos generaría un ciclo: obtención de requerimientos, validación con el sistema existente, validación con el usuario, re especificación. Esto nos genera un término conocido como re trabajo.

En un sistema existente, la obtención de requerimientos se lleva a cabo principalmente por una serie de pruebas, tanto de caja negra como de caja blanca. La finalidad es analizar las interfaces existentes tanto para el usuario como con otros sistemas y bases de datos. Se debe observar el tipo de información que se maneja y cómo es manejada, entrada y salidas de información, para posteriormente validar con el usuario si estos requerimientos siguen siendo correctos o es necesaria alguna modificación.

Una vez recolectados los requerimientos y digerido completamente el negocio, se modelan los conceptos recolectados a fin de entender el dominio del sistema y facilitar el entendimiento de la interacción entre unos conceptos y otros para todos los involucrados en el proyecto.

Análisis. El análisis de los requerimientos puede llegar a ser una etapa que requiere mucho trabajo. Es muy posible que en la etapa de extracción, alguno de los requerimientos no cumpla con las características establecidas por los estándares de calidad, así que en esta etapa se revisa uno a uno los requerimientos y sus características, se valida que estén dentro de lo establecido en la negociación del proyecto.

Es muy posible que de esta revisión surjan dudas sobre los requerimientos y sea necesario acudir con el cliente para aclararlas. Aunque es recomendable hacer

una revisión de todos los requerimientos y acordar una reuniones con el cliente para subsanar la mayor cantidad de dudas en la menor cantidad de tiempo posible.

Se analizan, jerarquizan y clasifican los requerimientos. Una vez clasificados se plantea como serán abordado, cada uno de los requerimientos y sus características, por la tecnología utilizada (aquí interviene el equipo de diseño y posiblemente de desarrollo). Es posible que la técnica aquí utilizada sea la Lluvia de ideas (Brainstorm) donde participan todos los involucrados en el proyecto. Y en ocasiones el resultado de esta etapa puede llegar a ser prototipos.

Una vez digeridos todos los requerimientos y solventadas sus necesidades por la tecnología utilizada, se procede a un modelado de los requerimientos, mediante el modelo de casos de uso. Es necesario tener un modelo de casos de uso antes de realizar la especificación, ya que en él se identifica la cantidad de casos de uso a especificar dependiendo de la funcionalidad del sistema.

Especificación. Documentar los requerimientos, requieren de una habilidad de redacción y orden por parte del analista. Es necesario entender que los casos de uso son una descripción textual de las interacciones entre un actor y un sistema para cumplir una funcionalidad determinada.

Para cubrir los marcos de calidad en esta etapa es necesario que el caso de uso cubra con las características establecidas en los estándares.

- ⇒ Están expresados desde el punto de vista del actor.
- ⇒ Se documentan con texto informal.
- ⇒ Describen tanto lo que hace el actor como lo que hace el sistema cuando interactúa con él, aunque el énfasis está puesto en la interacción.
- ⇒ Son iniciados por un único actor.
- ⇒ Están acotados al uso de una determinada funcionalidad –claramente diferenciada– del sistema
- ⇒ Deben tener un nombre único que sirva para identificar la funcionalidad especificada en él.
- ⇒ Debe contener:
 - El nombre de quien realizo la especificación y/o actualización y la fecha.
 - Los actores
 - Una descripción general de la funcionalidad
 - Precondiciones del caso de uso
 - El flujo principal de las interacciones actor – sistema

- Los flujos alternativos de las interacciones
- Poscondiciones específicas del sistema

Es muy común que los casos de uso se utilicen como acuerdo entre el cliente y proveedor, de ahí que es necesaria una clara descripción de la funcionalidad en ellos.

Validación. Al haber obtenido, entendido y documentado los requerimientos, es necesario obtener la aceptación por parte del cliente para iniciar la construcción del nuevo sistema. Aquí se presenta lo entendido y especificado por el analista al cliente a fin de autorizar la construcción del sistema con una especificación correcta.

Esta validación debe llevarse a cabo con la especificación por parte del cliente, para identificar anomalías y posibles omisiones por parte del analista. Se debe revisar de manera formal, en una reunión cada requisito especificado, tratando de revisar la mayor cantidad de requerimientos en el menor tiempo posible, pero sin descuidar los detalles.

Las técnicas más comunes para validar los requisitos son revisiones grupales, por pares y listas de comprobación; por los involucrados en el proyecto (cliente, consultor, diseñador y desarrollador)

Mantenimiento. En condiciones ideales, los requerimientos autorizados se mantienen desde el inicio hasta el final del proyecto. En la actualidad no son estáticos, por el contrario, tienen a modificarse a lo largo del desarrollo del sistema. Las principales causas de modificaciones a requisitos suelen ser que existen muchos usuarios, que el cliente directo no sea el usuario final, modificaciones en el negocio o en incluso, cambios en la tecnología.

El mantenimiento consta de dar un seguimiento a los requerimientos autorizados por el cliente. Este seguimiento suele llevarse a cabo con una matriz de trazabilidad o rastreabilidad (Duran, Bernáldez, Ruiz & Toro, 1999), que mapea cada requerimiento con los elementos asociados a él a lo largo de todo el ciclo de vida.

Esta matriz ayuda a controlar y registrar posibles modificaciones en el sistema, ya que si uno de los requerimientos cambia, se deben realizar modificaciones en cascada a lo largo del ciclo. Y a estimar el impacto de estas posibles modificaciones al sistema, tanto en tiempo como en recursos.

La información para rastrear los requerimientos debe contener quién propuso el requerimiento, quién lo autorizo, si existe un vínculo con otros requisitos o es independiente y caso de uso al que pertenece, entre otros aspectos.

Es necesario contar con políticas de modificación previamente especificadas en la negociación o contrato. El control de cambios se asegura que la modificación se incorpore al sistema y de forma correcta, previene y ajusta la modificación para cumplir con las necesidades.

Se debe tener siempre en cuenta que permitir innumerables modificaciones a los requerimientos implica una gran cantidad de tiempo desperdiciado, de ahí la necesidad del análisis de los requerimientos, para prever posibles modificaciones.

Tabla 3 . Cuadro de síntesis de las etapas de análisis de requerimientos

ETAPA	ACTIVIDADES
Comprensión del dominio	Estudio y comprensión del dominio
Extracción	Recolección de Requerimientos
	Modelo de dominio o conceptual
Análisis	Estudio del alcance de los requerimientos
	Identificación de problemas y búsqueda de soluciones
	Aclaración de dudas y acuerdos con el usuario
	Modelo de Casos de uso
Especificación	Documentación de requerimientos mediante casos de uso
	Diseño de prototipos.
Validación	Aprobación de los requerimientos especificados en un documento.
Mantenimiento	Seguimiento a los requerimientos
	Análisis de alguna modificación si es que existe (alcance, identificación de problemas y soluciones, aclaración de dudas y acuerdos, modificación al modelo) en tiempo y costo.
	Diseño de matriz de rastreabilidad y control de versiones
	Solución de dudas al resto del equipo de trabajo

1.2.1. Problemas en la administración de requerimientos

Estudios del Standish Group de 1997 al 2000 revelan que las principales causas para el fracaso de un proyecto de software corresponden a la administración de los requerimientos, cuyos principales defectos son:

- ⇒ Incapacidad para manejar los cambios en los requerimientos durante el desarrollo.
- ⇒ Falta de especificación detallada de los requerimientos.
- ⇒ Mala organización y control de requerimientos.
- ⇒ Requerimientos mal entendidos.

De ahí que es necesario no sólo tener un claro proceso para administrar los requerimientos, sino aplicar cada etapa. A simple vista, pudiera parecer un proceso innecesario, pero tener claramente ordenada la información nos permite un manejo más fácil. Ahorramos mucho tiempo en las siguientes etapas del ciclo de vida en cosas tan “sencillas” –que a veces no lo es tanto- como resolver una duda, hasta manejar de forma clara y sin errores una modificación. Una clara administración de los requerimientos, claramente detallada, nos permite darle una mayor oportunidad al proyecto para llegar a su fin, no olvidemos que existen otros factores que intervienen en la entrega del producto, dependiendo de la etapa del ciclo de vida en la que se encuentren.

2.4. Artefactos para evaluar la calidad de los requerimientos

Si bien ya se conocen las características con las que deben contar los requerimientos es necesario que se conozcan algunos artefactos o herramientas que nos permitan evaluar la calidad de su especificación, ya que a partir de esta será construido el sistema.

Bocanegra (2011) enumera siete herramientas básicas de calidad. A estas herramientas se les conoce como herramientas de calidad, de estadística, de administración, de la mejora continua, etc. Las herramientas más utilizadas para la validación de calidad son:

1. La lista de chequeo (verificación) – Checklist.

Una lista de chequeo o Checklist por su definición en inglés, consiste en un documento que enuncia las características principales que debe cubrir un elemento a fin de asegurar su calidad. Este artefacto, tiene la ventaja de ser utilizado para evaluar la calidad de distintos productos así como una elaboración fácil y tan detallada como sea necesario. Mancera (2008),

menciona que existen muchos formatos de listas de chequeo específicos para diversas actividades productos, sin embargo, no existe un formato estándar o una lista ya definida que pueda evaluar todos los elementos, es posible tomar como base un modelo, sin embargo, cada lista de chequeo debe ser particular para una actividad o producto específico.

Al elaborar una lista de chequeo es recomendable contemplar los siguientes puntos:

1. Identificar para que se elaborará la lista de chequeo. Lo primero es definir que se va evaluar con la lista, es decir, cual es el objetivo. Puede ser un punto en específico o un conjunto de características que definen un resultado. Todo artefacto, debe tener un objetivo que ayude al desarrollo de un buen producto o asegure el resultado de una tarea y la lista de chequeo no es la excepción.
2. Definir quién elaborará la lista. Una vez que se ha identificado el criterio o los criterios a evaluar, se debe consultar a un elemento con el suficiente conocimiento y experiencia para definir cada uno de los criterios a evaluar, esto de acuerdo al objetivo de la lista. Si se desea construir una lista de chequeo para el motor de un avión, se debe consultar a un especialista en mecánica de aviación con la experiencia suficiente para saber en bajo qué condiciones el motor podría fallar o no.
3. Ubicar la actividad o el producto en su proceso común. Una vez identificado el punto a evaluar y quien realizará la lista de verificación, se debe recorrer el flujo común del elemento con el fin de identificar todos los riesgos que pudieran desviarlo de su objetivo. El conocimiento y la experiencia de quien elabora la lista, le facilita identificar los puntos más críticos en el proceso.
4. Analizar toda la información e ir haciendo un listado describiendo los puntos que conformarán la lista de chequeo.
5. Documentación del Check list. Una vez obtenida toda la información del producto en su proceso común se deben documentar todos los puntos que formarán el check list y se deben proporcionar los niveles o criterios de evaluación y/o aceptación. Todo esto debe ser dentro de un formato que permita una validación rápida, una retroalimentación y análisis final de la información, así como una guía que facilite el conocimiento del contenido.

Proyecto:		
Área	Fecha:	
Responsable:		

Puntos de control	Criterios		Comentarios
Punto 1	A ☐	B ☒	
Punto 2	A ☒	B ☐	
Punto 3	A ☒	B ☐	
Punto 4	A ☒	B ☐	
Punto 5	A ☐	B ☒	
Punto 6	A ☒	B ☐	
Punto 7	A ☒	B ☐	
Punto 8	A ☒	B ☐	
Punto 9	A ☐	B ☒	
Punto 10	A ☒	B ☐	

Figura 3. Ejemplo Checklist

2. Diagramas de Pareto.

El diagrama de Pareto está enfocado en las causas de los defectos, de acuerdo a su prioridad, es decir, se orienta a aquellas características que al ser previstas o solucionadas tendrán un mayor impacto en el resultado esperado. Este diagrama, permite visualizar e interpretar aquellas características que tienen más relevancia basándose el principio “pocos vitales, muchos triviales” que se interpreta con la regla 80-20, donde el 80 corresponde a muchos problemas sin importancia y el 20 a problemas que realmente son graves.

El diagrama de Pareto puede ser utilizado para monitorear elementos de diferentes aspectos. En este caso se enfoca como un monitoreo de elementos de riesgo para la calidad de un producto, basándose en la prioridad de los requerimientos proporcionados. Como ya se ha mencionado, el 80% corresponde a los requerimientos con prioridad más bajos y el 20% a los requerimientos de prioridad más alta. Dando un porcentaje proporcional a cada requerimiento. Esto nos permite organizar la atención a cada uno de acuerdo a su nivel de importancia o afectación en el proyecto.

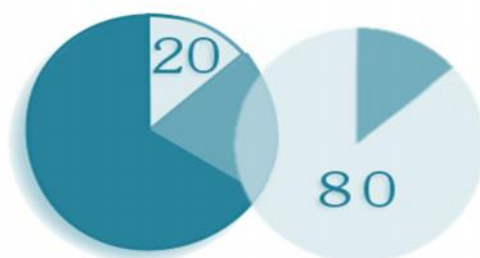


Figura 4. Gráfico de la teoría de Pareto

El diseño de este diagrama requiere fundamentos de probabilidad y estadística ya que principalmente utilizan frecuencias y errores como unidades de medición, sin embargo, puede usar otras variables. Basado en los puntos generales mencionados por la química Cravioto (2008) se describirá la construcción de este diagrama.

- a. Definir el problema a evaluar.
- b. Escoger los elementos que serán monitoreados, comparados y clasificados, estas causas pueden obtenerse con una lluvia de ideas. Es recomendable contar con algún recurso o elemento del equipo de trabajo que tenga experiencia en proyectos similares, ya sea en la planeación o contexto del negocio.
- c. Escoger las unidades de medición que serán utilizadas como criterios de medición, generalmente suelen ser el tiempo, la frecuencia de ocurrencia, el costo, esfuerzo o errores.
- d. A diferencia de solucionar un problema, este caso infiere en prevenirlo y por tal motivo es necesario definir periodos de evaluación a lo largo de todo el proyecto mantenerlo en monitoreo.
- e. Obtener las características de los requerimientos.
- f. Documentar las características de cada requerimiento dentro de una tabla reflejando las unidades seleccionadas como criterios de evaluación y su porcentaje del total con respecto a todos los demás.
- g. Comparar la frecuencia relativa de cada categoría de problema
- h. Distribuir en una gráfica los elementos del problema categorías del problema. En un eje las unidades de medición del problema, iniciando el eje por aquella que tiene el valor más alto del porcentaje y terminando con el de menor porcentaje. El otro eje el porcentaje de 0 a 100.
- i. Dibujar las líneas del porcentaje acumulado. Verificar que las primeras causas contribuyen en porcentaje mayor que todas las demás causas

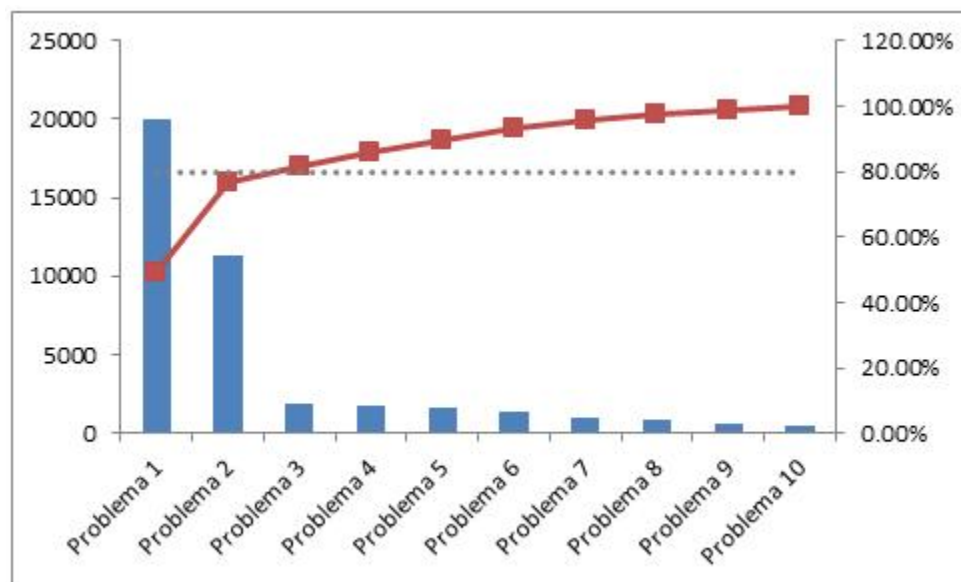


Figura 5. Gráfica de análisis de Pareto

3. Histograma.

El histograma es una representación gráfica de la variación dentro en un proceso. Es una herramienta muy útil para visualizar los progresos o desviaciones que puede tener el proyecto, variable por variable y periodos de tiempo.

La elaboración de un histograma es muy similar a la de un diagrama de Pareto, ya que la información se representa mediante gráficos. Este artefacto tiene una estrategia de diseño más general pero sus conceptos de probabilidad y estadística pueden llegar a ser más profundos.

Elaboración de la Tabla de Frecuencias y el Histograma

- Recopilar los datos que formaran los criterios de evaluación
- Determinar la unidad mínima de medición de los datos.
- Encontrar el valor máximo ($X_{\max}$) y mínimo ($X_{\min}$) de los datos.
- Calcular el "Intervalo de clase el cual ser múltiplo de la unidad mínima de medición.

$$c = \frac{X_{\max} - X_{\min}}{k}$$

Donde k = es el número de Clases (número de barras en la gráfica). Se calcula como:

NÚMERO DE DATOS	VALOR DE “k”
50 - 100 aprox.	6 - 10
100 – 250	7 - 12
más de 250	10 – 20

Siendo n = número de datos.

- e. Se calcula el punto medio de cada Intervalo de Clase:

$$\text{Punto Medio Clase} = \frac{\text{Límite Superior de Clase} + \text{Límite Inferior de Clase}}{2}$$

- f. Se llena la Tabla de Frecuencias como sigue:

No. CLASE	PUNTO. MEDIO	MARCAS	FRECUENCIA
1			
2			
..			
n			
Total			

- g. Se llena la tabla y se va marcando la Clase donde corresponde cada dato.
 h. Se suman las marcas y se determina la Frecuencia de cada Clase
 i. Se hace una Gráfica de Barras, en donde el eje de “X” representa los valores de medición (las Clases), y el eje “Y” la frecuencia. Esta gráfica se llama Histograma.

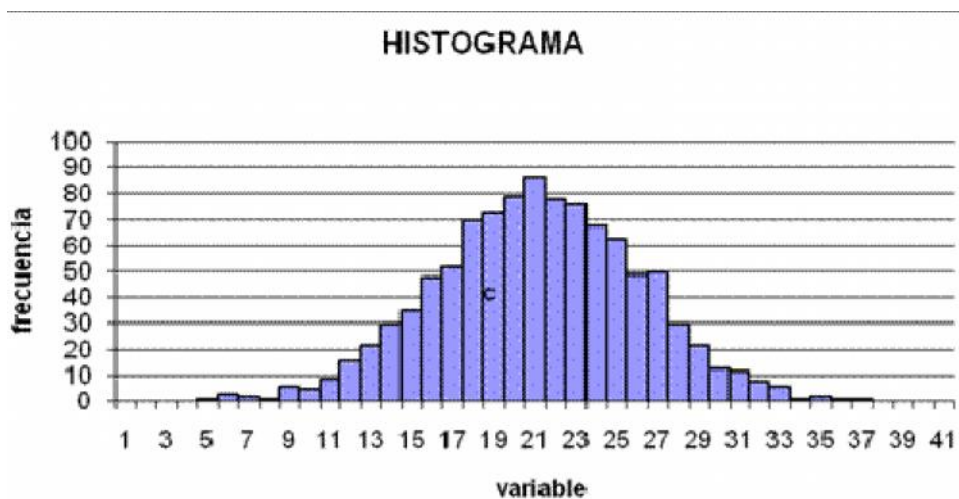


Figura 6. Histograma o gráfica de barras

4. Gráfica de corrida – grafica de control.

La calidad de un producto debe contener elementos que permitan evaluar, tanto su proceso como el resultado final. Oliveros, define la gráfica de corrida como una curva de datos puntuales ordenados en el tiempo. Este diagrama brinda la posibilidad de reconocer las causas y efectos en un problema y las relaciones entre estos o en otras palabras, permite visualizar la conducta de una variable bajo distintas condiciones. En él se pueden detectar tendencias o ciclos de una característica ya que es posible determinar causas especiales de variación.

La gráfica de corrida no tiene límites de control, sin embargo es útil para analizar los datos, especialmente en la etapa de desarrollo de un producto o previamente al estado de control estadístico. Los puntos correspondientes a los datos se grafican por número de orden de producción. El graficar los puntos correspondientes a los datos es una manera muy efectiva de obtener información sobre el proceso. Este método, por lo general se complementa con los diagramas de control, para mejorar el análisis de los resultados obtenidos.

La interpretación de la gráfica se deberá hacer de acuerdo con la característica que se desee controlar, ya que ésta por sí misma nos da una información sobre el comportamiento del proceso y no sirve para llevar un control específico del mismo. El diseño de esta gráfica, con rangos de control, no es tan complicado como la anterior.

- Lo primero es determinar la característica a evaluarse.
- Lo segundo es definir el periodo de tiempo durante el cual se obtendrán las muestras.
- Posterior a esto se debe recolectar la información por corridas durante el periodo de tiempo definido.

- d. Calcular los límites de control
- e. Identificar las desviaciones del producto.

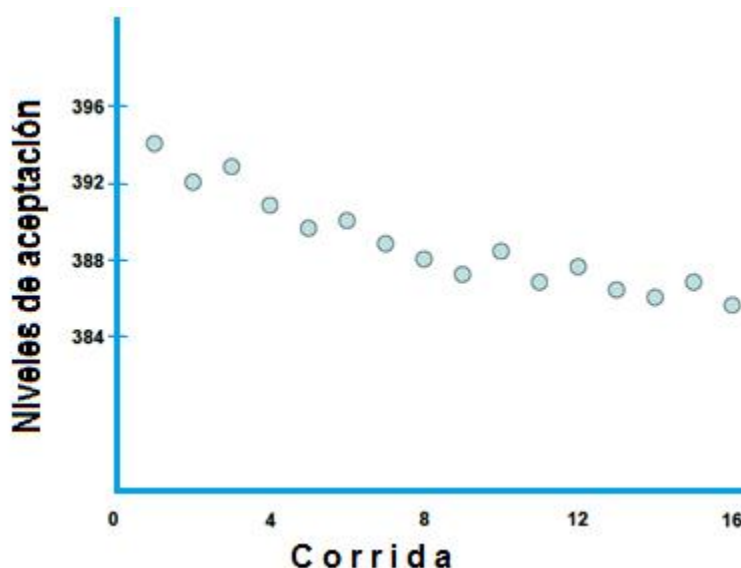


Figura 7. Gráfica de corrida

Una gráfica de corrida limitada por elementos de control permite conocer la evolución de las características de un proceso a partir de sus variaciones y corregir el flujo antes de generar un error.

Las gráficas de control muestran las causas especiales de variación. Conocer la naturaleza de las características permite detectar causas especiales de la variación. Existen varios gráficos de control, el enfoque de este estudio es sólo dar a conocer algunos de estos gráficos debido a la extensión del tema.

- Gráficos de control para atributos
- Gráfico de Control pn.
- Gráfico de Control p.
- Gráfico de Control c.
- Gráfico de Control u
- Gráficos de control para variables
- Gráficos X-R, Gráficos R

5. Diagrama de Correlación – Scattergram

Llamado también diagrama de correlación, pretende determinar si existe reciprocidad entre dos características de un proceso. En unos ejes de coordenadas se sitúa en abscisas la escala correspondiente a una de ellas (por ejemplo, la velocidad de corte de una determinada pieza) y en ordenadas la correspondiente a la otra (por ejemplo, la rugosidad en la superficie de corte). En cada realización del proceso se miden los valores x_i e y_i de ambas características, los cuales definirán un punto en el plano antes definido. Tras una serie de medidas se tendrá una nube de puntos de cuya estructura podría deducirse si existe o no correlación y si ésta es positiva o negativa. Las figuras siguientes reflejan dichas posibilidades.

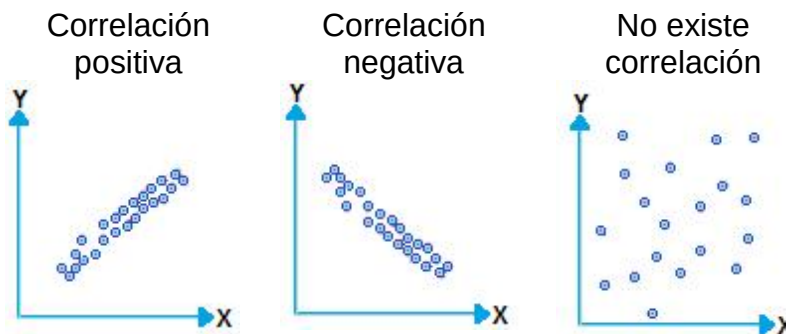


Figura 8. Diagramas de correlación

La ventaja de esta técnica es que permite comprobar cómo la aplicación de una serie de mejoras en un sentido se corresponde con otras apreciadas en otra variable. Como ejemplo, se puede emplear en la demostración de la relación existente entre las causas y efectos del diagrama causa-efecto.

6. Diagrama de Ishikawa

El diagrama de Ishikawa (en honor a su autor Kaoru Ishikawa), también es conocido como diagrama de causa y efecto o espina de pescado. Este diagrama describe gráficamente las causas y efectos de los elementos que conforman un problema. La elaboración de estos diagramas es muy sencilla y facilita el entendimiento de aquellos elementos que puedan resultar un riesgo para el desarrollo de un buen producto.

- Se deben definir las características a analizar
- Se clasifican las causas, por lo general la clasificación se basa en las 4M, materiales, métodos, máquinas y medias, sin embargo en la actualidad se han agregado dos causas más, mano de obra y medio ambiente.
- Ubicar del lado derecho del diagrama los efectos y de la izquierda las causas.

Como se puede observar en todos los puntos anteriores, existe una amplia serie de recomendaciones para la elaboración de los artefactos de evaluación de calidad, sin embargo se deben resaltar tres que son fundamentales. El primero es que el artefacto sea elaborado por todos los involucrados, el segundo es que exista un especialista de cada área y el tercero es que el enfoque sea directamente en el problema y no en desviar el tema en tropiezos ajenos.

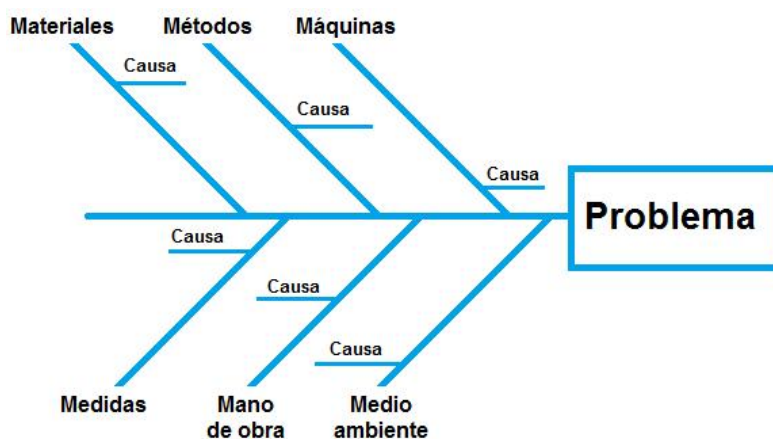


Figura 9. Diagrama de Ishikawa

2.4.1 Técnicas de validación de requerimientos



Figura 10. Proceso de validación de requerimientos

La validación en la etapa del análisis, tiene por objetivo comprobar que los requerimientos especificados son los que realmente requiere el sistema y son los solicitados por el usuario. Esta etapa de validación debe tener como entrada el documento de requisitos, los estándares relacionados y el conocimiento de la organización, y como salida se obtiene una lista de problemas y una lista de acciones recomendadas.

Algunas técnicas para la validación de requisitos propuestos por autores como Duran, Bernáldez, Ruiz y Toro (1999) entre otros. Estas técnicas son:

7. Revisiones de requerimientos (Reviews o Walk-throughs): Está técnica consiste en la lectura y corrección de la completa documentación o modelado de la definición de requisitos. Con ello solamente se puede validar la correcta interpretación de la información transmitida. Más difícil es verificar consistencia de la documentación o información faltante.
8. Auditorías: La revisión de la documentación con esta técnica consiste en un chequeo de los resultados contra una checklist predefinida o definida a comienzos del proceso, es decir sólo una muestra es revisada.
9. Matrices de trazabilidad: Esta técnica consiste en marcar los objetivos del sistema y chequearlos contra los requisitos del mismo (Durán, Bernáldez, Ruíz & Toro, 1999). Es necesario ir viendo qué objetivos cubre cada requisito, de esta forma se podrán detectar inconsistencias u objetivos no cubiertos.
10. Construcción de prototipos: Algunas propuestas se basan en obtener de la definición de requisitos prototipos que, sin tener la totalidad de la funcionalidad del sistema, permitan al usuario hacerse una idea de la estructura de la interfaz del sistema con el usuario (Olsina, 1999). Esta técnica tiene un problema, el usuario debe entender que lo que está viendo es un prototipo y no el sistema final.

2.5. Modificaciones a los requerimientos

Como se dijo con anterioridad, los requerimientos son dinámicos, es decir, tienen cambios de manera constante. Pero es necesario tener la menor cantidad de modificaciones ya que de esto depende el tiempo de desarrollo y una pronta entrega del producto final.

Los cambios en los requerimientos son un punto muy delicado que deben tratarse con cuidado ya que dejarle la carta abierta al cliente para modificaciones es poner en riesgo el proyecto mismo.

Es importante recordar que las afectaciones a los requerimientos son progresivas en cascada, es decir, la modificación de un requerimiento en el análisis puede afectar a 2 o más módulos del diseño, posteriormente a n clases del desarrollo y n

casos de prueba, de forma progresiva. Dentro de todo esto debemos estudiar el tiempo y esfuerzo extra que implica la modificación y costeo.

La entrada de documentos para hacer una modificación es la especificación del requerimiento, la ERS y el control de cambios y el contrato o acuerdo de la negociación.

2.5.1 Análisis de la modificación.

Cuando es solicitada una modificación por parte del cliente, se debe entender a qué se debió la misma y cómo será la modificación, es posible que sólo sea una modificación parcial o puede ser una modificación total del requerimiento. Incluso, Eliminar uno de los requerimientos, se puede considerar una modificación, ya que a pesar de que será menos trabajo para el desarrollador, se debe analizar si existe relación con otros requerimientos y en qué aspecto afecta a la ERS.

Se analiza que el nuevo requerimiento contenga todas las características especificadas por los estándares. En este punto se debe tener mucho cuidado en vigilar que la modificación al requerimiento no supere el alcance previamente establecido del convenio.

Una vez realizado este análisis, si no hay afectación con otros requerimientos y se aceptara la negociación, en tiempo y costo, se debe especificar el cambio en el caso de uso y registrar en el control de cambios. Lo siguiente es validar nuevamente con el usuario el requerimiento y esperar nuevamente la autorización para llevar a cabo el cambio.

2.5.2 Análisis del cambio en costeo

Es necesario entender el costo que pueden tener las modificaciones de los requerimientos. Por lo general, las estimaciones más importantes se llevan a cabo en la etapa de la negociación. Cuando surge una modificación, se evalúa la modificación.

2.5.3 Autorización del acuerdo e Implementación del cambio

Un punto muy importante en la autorización de los casos de uso y que no se debe olvidar, es que no se debe realizar ninguna construcción ni modificación al sistema, si este no ha sido autorizado por ambas partes, el cliente y el proveedor.

No siempre las modificaciones surgen por parte del cliente. En algunas ocasiones, es posible encontrar defectos en los requerimientos en alguna de las etapas posteriores. Esta, no se debe corregir y programar de acuerdo a lo que el desarrollador cree que debe funcionar. Hacer esto es programar algo diferente a lo especificado y por lo tanto ya no cumple con lo solicitado, este defecto se debe notificar al analista para rectificar si no es una excepción dentro del sistema o si efectivamente se requiere una modificación. Si este fuera el caso, es deber del analista acudir con el cliente para hacer de su conocimiento la necesidad del cambio y su autorización.

2.5.4 Problemas por ausencia de administración de cambios.

En algunas empresas resulta tedioso llevar un registro de los cambios realizados en los requerimientos, pues se cree que sólo se sustituyen unos por otros, sin embargo, la necesidad de tener un control de estos cambios crece con el sistema mismo.

Si se autorizan cambios sin tener un control de estos, solo se realizan modificaciones constantes y sin control al sistema corriendo el riesgo de que horas antes de entregar el sistema se autorice una nueva modificación. O que las constantes modificaciones se pierdan a lo largo del ciclo y las pruebas no coincidan con el desarrollo.

A pesar de ser muy vaga la idea, sucede de una manera más común de lo imaginable. Tener un control de estas modificaciones nos permite limitar de algún modo al cliente y mantener a todo el equipo informado de ellas.

REFERENCIAS BIBLIOGRAFICAS

- Presman, Roger S. Ingeniería de Software (1993). Un enfoque práctico. Editorial McGraw-Hill. Tercera edición
- Weitzenfeld, Alfredo. (2005) Ingeniería de software orientada a objetos con UML, java e internet. Editorial Thomson.

- Cravioto Ruelas, Martha Alicia. 2008. Propuesta metodológica para evaluar la robustez de los procesos de validación en una empresa farmacéutica. UNAM, Facultad de Química,

REFERENCIAS ELECTRONICAS

- Chiosso, Hernan (2008). The Chaos Report. CBASQA – Desarrollo de Software, SQA, Testing, Servicios Informáticos, Project Management. Recuperado de <http://cbasqa.wordpress.com/2008/09/16/the-chaos-report/>
- ISO/IEC 9126, (2009). de <http://iso25000.com/index.php/iso-iec-9126.html>
- Cueva Lovelle, Juan Manuel (2001). Calidad del software. Factores que determinan la calidad de un producto Software. www.di.uniovi.es/~cueva/assignaturas/doctorado/2001/Calidad.pdf
- Flores Mejía Fernando, Mora T. Manuel, Álvarez R. Francisco, Fernández B. Jesualdo (2006). Ingeniería de requerimientos: Clasificación de requerimientos y desarrollo de un proceso genérico para la ingeniería de requerimientos. Recuperado de http://www.uaa.mx/investigacion/memoria2006/pdf/ingenierias/Flores_Mejia_Fernando.pdf
- Flores Mejía, F.(2006). INGENIERÍA DE REQUERIMIENTOS: CLASIFICACIÓN DE REQUERIMIENTOS Y DESARROLLO DE UN PROCESO GENÉRICO PARA LA INGENIERÍA DE REQUERIMIENTOS. (Noviembre, 2006). Recuperado de: <http://www.uaa.mx/investigacion/memoria2006/area1.htm>
- alarcos.inf-cr. Tema06.Ingeniería del software. (2005).Recuperado de www.ie.inf.uc3m.es/grupo/docencia/reglada/psi/unidad8-DOC.pdf
- Adrián, (2008). Administración de requerimientos. Recuperado de <http://clases3gingsof.wetpaint.com/page/Administraci%C3%B3n+de+Requerimientos>
- 2008. Unidad I. Procesos de la ingeniería de Requerimientos. De: <http://www.slideshare.net/quest409adc/unidad-i-requerimientos-presentation>
- Monografias.com. Ingeniería de requerimientos Ingeniería de Software <http://www.monografias.com/trabajos6/resof/resof.shtml#inge>
- Thompson ,Ivan (2008).Definición de administración. De <http://www.promonegocios.net/administracion/definicion-administracion.html>
- Mena Mendoza Gonzalo (2005). Procesos de la Ingeniería de Requerimientos. Recuperado de http://mena.com.mx/gonzalo/maestria/ingreg/presenta/procesos_ir/
- Arias Chávez, Michael (2007).La ingeniería de requerimientos y su importancia en el desarrollo de proyectos de software. Recuperado de http://www.latindex.ucr.ac.cr/interdes10/10-art_11.pdf
- Standish Group. 2000. Recuperado de http://macroos0.tripod.com/n_de_requerimiento3.htm
- Durán, Bernáldez, Ruíz &Toro, 1999. Recuperado de http://www.ecured.cu/index.php/Validaci%C3%B3n_de_Requisitos#T.C3.A9cnicas_de_Validaci.C3.B3n_de_Requisitos
- Bocanegra Bustamante, Carlos Alberto. (2011) Control estadístico de la calidad – CEC-CEP. Recuperado de: mba.americaeconomia.com/sites/mba.../herramientasesdisticas.ppt
- Mancera Ruiz, Mario Ramón. 2008. Lista De Chequeo. <http://www.manceras.com.com> , <http://www.manceras.com.co/artchecklist.pdf>
- Graficas de control. Dirección General de Planeación y desarrollo en salud. (2013). http://www.dgplades.salud.gob.mx/descargas/dhg/GRAFICAS_CONTROL.pdf
- Oliveros, Miguel. MÉTODOS ESTADÍSTICOS PARA EL CONTROL DE CALIDAD. (2013). http://webdelprofesor.ula.ve/economia/oliverosm/materiasdictadas/produccion2/metodos_estadisticos_para_el_control_de_la_calidad.pdf
- Sitios recomendados http://www.slideshare.net/la_pampa/herramientas-de-calidad-2179807
- Sitios recomendados www.revistas.unal.edu.co/index.php/avances/article/.../9982/10514
- Sitios recomendados <http://www.overti.es/software/analizador-calidad-de-requisitos-rqa.a>

CAPITULO 3. INGENIERÍA INVERSA

Actualmente las empresas cuentan con varios sistemas ya desarrollados, el problema inicia cuando dichos sistemas requieren de alguna modificación y la persona que los desarrollo ya no se encuentra en ella. Por lo cual, la tarea se vuelve muy complicada para el nuevo analista. En algunos casos lo más complicado en el desarrollo de los sistemas en la actualidad es que muchos ya se encuentran desarrollados y corresponde al nuevo analista recuperar todo aquello que ya existía en otro sistema. En algunos casos ni siquiera existe una documentación previa, por lo que el código o el ejecutable se vuelven la única fuente de información y es requerida una mejora o mantenimiento en el sistema. En estos casos es cuando se recurre a una ingeniería inversa.

La mejor definición para ingeniería inversa es ir de lo general a lo particular.

3.1. INTRODUCCIÓN A LA INGENIERÍA INVERSA

Es naturaleza del ser humano entender el entorno que lo rodea. Sus principales inventos se basan en una imitación de la naturaleza abstrayendo sus características más importantes y adaptándolas a sus necesidades. La ingeniería inversa es algo muy similar a esto, un proceso de entendimiento de un producto de la ingeniería, a partir de sus características más superficiales hasta las más fundamentales, para entender la razón de su funcionamiento y los elementos que lo componen.

Tratar de dar un marco histórico a la ingeniería inversa nos remontaría hasta la historia más antigua del hombre, donde se imitaba y perfeccionaba algunos inventos de otros o de la naturaleza misma, para asegurar su supervivencia. Evitando entonces este marco tan largo de la historia, se plantea que la ingeniería inversa tiene sus fundamentos más importantes en la II Guerra Mundial, donde se buscaba entender y mejorar el armamento de los países adversarios.

Posterior a esta época, se ha venido practicando este proceso en infinidad de países con el fin de encontrarse en la punta de la evolución tecnológica. A pesar de estar prohibido en muchos aspectos, por considerarse un atentado contra los derechos de autor, este proceso se ha abierto paso gracias a su utilidad para un proceso de mejora de los productos (reingeniería), no obstante no es su única utilidad.

El uso de este concepto en el desarrollo de software se fortaleció alrededor de los 80's, cuando se da énfasis a la crisis del software y es necesario reestructurar los sistemas existentes. Para esta época, los sistemas no se encontraban documentados o su documentación era muy pobre y se necesitaba entender su código fuente para poder reestructurar sus necesidades.

En la actualidad existen muchas definiciones de ingeniería inversa, para algunos es un proceso de recuperación del diseño (Pressman, 1993), es cierto que este concepto se puede utilizar en diferentes ramos del conocimiento, pero para propósitos en este estudio, se propondrá la definición de ingeniería inversa como el proceso de entendimiento de un producto, con el fin de obtener todas las características que lo conforman.

Una definición más formal para la ingeniería inversa de los sistemas sería: proceso de entendimiento mediante el análisis y la abstracción de los elementos que conforman el sistema, permiten su funcionamiento y justifican su existencia.

Las herramientas más importantes de este proceso son básicamente dos, el análisis y la abstracción. El análisis es la base de cualquier sistema, independiente

a si es un sistema de información o de cualquier otro tipo. La razón es muy sencilla. Todo sistema tiene un propósito: resolver un problema, pero ¿cómo se podría hacer esto, sin tener bien claro el problema mismo? Es aquí donde el análisis entra en juego. Entiende el problema y todos los factores que le rodean a fin de encontrar la mejor solución y ajustarse a las posibilidades que pueden ser provistas.

Para el desarrollo de los sistemas de información, el análisis es la mayor inversión de cualquier recurso, ya que si bien, sólo se tiene un analista, el nivel de conocimiento que se debe poseer es alto, y dependiendo del problema será el tiempo para encontrar la solución. Sobre este tema se hablará a fondo en el siguiente capítulo, mencionando varias definiciones del análisis y los métodos de administración para la información obtenida de estos.

3.2. INGENIERÍA INVERSA Y DISEÑO TRADICIONAL

Inicialmente, la ingeniería inversa es un proceso costoso, debido a la complejidad de sus etapas, sin embargo, los beneficios que pueden llegar a obtenerse son aún mayores. Bien aplicado este proceso permite desarrollar un producto con mayores beneficios, principalmente en el marco de calidad.

Un claro ejemplo de los beneficios de este proceso es Japón quien ahora es una potencia tecnológica. Weitzenfeld (2005) menciona que los fundamentos de la tecnología japonesa, se basa en lo que algunos llaman imitación de otros productos. En un inicio esto pudo ser cierto pero su depuración de los productos y su constante uso de este proceso, les ha permitido identificar, cada vez con mayor facilidad, las deficiencias en los elementos analizados y mejorar e incrementar la calidad en las nuevas producciones.

Algunas empresas, prefieren hacer un diseño tradicional en lugar de una ingeniería inversa y esto es comprensible ya que los costos en una ingeniería inversa pueden ser superiores, sin embargo, los beneficios que se pueden obtener de este proceso, pueden superar por mucho los costos iniciales. Independiente de conocer un sistema, la ingeniería inversa, nos brinda beneficios que pueden ser utilizados de diversas formas según sea la necesidad (Sicilia, 2009). Algunos beneficios mencionados en la misma obra de Kendall (1991) son:

- ⇒ Reducción del tiempo requerido para el mantenimiento del sistema, con lo cual queda más tiempo para nuevos desarrollos.
- ⇒ Se genera documentación, que podría haber sido inexistente o mínima en los programas anteriores.

- ⇒ Se crean programas estructurados a partir de código de computadora no estructurado o pobremente estructurado.
- ⇒ Los cambios futuros al mantenimiento son más sencillos, por que se pueden realizar al nivel del diseño más que al nivel del código.
- ⇒ Es posible analizar el sistema con el fin de eliminar algunos módulos o parte de estos, sin utilizar de código de computadora, el cual aun podría estar presente en programas anteriores a pesar de que las revisiones hechas al programa a lo largo de los años lo hayan vuelto obsoleto.
 - Reducir la complejidad del sistema.
 - Generar diferentes alternativas.
 - Recuperar y/o actualizar la información.
 - Detectar efectos laterales.
 - Facilitar la reutilización.

Algunos de los resultados obtenidos de la ingeniería inversa son:

- ⇒ Estructura de datos y elementos que describen los archivos y registros almacenados por el sistema.
- ⇒ Diseños de pantallas, si el programa es en línea.
- ⇒ Esquemas de enfoques para programas por lotes.
- ⇒ Un diagrama estructural que muestra la jerarquía de los módulos del programa.
- ⇒ Diseño y relaciones de bases de datos.

3.3. Proceso de la ingeniería inversa

Es muy común que al escuchar el concepto ingeniería inversa, lo primero que pase por nuestra mente sea una reingeniería. A pesar de que estos procesos se encuentran íntimamente entrelazados, debemos recordar que son conceptos diferentes. La ingeniería inversa es necesaria para un proceso de reingeniería, sin embargo también suele utilizarse como apoyo para el mantenimiento de un sistema o como una herramienta de reciclaje (este reciclaje puede ser de código, diseño, proceso, etc.).

Dependiendo del enfoque que se de a este proceso, la ingeniería tiene la siguiente propuesta (Sicilia, 2009):

- ⇒ Ingeniería inversa de datos: Es el proceso aplicado a un código de bases datos (aplicación de almacenamiento de información, código SQL o cualquier repositorio de datos) para obtener las relaciones entre la información (modelos relacionales) y modelar un diagrama entidad-relación.
- ⇒ Ingeniería inversa de lógica o de proceso: Es el uso de la ingeniería inversa en un código de programa para averiguar su lógica, diseño detallado o arquitectura del sistema; para obtener documentos de análisis o de requisitos.
- ⇒ Ingeniería inversa de interfaces de usuario: Proceso de la ingeniería inversa aplicado para obtener los modelos y especificaciones que sirvieron de base para la construcción de un sistema con objeto de mantener la lógica interna del programa y permitir la modificación de la interfaz con el usuario (UI por sus siglas en ingles).
- ⇒ Ingeniería inversa de protocolos de comunicación (IISA): Es el análisis del comportamiento, tramas y protocolos de comunicación, observar si el sistema se está comunicando con otros, de ser así, de qué manera lo está haciendo. Y así poder implementar un nuevo sistema, replicas o interfaces.

Dado que es una labor estratégica muy costosa, es conveniente conocer cuándo realizar la tarea de reingeniería para una aplicación y cuándo es más rentable sustituirla e implementar una nueva. Algunos puntos que nos pueden ayudar a identificar un buen momento para el uso de la ingeniería inversa, con la posibilidad de derivar en una re ingeniería son las siguientes:

- ⇒ Falta de documentación del sistema.
 - El sistema actual tiene fallos constantes y difíciles de localizar.
 - El mantenimiento del sistema consume muchos recursos.
- ⇒ Sistema carente de calidad.
 - El tiempo de respuesta del proceso es demasiado largo, por lo cual en vez de agilizar el proceso lo entorpece.
 - La pérdida o robo de información es constante y provoca inseguridad del sistema
 - El software carece de un buen nivel de calidad.
 - El sistema tiene deficiencias, pero cumple la función deseada.
 - Existen dificultades para realizar pruebas

- ⇒ El sistema requiere de una modificación o reingeniería.
 - El sistema tiene dificultades en la integración con nuevos sistemas.
 - El sistema carece de flexibilidad para adaptarse a modificaciones.
 - Pocas personas capacitadas para realizar modificaciones.
 - Es necesario incluir nuevos requisitos, pero los básicos se mantienen.

En la actualidad no existe una estructura formal para llevar a cabo el proceso de la ingeniería inversa, sin embargo, en el flujo de los pasos para ir de lo particular a lo general es constante. Para este estudio, nos referimos a la propuesta realizada por Coronado en su tesis “Ingeniería inversa de base de datos” para las etapas de la ingeniería inversa. Su estudio nos permite plantear el proceso de la siguiente manera:

- ⇒ Análisis de la funcionalidad.
- ⇒ Lectura del código fuente.
- ⇒ Restructuración del código fuente.
- ⇒ Depuración del código fuente.
- ⇒ Abstracción del sistema.
- ⇒ Documentación del sistema.

Cada una de las etapas es un proceso en sí, que puede llegar a tener un nivel de dificultad, dependiendo del tipo y dimensión del sistema. Afortunadamente, en la actualidad se cuenta con herramientas que nos facilitan el trabajo. Para mayor detalle, estas se encuentran descritas en el punto 3.4 del capítulo actual.

3.3.1. Análisis de la funcionalidad

El análisis es descomponer un todo en sus partes. Pressman (1993) describe algunos principios para realizar el análisis de software. Este se podría considerar el punto más importante en el proceso, ya que es aquí donde se inicia la descomposición del sistema. Es un proceso parecido al que llevan a cabo los niños, al estudiar un juguete, Su primer paso es conocer cómo funciona a simple vista, como se comunica con el exterior y que respuesta puede tener ante los estímulos externos. El objetivo es saber **¿qué** hace el sistema?

Lo primero para entender un sistema, cuando no existe documentación es conocerlo, identificar todos los componentes que integran al sistema y las relaciones entre estos (Suger, 1997). Algunas empresas inician este análisis obteniendo las necesidades reales del usuario y un modelado de sus procesos,

esto es con el fin de desarrollar una hipótesis que describa las características que debe poseer el sistema.

Otro método utilizado en las empresas actualmente son las pruebas de caja negra. Su objetivo en este aspecto es conocer los datos de entrada y salida del sistema, así mismo, y entender su funcionalidad como el objetivo de su desarrollo y descubrir cuales realmente cumplen con las necesidades reales del usuario y cuales están fuera de las expectativas. Sin embargo muy pocas veces se toma en consideración y es precisamente ahí donde se pierde una buena parte de análisis y requerimientos que fueron olvidados u obviados por el usuario.

3.3.2. Lectura del código fuente

Una vez entendida la funcionalidad que debe cubrir el sistema, lo más común en esta etapa es depurar el código siguiendo un flujo para cubrir la funcionalidad y encontrar así todas las excepciones y condiciones del sistema.

Aquí, se debe entender como está estructurado el sistema para cubrir la funcionalidad. Una vez recorrido el flujo básico obtenido de la caja negra e identificadas las condiciones del sistema, contenidas en el código, se continúa con pruebas de caja blanca a modo de conocer el manejo que se le da a todas las excepciones en el sistema en un ambiente funcional.

Es en esta etapa cuando se identifican todas las características, conexiones y elementos que forman el sistema y el modo en que lo están haciendo. Se debe tener especial cuidado y orden para no perder información, de hacerlo, se podría dejar un hueco que provoque conflictos en las siguientes etapas o en el resultado del proceso.

3.3.3. Restructuración del código fuente

Esta etapa consiste en ordenar el código original de tal forma que cualquier programador pueda entender el flujo que se está brindando a la información. El objetivo en este paso no es modificar el código, si no clarificar su lenguaje e identificar más fácilmente sus componentes fundamentales. Dicho en otras palabras se debe ordenar el código a fin de agilizar su entendimiento en las siguientes etapas.

3.3.4. Depuración del código fuente

Una vez entendido y reestructurado el código, éste se debe limpiar, para ignorar u omitir líneas innecesarias (estas pueden ser comentarios que a pesar de haber sido de apoyo en etapas anteriores, aquí ya no son necesarias). Una vez identificadas las condiciones y excepciones de la información, se obtienen y se realiza un primer diagrama para describir el flujo de la información, dentro del sistema.

En esta etapa se busca identificar aquellas características que pudieron ser los requerimientos originales del sistema, a partir de las condiciones en cada flujo.

3.3.5 Abstraer el sistema

La abstracción de un sistema es un tema digno de un estudio independiente ya que dependiendo de la solución modular para cualquier problema, pueden formularse muchos niveles de abstracción, siendo el nivel superior un planteamiento en lenguaje del entorno del problema y el nivel inferior una orientación más procedimental (Pressman, 1993). Cuanto más alto sea el nivel de la abstracción, menor será el número de elementos necesarios para representar un sistema completo y más fácil será el manejo de su complejidad (Weitzenfeld, 2005). Finalmente para nuestro propósito lo describiremos mediante una serie de modelos del sistema, mediante diagramas que describan su comportamiento, los elementos y las características que lo componen.

A través de la información obtenida en los pasos anteriores se hace posible la abstracción y el desarrollo de los modelos y diagramas. El objetivo de esta etapa es modelar el sistema de tal forma que se entienda por cualquier integrante del equipo o involucrado en el proyecto. Si es cierto que no todos tienen el mismo nivel de conocimiento en diferentes ramos, también es cierto que en la actualidad existen diferentes diagramas para cubrir diferentes necesidades. La abstracción en esta etapa generalmente se lleva a cabo mediante diagramas en UML ya que su flexibilidad les permite adaptarlo a cualquier técnica de desarrollo, pero esto se verá en el siguiente capítulo y en el anexo diagramas UML.

- ⇒ Procesamiento.- El procesamiento ya no se enfoca en qué debe hacer, como su nombre lo indica, describe cómo se está realizando la operación del sistema actual para satisfacer la necesidad. En muchas ocasiones se

puede encontrar que existen muchos flujos para una tarea que resulte ser muy sencilla.

- ⇒ Interfaz.- Se identifican las comunicaciones entre los elementos del sistema, incluso aquellos que sirven de comunicación con el usuario (User interface, UI por sus siglas en inglés) para entender por qué se está presentando sólo cierta información a los usuarios y porque se está solicitando alguna otra.
- ⇒ Bases de datos.- Como es bien conocido, en los sistemas lo más importante es la información, esta, principalmente se encuentra almacenada con cierto orden en una base de datos. Suele suceder que sólo se estudia el ¿qué? y el ¿cómo? de un sistema, pero no el ¿dónde?
 - Las características de una base de datos que ya existe son muy importantes para el sistema, porque puede ser que se modifique al realizar una reingeniería, pero las condiciones iniciales tienen una razón de ser. Por ejemplo, un campo que sólo acepta enteros, quizá sea porque es un número de cuenta y este no debe ser decimal. Quizá en la parte de la vista al usuario se dejó libre este campo, pero al estar limitado desde la base de datos, se sabe que esto será un error y por lo tanto, que debe ser prevenido.

3.3.6. Documentación de sistema (requerimientos obtenidos)

La finalidad de la ingeniería inversa descomponer un sistema existente hasta su mínima expresión (dato), encontrar las relaciones existentes entre los elementos que la conforman y comprender su funcionamiento. Existen diferentes métodos para documentar un sistema software (Simoes & Lino. 2009), aquí se propone el uso de fichas técnicas, diagramas de colaboración y diagramas de secuencia para la documentación referente al diseño, desarrollo e interfaz del sistema. Y casos de uso para la documentación formal.

El objetivo de tener estos dos documentos es tener una clara comprensión del sistema por el usuario y el cliente (quienes por lo general no entienden de tecnología), así como por el especialista en las tecnologías de desarrollo (quien por lo general no entiende el negocio para el que fue diseñado el sistema), es decir, tener una descripción del sistema en alto y bajo nivel.

A lo largo de este estudio, nos referiremos a la ficha técnica como un documento en forma de sumario que contiene la descripción de las características de un objeto, material, proceso o programa de manera detallada (Brusola, 1999). Anexo a este capítulo se propone un formato de la ficha técnica que recopile las características necesarias con las que será desarrollado el sistema. Estas características serán orientadas al diseño y desarrollo del mismo, desde la etapa de análisis.

Como hemos podido ver, este proceso puede escucharse muy sencillo, pero tiene su complejidad en el nivel de conocimientos con los que se debe contar y el modo ordenado con el que se debe administrar la información, no es cuestión sólo de hacer las cosas, sino hacerlas bien. No obstante, a las características de cada persona, es indispensable que el analista tenga firmes conocimientos de desarrollo y diseño de software e ideal que cuente con buenos hábitos de redacción y experiencia en especificación de casos de uso.

3.4. Ingeniería inversa y reingeniería

Como se presentó anteriormente, la ingeniería inversa no siempre implica un proceso de reingeniería. Como primer punto, es necesario identificar estos dos conceptos, ya que de forma errónea, se han llegado a utilizar como uno sólo. Es cierto que una reingeniería implica comúnmente una ingeniería inversa, pero una ingeniería inversa no siempre desemboca en una reingeniería del producto, es posible que continúe con un mantenimiento (por falta de documentación o fallas constantes en el sistema) o con un reúso del código.

Una ingeniería inversa nos lleva de lo particular a lo general, mientras que la reingeniería es un proceso que utiliza la información del diseño existente para alterar o reconstruir el sistema, en un esfuerzo de mejorar la calidad general (Pressman, 1993). Debemos tomar en cuenta que no siempre es así, ya que es posible que la documentación del sistema actual sea lo suficientemente claro y ordenada como para apoyarnos de esta, sin la necesidad de cubrir este recorrido, aunque esta posibilidad es poco probable.

En este estudio sólo se toma en cuenta aquellos puntos que refieren a la obtención, definición, validación y aceptación de los requerimientos abstraídos de un sistema y posiblemente complementados por un usuario.

3.4.1 Ingeniería inversa como etapa de obtención de requerimientos de la reingeniería

Partiendo de que una reingeniería comúnmente requiere de una ingeniería inversa, esta se considera una etapa del proceso de la reingeniería, puesto que se requiere entender cómo funciona un producto para poder rediseñarlo, depurando sus defectos y detallando el proceso a la medida de sus necesidades, con el fin de incrementar su calidad.

Siendo esta la primera etapa, ya que el resultado de la ingeniería inversa se considera la entrada de la reingeniería. O dicho de otro modo, la ingeniería inversa obtiene las necesidades y requerimientos originales del sistema, para inyectarlos en un nuevo ciclo de vida de software, como si la primera fuera el levantamiento de los requerimientos, sólo que obtenidos de una aplicación y no de un usuario. Esto podría parecer falso, ya que en algunas ocasiones el usuario administrador del sistema es quien nos proporciona los requerimientos “obtenidos” del sistema, pero aun así, la fuente original suele ser un sistema o producto ya existente.

3.4.2. Refinamiento y simplificación de requerimientos

Todos los requerimientos recuperados de una ingeniería inversa, deben ser verificados y evaluados por un usuario del sistema con un conocimiento profundo del proceso. Para este caso no es admisible presentar la información obtenida del sistema en un aspecto abstracto de alto nivel. Se describe la información obtenida en el nivel de abstracción más bajo para que el sistema sea digerible al usuario y poder expresar más fácilmente su funcionamiento y lo que fueron sus requerimientos iniciales. Esto se puede realizar clasificando los requerimientos (véase capítulo anterior) y clarificando su funcionamiento a modo de describir lo que fue su especificación inicial.

En esta etapa y siendo una reingeniería se debe verificar que los requerimientos ya cuenten con las características de calidad especificadas en el capítulo anterior, para prever desde este momento posibles errores en la etapa de diseño. Si existen estos defectos se deben hacer notar como posibles mejoras para el nuevo sistema.

3.4.3. Especificación final de los requerimientos

Llegada a la etapa final de la ingeniería inversa y habiendo entendido, refinando y depurado los requerimientos, se especifican, como si estos fueran los obtenidos directamente del usuario, con la finalidad de validar con el cliente si éstas aún deben ser las condiciones que debe tener el sistema a construir o si es necesario realizar algún cambio o mejora dentro del nuevo producto.

Al igual que en la etapa de validación de requerimientos (véase capítulo anterior), aquí se debe tener mucho cuidado en los márgenes de la negociación y los alcances de las modificaciones, ya que de esto depende que se alcancen los objetivos y el sistema pueda ser entregado en las fechas convenidas.

A partir de aquí, se inicia un nuevo ciclo de vida del sistema, siendo esta la etapa del análisis.

3.5. Herramientas para la Ingeniería Inversa

En la actualidad se cuentan con herramientas que facilitan el proceso de la ingeniería inversa conocidos como herramientas case. Obtienen y en algunos casos abstraen la información necesaria de la arquitectura, bases de datos y otras características de los sistemas. Estudiar todas las herramientas que auxilian el análisis y diseño de software ha generado estudios completos como el caso de Fragoso de la Torre (1997) en su tesis “Selección de herramientas CASE de acuerdo a las necesidades de una institución” o Casas Valadez (2001) “De las herramientas CASE a las herramientas de apoyo a procesos de desarrollo software” entre muchos otros. El objetivo no es profundizar en este tema, sino simplemente dar a conocer las herramientas y una breve visión de sus características.

3.5.1. Las Herramientas CASE

Las herramientas de ingeniería de sistemas asistida por ordenador (Computer-Aided Systems Engineering – CASE) aplican la tecnología informática a las tareas y metodologías de desarrollo de sistemas, automatizando o apoyando las fases

del ciclo de vida del desarrollo de sistemas. En la ingeniería inversa, estas herramientas pueden contener una o más de etapas del ciclo de vida, junto con otras que mejoran el rendimiento y la eficiencia. En la actualidad muchas de las herramientas mencionadas, son utilizadas como herramientas case, para obtener o administrar las características de un sistema existente. En otras palabras, nos permiten entender un sistema y abstraer las posibles necesidades que dieron origen al mismo.

Estas necesidades obtenidas, suelen dar origen o ser parte de un nuevo sistema, lo que algunas personas definen como re ingeniería. Sin embargo, para poder realizar una re ingeniería primero es necesario obtener de algún modo las necesidades y características originales de algo ya existente, para poder brindarle mejoras. Sobre el proceso de obtención y administración de esta información se hablará más a detalle en el siguiente capítulo, sin embargo no se debe olvidar la complejidad mencionada en el presente, esto con el fin de no subestimar la administración de los requerimientos y características obtenidas de una ingeniería inversa.

3.5.2. Los Depuradores

Un depurador es un programa que controla otros programas. Recorre paso a paso el código, persigue defectos, determina puntos estables dentro del sistema y rastrea las variables y estados de memoria. Los depuradores son muy valiosos a la hora de determinar el flujo lógico del programa. Si lo ubicamos dentro del flujo propuesto para el proceso de ingeniería inversa, se diría que cubre la etapa de Análisis de la funcionalidad.

En la tabla 1 se mencionarán de manera general algunos ejemplos y una breve descripción.

Tabla 1. Ejemplo de depuradores con breve descripción

DEPURADORES	
Depurador	Breve descripción
Visual DuxDebugger	Visual DuxDebugger posee un desensamblador especialmente creado para utilizar todos los recursos proporcionados por Windows de 64 bits, especialmente útil cuando el código fuente no está disponible. No sólo te permitirá depurar varios procesos primarios, también puedes procesar varios procesos secundarios.
<u>GNU Debugger (gdb)</u>	GDB, permite ver lo que está pasando 'dentro' otro programa mientras se ejecuta - o lo que otro programa estaba haciendo en el momento en falló. Esta herramienta puede hacer cuatro tipos de tareas para ayudar a capturar errores en el acto.
<u>SoftICE</u>	La palabra Softlce define perfectamente lo que esta gran herramienta significa, "congela" el software para su posterior preparación. Es un estándar de facto para depuración Windows. A pesar de que el nombre pudiese inducir a pensar que sólo se pueden depurar drivers, SoftICE es capaz de depurar casi cualquier tipo de aplicación, incluyendo EXE's, DLL's, OCX's y dispositivos VxD tanto estáticos como dinámicos.
OllyDbg	OllyDbg es un potente depurador Windows con un motor de ensamblado y desensamblado integrado. Tiene numerosas características incluyendo un precio nulo. Muy util para parcheado, desensamblado y depuración. La aplicación OllyDbg se desarrolló para ser un depurador de análisis a nivel de ensamblador de 32 bits para Microsoft Windows. Pone énfasis en el análisis del código binario lo que es especialmente útil en casos donde el origen no está disponible.
IDA Pro	El desensamblador interactivo multiprocesador y multiplataforma, por DataRescue. Es un desensamblador programable, interactivo y multiprocesador combinado con un depurador local y remoto y aumentado por un entorno de programación de plugin completo.
Cheat Engine	Cheat Engine, comúnmente abreviado como CE, es un software para escanear y depurar memoria es una herramienta de código abierto diseñado para ayudarle con todo tipo de sitios web o juegos creado por Starkc319 (Hacker mundial) para el sistema operativo Windows.
WinDbg	WinDBG es una pieza de software gratuita de Microsoft que puede ser usada para depuración local en modo usuario, o incluso depuración remota en modo kernel. WinDBG no es lo mismo que el mejor conocido depurador de Visual Studio, pero viene con un interfaz gráfico de todas maneras. Viene en versiones de 32 y 64 bits.

3.5.3 Las Herramientas de Inyección de Fallos

Son herramientas que proporcionan entradas formato con características inadecuadas para el software con el objeto de provocar errores. Algunos defectos pueden ser de seguridad, como permitir acceso directo al ordenador principal o red. Existen herramientas de inyección de fallos apoyados en el anfitrión; funcionan como depuradores. Permiten alterar las condiciones del programa para observar los resultados. Otro tipo de herramienta es el inyector basado en redes, que manipulan el tráfico de la red para determinar el efecto en el aparato receptor.

Estos son algunos ejemplos de herramientas de inyección de fallos publicados por Jeremiah Grossman, fundador de security-hacks en el 2006.

Tabla 2. Ejemplo de herramientas de inyección de fallos y breve descripción

HERRAMIENTAS DE INYECCIÓN DE FALLOS	
Herramienta	Breve descripción
SQLler	SQLler es una herramienta que determina toda la información necesaria para construir y explotar un agujero de inyección SQL a partir de una URL. Esto lo realiza casi sin intervención del usuario. Esta acción se realiza mediante una consulta diseñada para contraseñas de fuerza bruta, fuera de las bases de datos. Una contraseña de 8 caracteres (que contiene cualquier carácter de código ASCII decimal 1-127) tarda aproximadamente 1 minuto de roer.
SQLbftools	Son un conjunto de herramientas escritas en lenguaje C destinadas a los ataques a ciegas en motores de bases de datos MySQL basadas en el sistema utilizado en SQLInjector de NGS Software. Esta herramienta es mantenida por la web http://www.unsec.net .
Bfsql	Evolución de SQLBfTools. La es de código abierto y herramienta sigue utilizando el sistema de palabra clave en valores positivos. La herramienta no pide la intervención del usuario para averiguar cuál es la palabra clave, sino que realiza peticiones con inyecciones de cambio de comportamiento cero e inyecciones de cambio de comportamiento positivo. Recibe las respuestas, un archivo de respuesta para el valor correcto y otro archivo para el valor incorrecto y las compara línea a línea buscando la primera diferencia. A partir de ese momento realiza peticiones y mira a ver a qué valor corresponde.
SQL Injection Brute-forcer	SQLibf puede trabajar en inyección a ciegas y visible. Funciona ejecutando operaciones SQL lógicas para determinar el nivel de exposición de la aplicación vulnerable.
SQLBrute	SQLBrute es una herramienta para extraer datos de las bases mediante ataques de fuerza bruta usando vulnerabilidades de inyección SQL a ciegas. Soporta exploits basados en tiempo y error en un servidor Microsoft SQL, y exploits basados en errores para Oracle. SQLBrute está escrito en Python, utiliza multi-proceso y no requiere librerías no-estándar.
BobCat	BobCat es una herramienta que permite ayudar a un auditor a tomar completa

		ventaja de las vulnerabilidades de inyección SQL. Está basado en AppSecInc. Puede listar los servidores enlazados, esquemas (schema) de bases de datos y permite obtener datos de cualquier tabla a la cual la aplicación tenga acceso.
SQLMap		Es quizá la herramienta más completa, y mi favorita, escrita en python y con una gran variedad de opciones, además de tener integración con herramientas como metasploit permite explotar al máximo un vulnerabilidad SQL Injection.
Absinthe		Absinthe es una herramienta basada en interfaz gráfica que automatiza el proceso de descargar el esquema y los contenidos de una base de datos que es vulnerable a SQL Injection.
SQID		SQL Injection digger (SQLID) es un programa que funciona mediante línea de comandos que busca inyecciones SQL y errores comunes en sitios web.
SQL Power Injector		SQL Power Injection ayuda a un auditor a inyectar comandos SQL en una página web. Su fuerza principal radica en la capacidad de automatizar inyecciones SQL tediosas utilizando para ellos múltiples procesos.
FJ-Injector Framwork		FJ-Injector es un framework opensource diseñado para ayudar a encontrar vulnerabilidades SQL en aplicaciones web. Incluye características de Proxy para interceptar y modificar solicitudes HTTP y una interfaz para realizar explotación SQL automática.
SQLNinja		SQLNinja es una herramienta para explotar vulnerabilidades de inyección SQL en aplicaciones web que utilizan Microsoft SQL Server como motor de base de datos.
Automagic SQL Injector		Automagic SQL Injector es una herramienta de inyección SQL automatizada que fue diseñada para ahorrar tiempo en los tests de intrusión y solamente funciona con agujeros de Microsoft SQL que devuelven errores
NGSS SQL Injector		NGSS SQL Injector explota vulnerabilidades de inyección SQL en distintos servidores para obtener acceso a la data almacenada. Actualmente soporta las siguientes bases de datos: Access, DB2, Informix, MSSQL, MySQL, Oracle, Sysbase.

3.5.4 Los Desensambladores

Es una herramienta que convierte código máquina en lenguaje ensamblador. Los desensambladores revelan las instrucciones máquina que son usadas en el código. El código máquina puede ser específico para una arquitectura determinada de hardware. De forma que los desensambladores son escritores expresamente para la arquitectura del hardware del software a desensamblar.

Se debe tomar en cuenta que los desensambladores brindan las instrucciones, pero no así la estructura original del código. Algunos ejemplos de desensambladores son:

Tabla 3. Ejemplos de desensambladores y breve descripción

DESENSAMBLADORES	
Desensamblador	Breve descripción
IDA Pro	Es un desensamblador y debugger programable e interactivo. IDA Pro permite realizar ingeniería inversa con cualquier tipo de ejecutable o aplicación existente. IDA Pro puede manejar archivos de consolas y máquinas como Xbox, Playstation, Nintendo, sistemas para Macintosh, PDA. Plataformas, Windows, UNIX, y un sinnúmero de archivos.
PE Explorer	PE Explorer es la más característica-embalado programa para inspeccionar el funcionamiento interno de su propio software, y más importante aún, de terceros aplicaciones de Windows y las bibliotecas para las que no tienen código fuente.
BORG Disassembler	Borg es una utilidad pequeña y útil que permite a los usuarios desmontar archivos PE de Win32 y devolver valores utilizando las pilas de llamadas. Asimismo, Blog le permite desmontar archivos en código o en los datos. Aunque otros archivos se cargan, el soporte para otros tipos de archivo no se ha completado todavía (lo más notablemente posible NE / LE / LX archivos no están debidamente soportados).
HT Editor	HT Editor es un programa que sirve para editar, analizar y visualizar archivos ejecutables. El objetivo principal de HT Editor es combinar las funciones de bajo uso de un depurador con la usabilidad de los entornos de desarrollo. Incluye grandes capacidades de edición y soporte para la mayoría de archivos con formatos PE, LE, NE, MZ, COFF, LZ y ELF.
Bastard Disassembler	Un desensamblador - o, más apropiadamente, un entorno de desmontaje. La idea es que usted tiene un intérprete, del mismo modo que lo haría en Perl o Python, lo que le permite cargar archivos, desmonte ellos, vuelque el de-smontaje, escribir / ejecutar macros, y varias otras operaciones. El desensamblador de instrucciones x86 escrito para este proyecto ha sido empaquetado por separado como libdisasm, y está destinado a ser utilizado en otros proyectos de código abierto.
Objdump	Objdump despliega información sobre uno o más ficheros objeto. Las opciones controlan qué información mostrar en particular. Esta información es útil mayormente para programadores que estén trabajando en las herramientas de compilación, en vez de para los que simplemente quieren que sus programas compilen y funcionen. Se puede utilizar la herramienta objdump para desensamblar el segmento de código del archivo binario ejemplo o y analizar su contenido
lida linux interactive disassembler	Lida es básicamente un desensamblador y herramienta de análisis de código. Se utiliza el código de operación para libdisasm cabrones solo decodificación. Permite control interactivo sobre el deadlisting generado a través de comandos y herramientas integradas.

3.5.5. Los compiladores Inversos o Decompiladores

Un decompilador es una herramienta que transforma código en ensamblador o código máquina en código fuente en lenguaje de alto nivel. También existen de compiladores que transforman lenguaje intermedio en código fuente en lenguaje de alto nivel. Estas herramientas son sumamente útiles para determinar la lógica a nivel superior como bucles o declaraciones if-then de los programas que son de compilados. Los de compiladores son parecidos a los desensambladores pero llevan el proceso un importante paso más allá (Martin).

Tabla 4. Ejemplo de decompiladores y breve descripción

DECOMPIADORES	
Desensamblador	Breve descripción
Jdec	JDEC es un descompilador de Java. Se puede utilizar para revertir el actual código de bytes en un archivo. Clase para producir un archivo de código fuente Java que puede ser casi correcto o equivalente (debido a diferentes interpretaciones) en el archivo original de java. [Con la interfaz de usuario y línea de comando]
Jreverse pro	JReversePro es una ligera aplicación de Java que fue diseñada para proporcionarte un medio simple de cargar archivos .class y descompilarlos al instante. El objetivo supremo de este proyecto es proporcionar un descompilador que genera una estructura de Java basada en objetos que puede controlarse mediante la programación utilizando una API específica.
Salamander	Salamander es un descompilador. NET que convierte los archivos ejecutables (. EXE o DLL.) De lenguaje intermedio (IL, MSIL, CIL) de formato binario a código fuente de alto nivel, como C #, C + + administrado, Visual Basic.NET, etc Desde hace más de 8.000 clases que se han probado, Salamander siempre produce códigos equivalentes y recompilable que son muy parecidas a los códigos fuente originales.

REFERENCIAS BIBLIOGRAFICAS

- Kendall, E. & Kendall, J. E. (1991). *Análisis y diseño de sistemas*. México: Prentice-Hall
- Presman, R. S. (1993). *Ingeniería de Software. Un enfoque práctico*. Editorial McGraw-Hill. Tercera edición
- Weitzenfeld, A. (2005) *Ingeniería de software orientada a objetos con UML, java e internet*. Editorial Thomson.
- Fontela, Carlos (2011). *UML. Modelado de software para profesionales*. Editorial Alfaomega. 1ra edición.
- Grady Booch, James Rumbaugh, Ivar Jacobson (2005). *El lenguaje unificado de modelado. UML 2.0. 2ª. Edición*. Peason Adison Wesley. España
- Coronado Pastor, Claudia Alejandra (2008). *Ingeniería inversa de base de datos*. UNAM
- Buscar Reingenieria de procesos de negocio de Hammer y Champy

REFERENCIAS ELECTRONICAS

- Simoes, Lino (2009). Técnicas y elementos de la documentación de sistemas. Universidad Nacional Experimental“ Simón Rodríguez”. Recuperado de: <http://es.scribd.com/doc/13735470/TECNICAS-Y-ELEMENTOS-DE-LA-DOCUMENTACION-DE-SISTEMAS>
- Martín Salvador, Manuel. I.T. Informática de Sistemas Traductores. Decompiladores. Universidad de Granada. De: <http://draxus.org/upload/decompiladores.pdf>
- Brusola, Fernando. (1999). Ficha Técnica. De: http://es.wikipedia.org/wiki/Ficha_t%C3%A9cnica
- Sicilia, Miguel Ángel. (2009). ¿Qué es la Ingeniería Inversa? De: <http://cnx.org/content/m17432/latest/>
- IISA. Ingeniería Inversa. De: <http://www.ingenieria-inversa.com.ar/index.php/off-shore/217>
- [Ingenieria inversa como un apoyo a las errameintas case](#)
- Visual DuxDebugger. <http://www.softpedia.es/programa-Visual-DuxDebugger-193234.html>
<http://www.duxcore.com/index.php/prod/visual-duxdebugger/overview>

- **GNU Debugger (gdb).** <http://www.gnu.org/software/gdb/> *Marzo 12, 2013.*
- **SoftICE.** Marino Posadas.
<http://www.dnmplus.net/columnas/laboratorio/softice-for-devpartner.aspx>
- **OllyDbg.** <http://www.ollydbg.de/> Noviembre 19, 2012
- **IDA Pro.** <http://ida-pro.softpedia.com/es/>
- <https://www.hex-rays.com/products/ida/index.shtml> Julio 25, 2012
- **Cheat Engine** http://es.wikipedia.org/wiki/Cheat_Engine Marzo 12, 2013
- **WinDbg .** <http://en.wikipedia.org/wiki/WinDbg>
- **10 Aplicaciones OpenSource para SQL Injection automatizado.**
HACKING, HERRAMIENTAS <http://xora.org/2012-10-aplicaciones-para-sql-injection-automatizado/> 27 NOVIEMBRE, 2012

CAPITULO 4. REQUERIMIENTOS PARA INGENIERÍA INVERSA

En este capítulo se hablará más a detalle de la administración de requerimientos, además de la documentación cuando proviene de una ingeniería inversa. El fin es dejar estos documentos planteados para diversos objetivos, sea mantenimiento, reingeniería o cualquier otro fin.

Es importante recordar que todo riesgo identificado a lo largo del proyecto debe ser informado al cliente, especialmente en la etapa de análisis. Es posible hacer sugerencias para su solución, pero debe ser el cliente mismo quien tome la decisión final sobre las características que el sistema tendrá o no para dicho riesgo.

4.1. INTRODUCCIÓN A REQUERIMIENTOS PARA INGENIERÍA INVERSA

Como se presentó en el capítulo anterior, la ingeniería inversa tiene diversas aplicaciones, ya sea para documentar un sistema, comprenderlo, replicarlo o mejorarlo, pero el objetivo es el mismo, obtener la información que dio origen al sistema u objeto que se desea entender. Dependiendo de la naturaleza y uso que se proporcione a la información obtenida por una ingeniería inversa, será el manejo que se le brinde. Realizar una ingeniería inversa de software puede parecer un proceso muy sencillo, pero esta característica principal de este proceso depende del nivel de conocimiento que se tenga en todo lo relacionado al desarrollo de software además de la comprensión del sistema mismo.

Es idóneo –por no decir necesario- que el análisis sea realizado por alguien cuyos conocimientos contemplen toda la información requerida para el desarrollo de un sistema, desde conocimientos de diseño y codificación, especificación de casos de uso y verificación de requerimientos. Las nociones de desarrollo serán muy útiles para poder modelar el flujo de los datos y la información obtenida de tal forma que cualquier programador lo pueda entender. La característica de especificación de casos de uso puede ser cubierta por una buena redacción, clara y objetiva. Sin embargo, la negociación para verificar los requerimientos, debe ser realizada por alguien con experiencia o alguien cuyas características y conocimientos de estimaciones le permitan llegar a acuerdos con el cliente encargado de la verificación de los requerimientos.

Basado en la publicación realizada por Aedo (2009), el analista de sistemas debe contar con los siguientes conocimientos y capacidades.

- ⇒ Abstracción.- Es necesario, para convertir las necesidades de un proceso en requerimientos específicos en el desarrollo de un sistema.
- ⇒ Administración de requerimientos.- El analista deberá poder identificar y priorizar los requerimientos basados en el impacto que tienen en el negocio que se lleva día a día.
- ⇒ Buena redacción.- Entender los requerimientos no es suficiente. Esta es una característica que muchas veces se ignora y su impacto puede resultar costoso. Toda información obtenida del cliente, que sea útil para el desarrollo del sistema debe ser claramente documentada a fin de que

cualquier elemento del equipo de trabajo, o por parte del cliente, entienda claramente lo que refiere al proyecto. Es muy común que el analista entienda perfectamente todo el negocio, pero la documentación no sea clara o este incompleta, de ahí la necesidad de una buena redacción.

- ⇒ Diseño y desarrollo de sistemas.- En caso contrario al punto anterior, existen empresas que utilizan a los documentadores como analistas, sin tomar en cuenta su falta de experiencia en el diseño y desarrollo de sistemas. Es aquí donde surge una gran cantidad de problemas, ya que un documentador que no tiene idea del diseño y mucho menos del desarrollo, no sabe que preguntar o que información obtener del usuario. Como su designación lo dice, obtiene y documenta las necesidades expresadas por el usuario, sin indagar más y el resultado nos lleva a una información incompleta para el diseño y peor aún, insuficiente para el desarrollo. Por lo tanto, una característica elemental del analista es tener conocimientos de diseño y desarrollo de sistemas, sería ideal que su experiencia fuera práctica, de no ser así, sus fundamentos teóricos deben ser lo suficientemente buenos para compensar esta falta de experiencia.
- ⇒ Estimación de proyectos.- El analista es el encargado de obtener, validar y estimar los requerimientos asociados al proyecto. Se espera que la estimación se realice al inicio de un proyecto, sin embargo a veces no se hace así, por lo tanto el analista tendrá que estudiar los requerimientos para determinar cuáles pueden estar fuera del alcance, ya sea en costo o tiempo y cuáles pueden ser aceptados dentro de las fronteras del proyecto. Aquellos requerimientos que puedan estar fuera de dichas fronteras deben ser informadas al líder de proyecto para que se realice una negociación, pero eso ya es tema de otro estudio “*Administración de proyectos*”.
- ⇒ Flexibilidad al trato con el cliente o usuario.- En ocasiones, el cliente es directamente el usuario o en otras, el cliente y el usuario son personas diferentes. Por ello se debe de obtener los requerimientos del usuario y validarlos con el cliente. Para obtener la información de un usuario, en ocasiones resulta difícil, si no se está acostumbrado al trato directo con el cliente. Se debe estar consciente que a pesar de trabajar con sistemas, máquinas e información, lo más importante es que todo parte de un cliente, una “persona” que tiene una necesidad dentro de su negocio y busca una solución. Obtener las necesidades, puede sonar muy sencillo, pero al entrar en contacto con el usuario o cliente se descubre que no saben cómo expresar una idea, o peor aún, no quieren brindar la información por distintas razones. Es bastante curioso que aquí, la mayoría de los ingenieros pueden llegar a tener conflictos, pero esta es una habilidad que

debe desarrollarse en cada uno, ya que al entrar en trato inmediato con el cliente pueden asegurar el proyecto o en su defecto, retrasarlo.

- ⇒ Habilidades de negociación.- La habilidad de negociación es una característica que va de la mano con la flexibilidad de trato al cliente. Una vez obtenidas las necesidades, se debe buscar un acuerdo que cubra las necesidades del cliente sin rebasar las fronteras del proyecto o satisfacer todas las necesidades evaluando el costo extra. Incluso, existen ocasiones en las que debe llegarse a una negociación para obtener la información necesaria para el sistema.
- ⇒ Liderazgo.- En muchos proyectos, el analista es quien controla el trabajo del equipo de diseño, para garantizar el cumplimiento de los requerimientos. Elige y diseña los métodos, técnicas y herramientas. Para algunos casos, es como un sub líder de proyecto.
- ⇒ Modelado de procesos.- El modelado de un proceso se acompaña con la capacidad de abstracción. Entender el negocio y documentarlo puede ser suficiente si la complejidad no es mucha. Sin embargo, entre más complejo sea el negocio, más complicado resulta expresar su operación. Por ello, es necesario tener la capacidad de concebir un proceso y modelarlo, ya sea en alto o bajo nivel a fin de despejar todas las dudas y expresar más claramente su función.
- ⇒ Objetividad.- Es muy cierto que dependiendo del negocio puede existir poca o mucha información, sin embargo se debe enfocar específicamente en el proceso en el que se basará el sistema y su entorno. Existen casos en los que se pierde la objetividad del sistema y se obtiene, documenta y modela información que no infiere al sistema a desarrollar, incluso se podría generar un nuevo proyecto de esta información. A pesar de que se debe observar todas las necesidades existentes en la operación, también se debe mantener en la mira, aquellas que son exclusivamente del proyecto actual.
- ⇒ Planeación.- Como ya se ha visto en capítulos anteriores, el análisis no sólo consiste presentarse frente al cliente y solicitarle sus necesidades. Al igual que un proyecto, el análisis requiere de una planeación de actividades y trabajo a desarrollar, por tiempos. Estos son generalmente marcados por el plan general del proyecto, pero dentro de las actividades propias de obtención de requerimientos deben planearse tiempos a fin de llevar un orden y entregas en tiempo.
- ⇒ Pruebas.- En la actualidad no se hace énfasis en este aspecto, sin embargo, un analista debe conocer todas las etapas de un ciclo de vida, incluidas las pruebas. Puede ser molesto para el cliente llevar un producto que no ha pasado por pruebas, debido a que los defectos se hacen notorios

más rápidamente a la vista de quien realizó la solicitud a quien elaboró el producto. Si bien ya ha pasado por las pruebas correspondientes a la funcionalidad, también deben realizarse pruebas del negocio. El analista con suficiente experiencia sabrá que es mucho mejor retrasar la entrega un periodo de tiempo definido que presentar un producto lleno de defectos.

Este capítulo propondrá una línea base de análisis basándose Lenguaje Unificado de Modelado (UML, por sus siglas en inglés), para abstraer la información obtenida de la ingeniería inversa y artefactos para validar la información obtenida. El objetivo es no perder información en el camino de la ingeniería inversa y conseguir llevar esta información hasta un nivel de abstracción donde se puedan redefinir los requerimientos, clasificarlos y re especificarlos para su validación. Todo esto siguiendo los lineamientos de calidad establecidos principalmente por los estándares ISO e IEEE, mediante sus definiciones y características especificadas en cada uno.

Se debe mencionar que este estudio sólo abarca hasta la obtención y validación de dichos requerimientos. Una vez obtenidos y re especificados se propondrán artefactos para su validación, tanto por el cliente como por el equipo de diseño (en el caso de una reingeniería) de esta forma se busca garantizar que la información pase integra de la aplicación existente a un nuevo producto ya depurado de defectos o características que puedan ser obsoletas para el negocio y permitir un nuevo diseño para el sistema.

4.2. ABSTRACCIÓN DEL SISTEMA.

Se ha visto ya, que la abstracción es un conjunto de características esenciales de una entidad que la distinguen de otros tipos de entidades. Una abstracción define una frontera desde la perspectiva del observador (Booch, Rumbaugh & Jacobson, 2006). Mientras la programación estructurada representa un sistema en términos de datos y funciones, la programación orientada a objetos representa un sistema mediante objetos omitiendo, en un alto nivel, los detalles correspondientes a datos y funciones (Weitzenfeld, 2005).

Los niveles de abstracción propuestos por Pressman (1993) son básicamente cuatro: representación de diseño procedimental, estructura del flujo de datos y control y el modelo relacional de entidades. Con base en estos cuatro niveles, se pretende obtener toda la información y modelarla mediante diagramas.

La abstracción ayuda a simplificar una realidad compleja, los mecanismos mencionados nos auxilian en el análisis de un sistema. De acuerdo a Coronado (2008) y lo sugerido por Weitzenfeld (2005) la ingeniería inversa brinda la siguiente información:

- ⇒ Diseño de procesamiento
- ⇒ Estructura de datos y de programas
- ⇒ Modelos de flujos de datos y de control
- ⇒ Modelos de entidades y de control

Fontela (2011) describe la abstracción como el proceso de enfatizar algunas cuestiones para comprenderlas mejor, dejando otras de lado o manteniéndolas con un nivel de detalle menor. Los mecanismos o etapas de abstracción basados en el dominio, descritos por este autor son:

- ⇒ Clasificación. Es un concepto que permite discriminar los elementos de un conjunto de acuerdo a sus características en común. La clasificación es el principio de identificación de clases, ya que basados en las características comunes de los elementos. Gracias a estas, es posible identificar y agrupar un conjunto de elementos descritos en un punto común o plantilla.
- ⇒ Asociación. Es una propiedad que permite la comunicación entre distintos elementos mediante una característica específica, siendo estos elementos de una misma clase o de clases distinta. La asociación posee una característica denominada cardinalidad que consiste en dar una correspondencia bidireccional basada en un rango.
- ⇒ Agregación o agrupación. La agregación es un tipo especial de asociación en la cual la asociación bidireccional es asimétrica. Es una propiedad transitiva indicando que el todo y las partes no son intercambiables, es decir si C es parte de B y B es parte de A, entonces C es parte de A.
- ⇒ Composición. Corresponde a una agregación en donde las partes no tienen sentido fuera del agregado o no pueden formar parte de más de un agregado
- ⇒ Generalización. Operación por la cual se establece que una o más clases tienen elementos en común que deseamos agrupar en una clase más genérica.

4.2.1. Funcionalidad en los niveles de abstracción.

El nivel de abstracción de un proceso de ingeniería inversa y de las herramientas que se usan para efectuarla se refiere a la sofisticación de la información de diseño que se pueda extraer el código fuente (Pressman, 1993). Toda la

información obtenida debe ser representada claramente y para esto se hará uso del lenguaje unificado de modelado. UML, tiene la característica de simplificar el entendimiento de la información contenida en él.

A continuación se muestra una tabla que ubica la funcionalidad obtenida de una ingeniería inversa de acuerdo a las etapas del análisis y el diagrama con el que se puede representar la información resultante de la ingeniería inversa.

Tabla 1. Diagramas UML para representar funcionalidad

ETAPA	ACTIVIDAD	RESULTADO	DIAGRAMA
Estructura de datos y de programa	<u>Análisis funcional</u> . Se recorren todas las opciones y flujos del sistema actual.	Se obtiene la funcionalidad que contiene el sistema, entrada y salida de datos.	Mapa de navegación mediante diagrama de clases, diagrama de actividad
Diseño de procesamiento	<u>Análisis del negocio</u> . Se analiza la información obtenida de la etapa anterior para identificar entidades en el sistema.	Se ubican las relaciones y dependencias del negocio en el sistema e identifican los conceptos y las entidades. Se entiende la necesidad general del sistema.	Modelo de casos de uso, modelo de negocio o conceptual.
Modelos de flujo y datos de control	<u>Debugger de la aplicación</u> . Se recorre línea por línea la aplicación. En cada uno de sus archivos, con el fin de identificar las condiciones que no son visibles a la ejecución del sistema, con el fin de obtener todas las restricciones del sistema.	Obtención de todas las condiciones del sistema, excepciones a la información, todas las reglas del negocio en el sistema, dependencias de otros sistemas y la clasificación de la funcionalidad, aquí se obtienen actores, roles y prioridades de los requerimientos.	Diagrama de actividad detallado, Diagramas de secuencia, Colaboración, estados y clases
Modelos de entidades y de control	<u>Análisis de la estructura del sistema</u> . Se analiza la forma en que está estructurado el sistema, es decir el orden y contenido general de sus archivos a fin de encontrar y entender la lógica con la que se diseñó el sistema.	Se obtiene la dependencia entre requerimientos para definir si las entidades identificadas en el análisis son entidades o son algún atributo de una entidad mayor. Se agrupan los archivos de acuerdo a su contenido para obtener la información según el orden y prioridad requeridos.	Diagrama de clases detallado y diagramas de colaboración detallado.

4.3. MODELADO DE UN NUEVO SISTEMA

El software necesita modelos para comunicar de manera sencilla una idea abstracta. El modelo más detallado de un producto de software es el código fuente. Pero es como decir que el mejor modelo de un edificio es el edificio mismo (Fontela, 2011). Los requisitos se pueden expresar de diversas formas, desde un texto común hasta un diagrama bien estructurado que describa todos sus detalles. Booch, Rumbaugh y Jacobson (2005) definen un requerimiento como una característica, propiedad o comportamiento deseado de un sistema.

De acuerdo a Fontela (2011), un modelo es una representación analógica para ayudar a visualizar algo que no se puede observar claramente y que se realiza con un propósito determinado y se destina a un público específico.

El software en si es un modelo de la realidad, esto suena algo complejo, pero debido a la dimensión de los elementos que contiene requiere ser representado en partes. Las principales razones para modelar el software son:

- ⇒ El software es invisible e intangible
- ⇒ El software es mucho más modificable que otros productos realizados por el hombre
- ⇒ El software se desarrolla por proyectos, no en forma repetitiva como los productos de la industria manufacturera.
- ⇒ El software es substancialmente complejo.

Con el fin de modelar la información obtenida y rediseñar un nuevo sistema se sugiere utilizar UML porque su objetivo es realizar un modelado visual utilizando diagramas para mostrar distintos aspectos de un sistema – cualquier sistema – (Fontela, 2011).

Como ya fue mencionado en el capítulo 1, este escrito contiene un cuadro con el uso de UML y sus principales diagramas en el desarrollo de software. La tabla 1 de este capítulo muestra el modelo y diagrama para las etapas que rodean el análisis, basándose en la información obtenida. Es importante puntualizar que a pesar de usar el mismo modelo para representar la abstracción obtenida de la ingeniería inversa y un nuevo diseño, la información contenida en éstos es diferente, de acuerdo a las arquitecturas del sistema.

La siguiente tabla, describe de forma breve los modelos recomendados por UML que se deben realizar de forma mínima en las etapas que rodean el análisis en un ciclo de vida, estos modelos se realizan mediante diagramas especificados por el mismo lenguaje. Los diagramas aquí descritos son necesarios,

independientemente del modelo de desarrollo que se utilice ya que facilitan tanto la administración del proyecto como el desarrollo y mantenimiento.

Tabla 2. Modelado con UML

ACTIVIDAD	MODELO	DIAGRAMA	DESCIPCIÓN
Requisitos	Modelo de dominio, modelo de dominio o modelo conceptual.	Diagrama de actividades (flujo del proceso del negocio) Diagrama de clases	El diagrama de actividades permite describir el negocio u organización para la cual se desarrolla el producto de software. Todo modelo tiene elementos estructurales y de comportamiento. Los aspectos de comportamiento pueden representarse con el diagrama de actividades, con el fin de describir el flujo entre los conceptos del negocio y los elementos estructurales con diagrama de clases.
Análisis	Modelado de dominio detallado.	Diagrama de clases con atributos y procesos	El diagrama de clases, enfocado al modelo de dominio, permite analizar y clasificar los conceptos de un dominio y sus relaciones. Esto se convierte en un diccionario o glosario gráfico que facilita la comunicación entre los involucrados en el proyecto.
	Modelo de Casos de uso	Diagrama de casos de uso	El diagrama de casos de uso modela las necesidades funcionales del sistema. Suele servir para delimitar el alcance del proyecto, esbozar quiénes interactuarán, las acciones esperadas y capturará un primer glosario de términos. Los diagramas de casos de uso ayuda a definir el contexto de un sistema antes de construirlo
	Modelado del comportamiento de requisitos	Diagrama de actividades detallado	El diagrama de actividades tiene la flexibilidad de definir de manera clara y detallada, el flujo de un proceso acorde a cada requisito. Ayuda a comprender el flujo de actividades de un caso de uso.
		Diagrama de	El diagrama de secuencia,

		secuencia	modela y facilita entender el comportamiento de los casos de uso y todos los elementos involucrados en el. Las entidades son los actores y el sistema. Se modelan las actividades con el paso de mensajes entre ambos.
Diseño del comportamiento	Modelo de comportamiento detallado	Diagrama de comunicación o de colaboración	Un diagrama de comunicación es lo que antes se conocía como diagrama de colaboración. Es un diagrama de objetos en un escenario, al que se le agregan los mensajes que los objetos se envían entre sí.
	Modelo de clases orientados al análisis basado en el comportamiento	Diagramas de clases	En los diagramas de clases puede indicarse la navegabilidad con una flecha en la asociación entre clases, para indicar cuál es la que necesita servicios de otra.
	Modelo de estados	Diagrama de máquina de estados	En algunos sistemas, este diagrama no es necesario, sin embargo es una herramienta que sirve para modelar cómo afecta un escenario a los estados que un objeto toma. Se pueden usar para modelar objetos reactivos, cuál es su comportamiento frente a estímulos provenientes desde fuera de su contexto.
	Modelo de interacción	Diagrama de secuencia detallado	El diagrama de secuencia permite indicar condiciones y ciclos iterativos. Los diagramas de comunicación y de secuencia son equivalentes.

UML brinda una gran cantidad de diagramas que tienen la flexibilidad de modelar diferentes aspectos dentro de un proceso. El objetivo de este estudio es obtener aquellos que auxilian el análisis y diseño de software facilitando la comunicación entre los integrantes del equipo de trabajo. El modelo de requisitos descrito en la tabla 2, tiene como meta definir y delimitar la funcionalidad del sistema de software, por esa razón es el primero en desarrollarse, ya que todos los demás modelos se basaran en él. Puede usarse como base de negociación inicial entre el

proveedor del sistema y el cliente. A partir de la autorización de este, se inicia el diseño de los demás.

4.3.1. Obtención de requerimientos

En el capítulo anterior se mencionaron algunos diagramas que sirven para el análisis de un sistema ya existente, con el fin de obtener sus requerimientos originales. En estos casos, se debe tener una especial atención en las condiciones que maneja el sistema y los ciclos de control, así como la selección de la información que obtiene de la base de datos; porque es muy posible que dentro de estas se encuentren reglas de negocio.

Para comprender el dominio de un sistema, se debe recolectar la información que defina sus límites y restricciones. El resultado de esta búsqueda, permite obtener un conjunto de requerimientos del sistema que pueden ser clasificados (Mejía, 2009). Para obtener esta información se utilizan técnicas de obtención de requerimientos, algunas de estas técnicas son:

⇒ Entrevistas. Es una técnica para obtener información mediante una conversación dirigida por los analistas a los clientes y/o usuarios, con el objetivo de entender el dominio del problema y sus necesidades (Mejía, 2009). La entrevista debe tener un estudio previo del negocio, una planeación previa con el fin de establecer objetivos y guiar la entrevista; selección de usuarios a entrevistar, ya que deben estar relacionados solamente con el sistema. Un tip muy útil para obtener los requerimientos es identificar los verbos y los sustantivos, ya que por lo general de éstos se derivan los casos de uso y las clases. La planeación de las entrevistas debe marcar una línea base y una estructura para obtener la información.

A pesar de que las entrevistas son una técnica muy útil, es posible que se compliquen, si no se guía correctamente al entrevistado. Suelen divagar entre una idea y otra y no concretar nada. Obtener de forma correcta la información depende mucho de la habilidad y experiencia del entrevistador, así como la disponibilidad del usuario. Esta técnica es recomendable cuando el usuario, conoce perfectamente la operación de su negocio y las necesidades que se deben satisfacer en el sistema. Principalmente, se cuenta con una buena disposición de tiempo por parte del entrevistado y el entrevistador cuenta con la experiencia necesaria para obtener toda la información requerida para el sistema.

- ⇒ Cuestionarios: Permite recopilar la información puntual de las necesidades del sistema. Un cuestionario puede guiar al cliente a brindar la información requerida, sin embargo, en ocasiones no permite obtener la información necesaria que brinda una entrevista, que por lo general es muy importante. Un cuestionario puede brindar información de forma limitada, pero es muy útil, cuando el usuario no tiene disponibilidad de tiempo para entrevistas continuas. Esta técnica es recomendable cuando se tiene un conocimiento amplio del negocio y el tiempo brindado por el usuario es limitado. Generalmente, es una técnica que se apoya de otras. Un cuestionario puede ser complementado con una entrevista guiada o un cuestionario puede ser complemento de una observación.
- ⇒ Lluvia de ideas. Esta técnica, recolecta todos los puntos de vista de los usuarios, organiza y estructura la información obtenida, clasificando los requerimientos de acuerdo a cada necesidad. Se realiza en una reunión entre el cliente y usuarios con el equipo de análisis del proveedor. Las normas son muy parecidas a un debate, la diferencia es que el objetivo no es defender una idea, si no aportar la mayor cantidad de ideas que ayuden a encontrar una solución para el problema.

A pesar de la gran utilidad de esta técnica, el coordinarla puede resultar una tarea muy tediosa y su complejidad crece con el número de participantes en la misma. Y en ocasiones puede no llegar al punto esperado que es obtener la mayor cantidad de información desde diferentes puntos de vista.

El uso de esta técnica es recomendable cuando los usuarios del sistema pertenecen a distintas áreas del negocio y todos tienen gran importancia en la operación del día a día. La mejor manera de identificar la necesidad de esta técnica es cuando el sistema se encargará de comunicar diferentes departamentos en la empresa o el proceso será un conjunto de sub procesos internos en dichas áreas.

- ⇒ Observación. Esta técnica consiste en introducirse al ambiente de trabajo de los usuarios para abstraer las tareas realizadas y la cooperación que existe entre los mismos. El objetivo es entender la realidad del negocio y su contexto social, para optimizar la el funcionamiento de los procesos. La observación tiene la ventaja de permitir obtener toda la información que rodea la necesidad, sin embargo, se deben tener bien definidos los límites y el proceso de abstraer el negocio puede resultar complejo dependiendo de la habilidad y experiencia del analista.

La observación resulta recomendable cuando no existe disponibilidad por parte del usuario para brindar una entrevista o la complejidad del proceso es demasiada para ser narrada en una sola entrevista. Generalmente en esta técnica se identifican sub procesos que intervienen concisamente en el sistema a desarrollar. Observar siempre brinda una gran cantidad de información que puede ser complementada con un cuestionario o con una entrevista guiada, para definir conceptos o puntualizar algunos aspectos identificados dentro del proceso.

Como es de imaginarse, las técnicas de obtención de requerimientos pueden ser de gran ayuda, pero depende mucho de las habilidades y/o experiencia del consultor para obtener la información así como la disponibilidad del usuario. Sin embargo, en una ingeniería inversa, se debe recordar que una de las características necesarias del analista es tener firmes conocimientos de desarrollo, para hacer la traducción entre el código, el negocio y de ahí al diseño de un nuevo sistema.

4.3.2. Evaluación de calidad para requerimientos

Una vez obtenidos los requerimientos, el siguiente paso es analizarlos y documentarlos para su aprobación y construcción. Dentro de la etapa del análisis, se deben contemplar todas las características con las que debe contar el requerimiento, el tipo y su posible relación con otros requerimientos.

Este análisis debe tener por límites los establecidos por los estándares a fin de cumplir con las expectativas de calidad. Es cierto que en ocasiones los límites del cliente pueden ser más o menos de los establecidos en el estándar, pero seguirlos, asegura el buen desarrollo y funcionamiento del sistema, así como la construcción del sistema necesario para el cliente.

Se debe recordar que la calidad de los requerimientos para este estudio, se basa en la norma ISO 2500 y la conjunción de las normas que la componen. Esta norma ya ha sido descrita en el capítulo 2 y sólo se enunciarán las características que serán utilizadas como marcos de evaluación para la calidad de los mismos. Estas características para los requerimientos son:

⇒ Necesarios.

- ⇒ Concisos.
- ⇒ Completos.
- ⇒ Consientes.
- ⇒ No ambiguos.
- ⇒ Verificables.

Las características antes mencionadas serán los puntos de referencia a partir de los cuales se debe realizar una validación al momento de obtener los requerimientos, ya que reduce la posibilidad de errores en el sistema y facilita su identificación y administración.

4.4. Análisis y documentación de requerimientos

Es necesario resaltar la importancia de una documentación de los requerimientos, ya que de esta, depende que sea digerible la necesidad del proyecto. En algunos casos esto puede ser un proceso arduo, pero se debe entender que los documentos o también conocidos artefactos tienen muchos objetivos. Algunos ejemplos son: mantener un orden en la información referente al proyecto, mantener líneas base para el desarrollo del mismo, limitar el alcance y la calidad de los mismos, asegurar el mantenimiento del sistema, de manera independiente del proveedor; en ocasiones, esto sirve como un respaldo legal, entre ambas partes, para cumplir los objetivos y acuerdos dentro del documento.

Una vez terminado el análisis de los requerimientos, se debe plasmar el resultado obtenido en algún artefacto que permita su consulta para cualquier fin. Dependiendo el tipo de requerimiento, es el artefacto utilizado. Este estudio, tomará la clasificación básica de los requerimientos utilizada para una ingeniería inversa, que se mencionaron en el capítulo 2. Estos requerimientos son:

- ⇒ Requerimientos funcionales
- ⇒ Requerimientos no funcionales
- ⇒ Requerimientos técnicos

Esta clasificación de los requerimientos ya fue descrita, por tanto, sólo se enunciarán las características que corresponden a su documentación, según su naturaleza.

4.4.1. Requerimientos Funcionales.

Los requerimientos funcionales, son aquellos que describen que debe hacer el sistema, las necesidades que debe cumplir, generalmente identificados por un verbo. La especificación de estos requerimientos se realiza mediante **Casos de Uso**.

Un caso de uso es una descripción textual de las interacciones entre cualquier actor y el sistema que describe una funcionalidad; siendo un actor, cualquier entidad externa que interactúa con el sistema. El caso de uso narra el comportamiento del sistema desde el punto de vista del actor.

Dentro de la documentación de un caso de uso, se debe tener en cuenta los siguientes puntos:

- ⇒ Identificador del caso de uso: Clave del caso de uso, que identifique de forma rápida el documento y su contenido. Este identificador debe ser único para toda la documentación referente al sistema.
- ⇒ Nombre del caso de uso: Nombre que describa de forma rápida y específica la funcionalidad contenida en el caso de uso. Este nombre debe ser único en todo el sistema.
- ⇒ Autor del documento: Identifica al responsable de analizar el requerimiento y documentar las necesidades.
- ⇒ Fecha de creación del documento: Identifica la fecha en la que se realizó el análisis y especificación del documento.
- ⇒ Histórico del documento (autor y fecha): Identifica modificaciones, actualizaciones y autorización del documento, contiene autor de la acción realizada al mismo y la fecha.
- ⇒ Actores: son aquellos elementos que interactuaran con el sistema, generalmente es un usuario asociado a un rol, pero en otras ocasiones es otro sistema que se comunica de forma independiente con el sistema.
- ⇒ Breve descripción: Describe la funcionalidad que contiene el caso de uso en grandes rasgos.
- ⇒ Precondiciones: contiene las condiciones necesarias para recorrer el flujo básico y los flujos alternos.
- ⇒ Poscondiciones: Describe el estado final del sistema o el resultado obtenido una vez recorrido el caso de uso.
- ⇒ Reglas de negocio asociadas: Son aquellas condiciones que pertenecen al proceso natural del negocio, sin las cuales, no tiene razón de ser. Una regla de negocio es aquella condición necesaria para que el proceso se cumpla. En caso contrario se detiene el flujo del proceso y es imposible continuar con la operación. Estas condiciones, por definición deben ser parte del sistema ya que forman parte medular de la necesidad misma.

- ⇒ Flujo básico: Es la especificación paso a paso para cubrir con la funcionalidad principal del sistema y llegar al resultado deseado sin ningún error o algo que interrumpa el proceso.
- ⇒ Flujos alternos: Es la descripción de los pasos o caminos que se desvían del flujo básico o distinto de este, que al final concluyen en el mismo resultado o en un resultado alternativo en el caso de uso.
- ⇒ Excepciones: son aquellos pasos que controlan las acciones externas que puedan generarse en la descripción de los flujos. Son todos los resultados no esperados dentro de los flujos.
- ⇒ Anexos: Pueden contener diagramas o prototipos que auxilian en el entendimiento de los flujos y excepciones dentro del sistema. Para fines del usuario, generalmente se utilizan prototipos, sin embargo estos deben ser utilizados con extremo cuidado, para no comprometer el diseño con el prototipo. Los diagramas son principalmente utilizados por los equipos de diseño y desarrollo, con el objetivo de entender de forma más clara los flujos del proceso.

4.4.2. Requerimientos no funcionales

Como se vio en el capítulo 2, los requerimientos no funcionales suelen ser atributos, restricciones o especificaciones de cómo debe funcionar el sistema o cómo debe ser construido. En algunas ocasiones pueden ser atributos de calidad que debe cumplir el mismo.

Una técnica muy utilizada para validar y documentar un requerimiento no funcional es la técnica de escenarios. Esta técnica, consiste en tener entradas, condiciones y salidas, donde las entradas son el estímulo generado por el usuario, las condiciones son entornos en los que se encuentra el sistema al presentarse el estímulo y quien recibe el estímulo; finalmente, dentro de las salidas, son aquellas actividades que se realizan una vez detectado el estímulo, las salidas tienen una métrica que permite cuantificar la respuesta.

4.4.3. Requerimientos técnicos

Los requerimientos técnicos son aquellos que describen el detalle de cómo se comunicará el sistema a construir con otros elementos tecnológicos bien definidos y que no se encuentran dentro del alcance para su adaptación. Por ejemplo, un sistema que se conectará a una base de datos con un gestor determinado, debe cumplir con ciertas características para permitir la comunicación. O pudiera ser

que el sistema a construir intercambiará información con otro sistema, por lo tanto los detalles de cómo se realizara esta comunicación deben ser adaptados al sistema ya existente.

Generalmente esta descripción de las características técnicas de un sistema se lleva a cabo por una descomposición en módulos del sistema actual y ficha técnica para cada funcionalidad que lo sea requerido. Este tipo de requerimientos son de gran ayuda a los casos de uso, ya que si el caso de uso describe que se debe hacer, una ficha técnica nos ayuda a resolver el cómo lo va a hacer. Se debe recordar que el análisis sólo describe el que, sin embargo, si un sistema ya existe o si el cliente realiza una solicitud específica de esta comunicación se vuelve parte del requerimiento funcional, pero con un enfoque técnico.

4.5. Validación de los requerimientos

Los requerimientos obtenidos de una ingeniería inversa, deben ser evaluados por los con el fin de asegurar que la información proporcionada es correcta para el proceso del negocio en la actualidad y es suficiente para entender el negocio, modelar el proceso y diseñar el sistema. Para ello se presentan tres propuestas de listas de verificación. El objetivo es presentar una guía que precise de una manera puntual y sencilla si la información contenida en la documentación representa de forma clara y completa el proceso y sus necesidades. Todo esto basado en estándares establecidos para la definición de los requerimientos.

4.5.1. Checklist guía para analista

El análisis comienza cuando se obtiene un requerimiento. Con la finalidad de mantener un margen de calidad para el desarrollo del sistema, los requerimientos deben cumplir ciertas características de acuerdo a las normas ISO 2009. Esto garantiza el correcto diseño, desarrollo y funcionamiento del sistema.

A continuación se propone una lista de revisión para el analista, conocido también como *checklist*. El objetivo de este artefacto es que apoyar al levantamiento de un requerimiento y facilitar su análisis. Sirve como una guía que permite identificar el tipo de requerimiento con base en las características que lo describen. Mediante la identificación y clasificación de los tipos de requerimientos. También ubica el nivel de prioridad y la interacción que pudieran tener con otros requerimientos a fin de prever los riesgos por las dependencias entre ellos.

Tabla 2. Justificación de checklist para Analista.

Cualidad de ISO.	Pregunta	Justificación
Necesarios	1. ¿El requerimiento representa alguna nueva entidad dentro del modelo de dominio?	Si es una nueva entidad es muy probable que se encuentre asociada a otros requerimientos o que requerimientos posteriores dependan de esta.
	2. Si el requerimiento representa una nueva entidad ¿se encuentra descrito dentro del artefacto de negociación?	Se debe tener especial cuidado en no rebasar las fronteras de la negociación, ya que de esto depende que el proyecto llegue a alcance en tiempo y costo. Es común que el cliente solicite alguna funcionalidad para una entidad que no se encuentra especificada en la negociación. Al aceptarla, el analista, se ve obligado a incluir la nueva entidad y por tanto extender así a otras funcionalidades que ocasionan un crecimiento del sistema y no alcanzar los objetivos en tiempo y costo.
	3. ¿El requerimiento interviene con alguna entidad definida en el modelo de dominio?	El cliente no siempre tiene idea de cómo expresar sus necesidades por lo que puede proporcionar los requerimientos en desorden, por ello es necesario diferenciar un requerimiento de una entidad. Una entidad es definida por V. Mannio, Michael como un conjunto de datos generalmente sobre un tema al que puede afectarse de forma conjunta. Un requerimiento es una necesidad para que el sistema funcione. Sin embargo, a pesar de que las entidades se encuentren dentro de la negociación se debe identificar que funcionalidades de esta no. Si el cliente especifica una funcionalidad es necesario tratar de ubicarla en alguna entidad, si no se encuentra como atributo de alguna entidad, entonces pertenece a una nueva y por lo tanto es necesario validar esta nueva entidad (Ver pregunta anterior).
	4. De omitir este requerimiento, ¿el sistema dejaría de funcionar de acuerdo a lo descrito por el flujo definido por el usuario?	En algunas ocasiones, entre más abierto dejemos los requerimientos más se puede perder la objetividad del sistema. Se debe analizar si el requerimiento especificado por el cliente es realmente necesario para él. Si no se tiene especial cuidado en estas situaciones se puede tener un sistema lleno de detalles y características, pero que no es funcional. Si el requerimiento se omite y afecta el flujo del sistema, existen dos opciones, la primera es que el flujo no fuera bien descrito y el requerimiento sea realmente necesario o el segundo es que el requerimiento es innecesario.
	5. De omitir este requerimiento, ¿podría ser afectada alguna entidad del modelo de dominio?	El impacto del requerimiento puede tener diferentes niveles, no es lo mismo que sea afectado un flujo a que sea afectado todo el proceso. Si alguna de las entidades desaparece es afectada directamente por el requerimiento, lo más seguro es que este es necesario para el sistema
	6. ¿El requerimiento fue definido por el cliente?	Si el requerimiento fue definido por el cliente se debe analizar si es un requerimiento dentro del alcance, funcional o no funcional, ya que de esto puede depender que sea o no necesario para el sistema. Quizá sea un requerimiento para el proceso del negocio, pero que ya fue cubierto por otro requisito, no olvidemos que el mejor conocedor del negocio es el usuario.
	7. ¿El requerimiento fue sugerido por el analista?	Generalmente el requerimiento sugerido por el analista, suele ser una necesidad para adaptar el sistema al negocio o porque existe un vacío dentro del proceso de negocio. En ocasiones el cliente esta consiente de esta deficiencia en el proceso y es su decisión si se cubre o no esta necesidad siempre y cuando la solución no exceda las fronteras del proyecto.
Concisos	8. ¿El requerimiento se encuentra descrito como una característica que se desea para el sistema?	Si un requerimiento es definido como una característica hace referencia a un requerimiento no funcional, es decir, una cualidad que debe poseer el sistema, que puede inferir en cómo debe funcionar el sistema, pero no así en que debe hacer. Estos requerimientos detallan los otros requerimientos, generalmente funcionales.

Cualidad de ISO.	Pregunta	Justificación
	9. ¿El requerimiento se encuentra descrito con el nombre bien definido de algún elemento (sustantivo)?	Si un requerimiento es definido como el nombre de algún elemento (sustantivo), se está haciendo referencia a un elemento o una entidad dentro del proceso. De estos requerimientos es necesario detallar, qué tareas realiza este sustantivo y cuáles son sus cualidades, ya que generalmente aquí nacen las clases que serán implementadas en el sistema.
	10. ¿El requerimiento se encuentra descrito como una acción (verbo)?	Si el requerimiento es descrito como una acción (verbo), la referencia es a un requerimiento funcional. Estos requerimientos se encuentran asociados a una entidad y pueden desencadenar otros requerimientos.
	11. ¿El requerimiento describe más de una necesidad del sistema?	Es muy común que de primera instancia, el usuario no tenga claro cómo definir las necesidades de su proceso, así que menciona muchos requerimientos en un sólo momento. Cuando esto ocurre, debe identificar y clasificar las necesidades, para ir detallándolas poco a poco, e ir obteniéndolos uno a uno, así como ir dando prioridades a los requerimientos.
Completos	12. Se describe ¿Quién o qué lo usa el requerimiento?	Cuando se describe, quien o que usa el requerimiento, se obtienen los actores del sistema, es decir, quienes interactuarán con él y bajo que lineamientos.
	13. Se describe ¿qué elementos intervienen con requerimiento?	Cuando el requerimiento describe otros elementos que intervienen con él, describe la posible dependencia con otros casos de uso o en su defecto su relación con otros casos de uso o actores que intervienen en forma indirecta.
	14. Se describe ¿cómo funciona el requerimiento dentro del flujo del negocio?	Es necesario que se tenga un flujo detallado y bien definido para referir un proceso del negocio, aunque por lo general se entiende en aspectos muy generales. Se debe tener identificado, que parte de todo el modelo corresponde el requerimiento que se está obteniendo, esto sirve como guía para asegurarse de cubrir todo el proceso.
	15. Se describe ¿cuándo se usa el requerimiento dentro del flujo del negocio?	Tener identificados los requerimientos no es suficiente, es necesario tener definido en qué momento son necesarios dentro del flujo del negocio, con el fin de facilitar su implementación.
	16. Se describe ¿dónde se usa el requerimiento, dentro de las entidades del modelo de dominio?	Es necesario asociar los requerimientos de a la entidad a la que pertenece, prioridad y tipo. De no ser así, se debe revisar la pregunta 12.
	17. Se describe ¿por qué es necesario para el sistema?	Todos los requerimientos deben tener una justificación, a veces la dependencia de unos requerimientos a otros es meramente formal, es decir, pueden o no ser implementados sin afectar al flujo del negocio, sin embargo. Si un requerimiento es omitido y afecta en alguna forma el proceso del negocio y por obvio sus resultados, dicho requerimiento es necesario para el sistema y no puede ser omitido ya que es la razón de ser del proceso mismo. Generalmente estos requerimientos contienen las reglas del negocio.
Consistentes	18. ¿El requerimiento es especificado por más de un usuario?	Existe la posibilidad de que los requerimientos sean proporcionados por más de un usuario o cliente. En este caso, se debe obtener la especificación de los requerimientos de mutuo acuerdo entre los usuarios o en su defecto, de ambos usuarios al mismo tiempo, con el fin de evitar inconsistencias en la solicitud. Un requerimiento puede ser definido dependiendo de las necesidades de cada cliente. Sólo se debe tener una definición por cada requerimiento, de no ser así, se consideran excepciones dentro del sistema.
	19. ¿El requerimiento contradice o se opone a otros requerimientos?	La asociación entre requerimientos pueden no sólo ser de dependencias, si no de oposición. Es decir, si un requerimiento contradice a otro se debe aclarar la oposición y definir cuál será el requerimiento final. Esta clase de contradicciones suelen surgir cuando se tiene la definición de los requerimientos por más de un usuario.

Cualidad de ISO.	Pregunta	Justificación
No ambiguo	20. ¿El requerimiento tiene coherencia dentro del modelo de dominio?	El requerimiento solicitado, debe estar asociado a cualquier entidad dentro del modelo de dominio o representar un nuevo elemento del negocio. Un modelo conceptual de un negocio contable, no puede tener requerimientos médicos, a no ser que generen algún gasto o inversión, de lo contrario se sale del entorno del sistema y de las fronteras del proyecto.
	21. ¿El requerimiento es inconfundible con otros requerimientos?	La claridad con la que se define un requerimiento debe evitar su confusión con otros requerimientos, que pudieran parecer iguales, pero sólo ser similares en cuanto a su necesidad. Si un requerimiento se confunde con otro, puede generar ambigüedad dentro del sistema, de ahí la necesidad de tener un glosario de términos bien documentado.
	22. ¿El requerimiento puede ser identificado dentro de otros requerimientos?	Un requerimiento no sólo debe ser inconfundible ante otros requerimientos, sino que además debe ser identificado de otros, ya sea por su nombre, por sus cualidades o cualquier otra característica o atributo que posea.
Verificable	23. ¿El requerimiento puede ser ubicado dentro del sistema?	Si un requerimiento puede ser ubicado dentro del sistema, quiere decir que puede ser verificado, es decir, se puede evaluar si se implementó o no.

A pesar de las características con las que debe contar una analista, antes mencionadas, esta herramienta resulta un buen apoyo para quienes comienzan en esta “disciplina”. La propuesta anterior se encuentra descrita como cuestionario con dos opciones de respuestas, sí o no. A pesar de que resulta muy largo nos permite cubrir un área más amplia de aquella información que a simple vista no es notoria, incluso a realizar el análisis mismo al obtener los requerimientos. La ventaja es que se van evaluando los requerimientos desde un levantamiento y es posible a ahorrar tiempo en entender las relaciones que existen entre todos los requerimientos.

El artefacto final de describe en la Tabla 1. Checklist para analista del Anexo 2. Artefactos de evaluación.

4.5.2. Checklist para usuario

El lenguaje que comúnmente se utiliza para los clientes son términos de negocio y no es recomendable hablar en términos técnicos. Por lo tanto al validar un requerimiento con el cliente el *checklist* debe verificar que toda la funcionalidad y todas las condiciones se encuentran claramente descritas, para llevar a cabo su ejecución. No se debe olvidar que a partir de la claridad de esta especificación se llevara a cabo la construcción o mantenimiento del sistema.

El *checklist* que aquí se propone, puede ser utilizado para validar principalmente requerimientos funcionales mediante casos de uso, sin embargo contiene elementos para validar requerimientos no funcionales y técnicos, independientemente del artefacto o técnica con la que sean documentados.

Este artefacto tiene el objetivo de validar la calidad en la especificación de requerimientos, ya sean funcionales, no funcionales o técnicos. A pesar del tipo de requerimiento o la metodología que se maneje, la documentación debe ser clara y entendible para cualquier interesado en el proyecto. Este documento se encuentra fundamentado en la norma ISO 2503n.

Tabla 3. Justificación de checklist para usuario

Pregunta	Justificación
1. ¿El nombre del documento describe claramente los requerimientos a los que se encuentra asociado?	El nombre del requerimiento especificado, debe ser intuitivo para la funcionalidad contenida.
2. ¿El documento describe claramente los tipos o niveles de usuarios que actúan con en el requerimiento?	Es un complemento de la pregunta 1 por que es necesario identificar los usuarios que tendrán contacto con el requerimiento o intervendrán en su proceso.
3. ¿Es posible confundir el requerimiento especificado con algún otro?	Este punto también corresponde que un requerimiento debe ser redactado con claridad, para evitar que pueda ser confundido con requerimientos similares.
4. ¿La descripción textual del documento refiere claramente las características del requerimiento?	La especificación debe evitar ambigüedades.
5. ¿Las características descritas cubren las necesidades especificadas inicialmente para el requerimiento?	La especificación del requerimiento debe ser un detallado del requerimiento y contener toda la información solicitada por el cliente.
6. ¿Existe una asociación coherente entre el documento y la naturaleza del requerimiento solicitado?	El documento en el que se encuentra especificado el requerimiento puede ser estándar para todos o especificado para el tipo de requerimiento. Sin embargo, debe estar ordenado de tal forma que su comprensión sea rápida.

Pregunta	Justificación
7. ¿La descripción, satisface condiciones de la etapa del negocio a la que pertenece el requerimiento?	La descripción del requerimiento debe llenar todas las condiciones que le confieren, por cada etapa del negocio, pueden existir muchos requerimientos y cada uno cubre una característica de la etapa.
8. ¿Se utiliza un lenguaje conocido por los involucrados?	La especificación de los requerimientos debe ser de forma sencilla y clara, por ello se debe usar un lenguaje común o uno cuyo contexto sea entendido por los interesados.
9. ¿Los términos desconocidos o complejos utilizados en estos documentos se encuentran definidos en éste u otro artefacto?	En ocasiones, el negocio requiere términos especializados que pueden no ser claros para todos, por lo tanto se deben tener detallados estos términos en esto o algún otro documento.
10. ¿La especificación, contiene las condiciones necesarias para realizar pruebas al sistema recorriendo sus condiciones o validando la existencia del requerimiento?	De acuerdo a la norma ISO 2500, todo requerimiento debe tener la característica de ser validado. Por su naturaleza, el requerimiento debe ser aprobado, ya sea por sus condiciones o por el simple hecho de existir en el sistema. Por ejemplo, un requerimiento funcional es validado de acuerdo a las condiciones que debe cumplir, para ser útil al negocio y al sistema. Un requerimiento funcional es validado a partir de su existencia en el sistema.
11. ¿Se describe paso a paso el flujo para cubrir el requerimiento?	Una buena especificación guía paso a paso el flujo hasta describir como debe existir el requerimiento en el sistema.
12. ¿El documento describe lo sugerido por el consultor?	La especificación no debe tener supuestos, si lo descrito en el documento, es lo sugerido por el consultor, se debe analizar claramente por el cliente ya que puede no ser lo que el negocio requiere.
13. ¿El documento describe una característica solicitada por el cliente?	Un requerimiento no debería ser especificado a detalle hasta no tener una solución sobre si es viable su implementación en el sistema. Si el documento describe lo solicitado por el cliente, quiere decir que el analista ya encontró una solución viable para satisfacer su necesidad.
14. ¿El documento fue especificado por el cliente y el consultor?	La especificación ideal es la que se realiza de común acuerdo entre cliente y consultor, tener esta especificación reduce el riesgo de error en el análisis, aunque por lo general la especificación es por los dos puntos anteriores.
15. ¿El documento describe alguna entidad del negocio?	Si se describe alguna entidad del negocio, se deben detallar las características y funciones que posee; así como los elementos que interactúan con él.
16. ¿El documento una característica del negocio?	Si la descripción es de alguna característica del negocio se debe detallar a que entidad pertenece y que funcionalidades interactúan con ella.
17. ¿El documento describe un proceso del negocio?	Si la descripción del requerimiento pertenece a algún proceso del negocio, se debe detallar que elementos intervienen en él y a que entidad pertenece.
18. ¿El nivel de detalle para la especificación en el documento es uniforme para todos los requerimientos de este tipo?	El nivel de detalle por tipo de requerimiento debe ser estándar. Si se especifican los requerimientos a diferente nivel, la abstracción puede crear ambigüedades en el diseño del sistema y conflictos de comunicación en su codificación.
19. ¿El documento justifica la necesidad del requerimiento solicitado?	Toda especificación de un requerimiento debe justificar su necesidad en el sistema
20. ¿El documento describe de forma clara en que momento hace referencia a otro artefacto?	Si la especificación requiere la cooperación de otro documento para su entendimiento se debe especificar de forma clara y precisa, en que momento es requerido y con qué finalidad.
21. ¿Se describe de manera explícita todos los elementos con los que se encuentra relacionado el requerimiento?	Independiente a la naturaleza del requerimiento, en algún punto del documento se debe describir que elementos se relacionan con el requerimiento y de qué manera, a fin de tener un panorama general y detallado de él.

Pregunta	Justificación
22. ¿Se describe, textualmente, en que momento surge la dependencia con otros requerimientos?	En el caso de los requerimientos funcionales, una funcionalidad puede provocar muchas más, sin embargo, se debe describir el momento y la acción específicos que inician esta serie de opciones. Esta extensión abarca a otros requerimientos funcionales, así como a requerimientos no funcionales. Según sea la necesidad.
23. ¿Una modificación a este documento, impacta a más de un requerimiento?	La abstracción del requerimiento debe ser lo suficiente, para que su especificación sea lo más atómica posible, es decir, una alteración a este documento debe afectar a la ningún o la mínima cantidad posible de otros documentos.
24. ¿Se describe claramente el tipo de dependencia que tiene con otros requerimientos?	Para tener una buena especificación, no basta con conocer la dependencia que el requerimiento tiene con otros, se debe describir de qué tipo es esta dependencia y a que se debe.
25. ¿Se han especificado todos los requerimientos para el sistema?	Una vez especificados todos los requerimientos del sistema, bajo un criterio aceptable, se debe cerrar como primera versión el análisis, bajo el entendido de que lo especificado es lo que tendrá el sistema, no más, ni menos requerimientos. Esto es con el fin de tener una línea base para iniciar el diseño, sin embargo no olvidemos que los requerimientos pueden sufrir modificaciones, por ello se deben versionar.

En un inicio es posible encontrar resistencia por parte del usuario. Por ello es necesario aclarar que la información que se brinde será documentada para fines del sistema. Si se explica claramente que el sistema debe cubrir sus necesidades dentro de lo pactado y es el sistema el que debe diseñarse y funcionar de acuerdo al proceso, es seguro que el usuario coopere para brindar la información necesaria; pero esto ya es trabajar con empatía hacia el usuario.

El diseño final de este checklist se encuentra descrito en la Tabla 2. Checklist para usuario, contenida en el Anexo 2. Artefactos para evaluación, donde sólo se contemplan las instrucciones, preguntas y respuestas.

4.5.3. Checklist para diseño

Es muy común que una vez aceptado el caso de uso por parte del cliente, se encuentre con el punto de que es insuficiente la información para el equipo de diseño. A fin de evitar este problema, la Tabla 4. Justificación de checklist para diseño, propone una lista de verificación para evaluar la información proporcionada a diseño contemplando los aspectos que resultan fundamentales para esta etapa. El objetivo de este *checklist* es validar que la información proporcionada sea completa y se encuentre claramente definida para que se realice un buen diseño.

Este tiene la facilidad de evaluar diagramas de UML, con el fin de validar que la funcionalidad especificada tenga un nivel de detalle entendible y digerible para el diseño. Se encuentra fundamentada en Unified Modeling Language (UML).

Tabla 4. Justificación de checklist para diseño

Diagrama de UML	Pregunta	Justificación
Diagrama de casos de uso	1. ¿Se contemplan todos los actores del sistema?	Identifica quienes interactúan con el sistema de manera directa
	2. ¿El nombre de cada actor del sistema es único e inconfundible?	Es muy común que los actores correspondan a roles dentro del sistema, donde las actividades a realizar por el actor corresponden a ciertos permisos Los actores deben estar bien identificados y ser inconfundibles, ya que una confusión puede brindar permisos a una funcionalidad que no corresponde al rol, como el caso de autorizar una póliza de banco o emitir un diagnóstico final en un hospital.
	3. ¿El nombre de cada caso de uso es único e inconfundible?	De acuerdo a Jacobson, Booch y Rumbaugh, la funcionalidad debe ser descrita por una acción (verbo) del sistema y esta debe ser única dentro de todo el proceso
	4. ¿Existe un caso de uso por funcionalidad?	Por definición un caso de uso describe funcionalidad, y el nivel de abstracción en el modelo debe ser uniforme o existir un caso de uso por funcionalidad.
	5. ¿Se visualizan correctamente las dependencias entre los casos de uso (inclusión o extensión)?	La dependencia o cualquier otra relación entre los casos de uso deben estar bien identificadas para evitar desarrollos erróneos por confusión.
	6. ¿Se encuentran bien especificados los límites del sistema?	Recordemos que es responsabilidad del analista recabar toda la funcionalidad que corresponde al sistema de acuerdo a lo negociado. Si se identifica un requerimiento aceptado que no se encuentre el límite puede afectar la entrega del proyecto en tiempos y costo.
	7. ¿Todos los casos de uso se encuentran asociados al menos a un actor u otro caso de uso?	Toda la funcionalidad depende de un actor con el que interactúa. Un sistema no puede funcionar por sí sólo, incluso el tiempo mismo puede ser un actor del cual dependen las acciones a realizar.
	8. ¿Todos los actores se encuentran asociados al menos a un caso de uso u otro actor?	Por definición actor es todo aquel ente que interviene con un sistema. Por lo tanto, todo actor debe intervenir en el sistema. En algunos casos, existe dependencia entre actores, como generalidades o especificaciones de un actor.
Diagrama de clases	9. ¿La información de cada clase se encuentra en su sección correspondiente?	La información de cada clase se encuentra en su sección correspondiente - En este diagrama, cada clase se encuentra dividida en tres partes. - La primera se encuentra en la parte superior y corresponde al nombre de la clase. - La segunda se encuentra en la parte media y corresponde a las propiedades o características de la clase. - La tercera se encuentra en la parte inferior y contiene los atributos o funciones que puede realizar la clase.
	10. ¿El nombre de la clase es único e inconfundible?	Por lo general los diagramas de clases elaborados en análisis representan el negocio, por lo que sus nombres corresponden a entidades del mismo. Si alguna pudiera confundirse o es igual a otro de debe validar porque existen 2 clases con el mismo nombre y como podrían diferenciarse.
	11. ¿Se encuentran bien definidos las características?	Se encuentran bien definidos las características, transacciones y permisos dentro de cada clase

Diagrama de UML	Pregunta	Justificación
	12. ¿Se encuentran bien definidas las transacciones dentro de cada clase?	Es importante identificar cuando las transacciones en una clase están restringidas para otras clases por condiciones del negocio, a pesar de que ya se encuentran especificadas en el caso de uso.
	13. ¿Se encuentran bien definidos los permisos dentro de cada clase?	Es importante identificar cuando las transacciones en una clase están restringidas para otras clases por condiciones del negocio, a pesar de que ya se encuentran especificadas en el caso de uso.
	14. ¿Se visualizan correctamente las relaciones entre las clases (asociaciones, herencias, agregaciones y dependencias o instancias)?	A pesar de que estas relaciones se establecen en diseño, en ocasiones análisis identifica algunas características como herencia, más allá de la relación.
	15. ¿Se muestra una cardinalidad específica para cada relación?	Es importante conocer la cardinalidad de las clases, ya que de esto depende un buen modelo en la base de datos.
Diagrama de estados	16. ¿El nombre de cada estado es único e inconfundible?	En un diagrama de estados deben quedar claros cuáles son los estados por los que pasará la entidad. Si el nombre de algún estado se parece o tienen el mismo nombre que otro, se debe aclarar por qué y especificar en qué se diferencian.
	17. ¿Todos los estados poseen al menos una transición que le permite comunicarse con uno o más estados?	Un diagrama de estados representa una secuencia de transiciones de una entidad entre etapas (estados) y bajo determinadas circunstancias. Un estado que no tiene comunicación con los demás, o no posee una comunicación con otros no tiene conexión con los demás.
	18. ¿Se identifica claramente las transacciones de entrada y salida de cada estado?	Es importante identificar las transacciones de entrada y salida de cada estado, porque de esta forma se identifica el flujo de la entidad a lo largo de todos los estados.
	19. ¿En las transiciones se encuentran claramente identificadas las condiciones de las que depende el cambio?	Las transiciones poseen las condiciones de las que depende un elemento para pasar de un estado a otro, estas condiciones deben ser claras y puntuales.
Diagrama de actividades	20. ¿El nombre del diagrama es intuitivo de la actividad a describir?	Los diagramas de actividad se guían por los casos de uso, en ocasiones describen un escenario específico del caso de uso o varios escenarios, sin embargo, el nombre debe identificar claramente las acciones a modelar.
	21. ¿Se encuentran claramente identificados los nodos (inicio y fin)?	Una actividad describe un conjunto de acciones determinadas para cumplir un objetivo. Y en estas se debe identificar donde es el comienzo y el fin.
	22. ¿El nombre de cada estado de acción es único e inconfundible?	Los estados acción, indica los puntos por los que puede pasar la actividad, donde se realiza una tarea en específico.
	23. ¿Se encuentran claramente definidas las transiciones en el flujo?	Es importante que estos diagramas muestren un flujo visualmente limpio para describir la transición dentro de los estados acción. Esto facilita el entendimiento tanto del diseñador como del desarrollador. En los diagramas donde las transiciones se encuentran mezcladas visualmente suelen confundirse los flujos.
	24. ¿Todos los estados acción, poseen una transición de entrada y una de salida?	Todo estado de acción debe poseer una transición de entrada y una de salida, ya sea a un punto de ruptura o a un punto final, de lo contrario el flujo podría ciclarse en sí mismo.
	25. ¿Se encuentran identificadas las actividades que se desarrollan de forma simultánea entre barras de	Es muy común que las tareas que se realizan de manera sincronizada no muestran gráficamente que son de esta manera, o su diseño es muy confuso, en este caso es necesario aclarar que tareas son de este tipo y que afectaciones pueden tener en otros puntos.

Diagrama de UML	Pregunta	Justificación
	sincronización?	
	26. ¿Los nodos de decisión muestran todas las transiciones de acuerdo a la condición que deben cumplir?	Un nodo de decisión comúnmente tiene una salida positiva y una negativa, sin embargo existen casos en los que puede tener más de 2 salidas, dependiendo de la condición. En este caso es conveniente
	27. ¿Los carriles agrupan de forma ordenada las responsabilidades en el flujo?	Los carriles como bien se sabe, aíslan tareas agrupándolas por responsables, por lo tanto deben estar bien delimitadas.
	28. ¿El flujo del diagrama es lógico y coherente?	En ocasiones pudiera parecer que los flujos del diagrama son correctos, pero al revisar el contenido es posible darse cuenta que no tiene coherencia una cosa con otra. Recordemos que el factor humano tiene riesgos, entre ellos por lo tanto se debe analizar el flujo de estos diagramas y resolver todas las dudas en el menor tiempo posible.
Diagrama de secuencia	29. ¿El nombre del diagrama describe de forma intuitiva el evento que desencadena las acciones en secuencia para el escenario específico?	Al igual que los diagramas de actividad, los diagramas de secuencia se guían por los casos de uso, en ocasiones describen un escenario específico del caso de uso o varios escenarios, sin embargo, el nombre debe identificar claramente las acciones a modelar, pero difieren en su representación ya que este diagrama muestra mensajes en continuidad de tiempo.
	30. ¿Se encuentran claramente identificados los objetos que deben interactuar en la secuencia de acuerdo al caso de uso?	Se deben identificar los objetos que intervendrán estos mensajes a fin de satisfacer la secuencia de comunicación.
	31. ¿La secuencia de comunicación entre los objetos se realiza de acuerdo al flujo indicado en el caso de uso?	El flujo de comunicación debe realizarse de acuerdo al caso de uso, ya que es este quien describe paso a paso la comunicación.
	32. ¿Todos los objetos en el diagrama poseen al menos un mensaje de comunicación con otros?	Todos los objetos dentro de este diagrama deben tener comunicación con otro, mediante un mensaje, de lo contrario se rompe la secuencia y el sentido del diagrama mismo.
	33. ¿Se describe de forma clara el período de activación de cada objeto?	Todos los objetos poseen una línea de vida y a lo largo de esta los momentos en los que se encuentra activos dependiendo de la comunicación que tenga con otros elementos.
Diagrama de colaboración	34. ¿Se encuentran claramente identificados los objetos que deben interactuar?	Este diagrama modela una vista estática de los diagramas de secuencia, por lo que es importante identificar los objetos que tendrán comunicación en un determinado momento y con quien será la comunicación directa.
	35. ¿La secuencia entre mensajes se encuentra especificada mediante un número y una flecha dirigida?	De acuerdo a UML los diagramas de colaboración deben especificar el flujo de su comunicación entre los distintos objetos y de donde hacia donde fluye la comunicación. Esto se realiza mediante una flecha que indica el flujo de la comunicación y un número que indica la secuencia.
	36. ¿Todos los objetos poseen al menos un enlace de comunicación con otro objeto?	Al ser un diagrama de comunicación, todos los objetos deben poseer al menos un punto de comunicación con otros, quizá no envíen una respuesta, pero si ejecutan una acción dependiendo de su condición de entrada.

Diagrama de UML	Pregunta	Justificación
Diagrama de componentes	37. ¿Se encuentran claramente identificados los componentes del sistema?	Es importante identificar los componentes de acuerdo a los módulos que formaran parte de la arquitectura.
	38. ¿Se encuentran claramente identificadas las relaciones entre componentes (dependencias)?	Es importante conocer las relaciones entre componentes. Si surge el caso en el que mover un componente afecta a otro (cosa que no debería suceder) se debe tener identificada la dependencia o cualquier otra relación que provoque ese cambio.
	39. ¿Las clases contenidas en el componente se encuentran bien definidas?	Se debe recordar que un componente contiene clases con un objetivo común. Antes de diseñar un diagrama de componentes se debe tener un diagrama de clases detallado que especifique cuales son las características por las cuales se están armando dichos componentes.

La versión final de este checklist se encuentra en la Tabla 3. Checklist para diseño, del Anexo 2. Artefactos de evaluación.

4.6. Re estructuración del nuevo sistema.

Es necesario recordar que la ingeniería inversa, no siempre resulta en una reingeniería, pues la documentación obtenida de esta, puede ser utilizada únicamente para dar mantenimiento al sistema. Sin embargo, la documentación obtenida de este debe ser lo suficientemente completa y explícita como para generar un nuevo sistema.

El punto 4.2 hace referencia al modelado de un nuevo sistema a partir de la información obtenida de una ingeniería inversa, esta información se expresa en forma de diagramas.

Sin embargo, para la construcción de un nuevo sistema a partir de requerimientos obtenidos de una ingeniería inversa, sólo existen dos razones: una es la migración del sistema, es decir, el traslado del sistema de una tecnología a otra, la segunda es una reingeniería, la cual implica validar si los requerimientos obtenidos siguen teniendo validez en el negocio o requieren de alguna modificación.

El caso de una migración puede no ser complejo en cuanto a negociación o análisis, ya que un sistema debe funcionar igual en una u otra tecnología, pero no por esto se debe decir que es sencillo ya que la complejidad se tiene al replicar las necesidades de una tecnología a otra. Si el paradigma con el que se construyó el sistema fue útil hace 30 años, y se requiere una migración a una tecnología actual,

es necesario el diseño que evolucione tres décadas para el nuevo desarrollo del sistema.

En contra partida, el caso de una re ingeniería, puede no siempre desprender de una ingeniería inversa. Se supondrá el caso de tener un sistema perfectamente documentado, en tal caso, sólo se toma la documentación del análisis y se valida si las necesidades continúan siendo las mismas, a partir de esta validación se diseña y construye un nuevo sistema con un enfoque actualizado. Si bien, este proceso no requiere de mayor esfuerzo técnico, llegar a un acuerdo entre cliente y proveedor puede llegar a ser un caos, si no se tiene bien definido el alcance de la reingeniería. Pueden existir muchas modificaciones a los requerimientos, por ser obsoletos para el actual desarrollo del negocio, sin embargo también se deben tener fronteras para estas modificaciones a fin de poder marcar alcances en tiempo y entrega.

Para el interés de este estudio el enfoque fue una re ingeniería y la información que requiere de una ingeniería inversa, sin embargo no se olvide que la ingeniería inversa tiene distintos usos mencionados en el capítulo anterior.

REFERENCIAS BIBLIOGRAFICAS

- Kendall Kenneth, E. & Kendall Julie E. (1991). Análisis y diseño de sistemas. Prentice-Hall
- Pressman, Roger S. Ingeniería de Software (1993). Un enfoque práctico. Editorial McGraw-Hill. Tercera edición
- Weitzenfeld, Alfredo. (2005) Ingeniería de software orientada a objetos con UML, java e internet. Editorial Thomson.
- Fontela, Carlos (2011). *UML. Modelado de software para profesionales*. Editorial Alfaomega. 1ra edición.
- Grady Booch, James Rumbaugh, Ivar Jacobson (2005). *El lenguaje unificado de modelado. UML 2.0*. 2ª. Edición. Pearson Addison Wesley. España
- Coronado Pastor, Claudia Alejandra (2008). Ingeniería inversa de base de datos. UNAM
- Marigo G. Pittini, Felix O. García (2003). Calidad en el desarrollo y mantenimiento del software. 6ª Edición. Alfaomega. Ra-Ma

REFERENCIAS ELECTRONICAS

- Fernández Aedo, Raúl (2009). Capítulo 11: Analista de sistemas. Funciones. <http://www.emagister.com/curso-sistemas-informativos-diseno-creacion-1/analista-sistemas-funciones>

CAPITULO 5. SEGUIMIENTO A LOS REQUERMIENTOS

A pesar que los requerimientos pueden encontrarse en constante movimiento, al final de su obtención y documentación, se debe dar un seguimiento a cada uno, es decir, se debe identificar lo que cada requerimiento despliega a lo largo de todo el proyecto.

Muchas veces se da poca importancia a este punto, sin embargo, es importante identificar cuantos elementos de todo el sistema son afectados por un sólo requerimiento, debido a que es muy común que los requerimientos cambien y se debe tener en cuenta el impacto en todos los elementos que serán afectados.

5.1. SEGUIMIENTO DE LOS REQUERIMIENTOS

La gestión de requerimientos en ocasiones es un proceso sub estimado, actualmente algunas empresas deciden no invertir en este aspecto, ya que es el proceso muy costoso, quizá esta sea la razón por la que pocos autores, como Piattini, Jacobson, Weitzenfel y Fontela, mencionan la trazabilidad o rastreabilidad de los requerimientos.

Es bien sabido, por quienes han realizado un análisis o una estimación que es necesario tener un control en las solicitudes del cliente. Así mismo se sabe que es imposible mantener un requerimiento estático (aunque sería lo ideal), sin embargo si se puede llevar un control de sus modificaciones y/o ajustes.

Mantener la traza de un requerimiento a lo largo del proceso de desarrollo de software, permite realizar estimaciones de una forma menos complicada, esto no quiere decir que sea sencilla, ya que depende de muchos otros factores como los recursos, las capacidades e incluso el costo mismo del proyecto. Piatini (2003) menciona la trazabilidad como un factor de calidad, una característica que un sistema debería incluir como si se tratase de un requisito no-funcional. Las estimaciones no sólo infieren a modificaciones a los requerimientos, sino a la solicitud de nuevos. Resulta sorprendente ver como se puede facilitar el manejo de un proyecto, con un rastreo de los requerimientos.

Existen algunos proyectos en los que únicamente se requiere la codificación, es decir la mano de obra, esto se debe a que el análisis ya fue realizado y ya se tiene bien definida la necesidad y especificado cada requerimiento a detalle con sus características. Incluso en otros casos, ya se llega con el diseño y únicamente se solicita la codificación. En estos casos, la trazabilidad se inicia a través del diseño obtenido.

El diseño es el mapa de solución que debe satisfacer la empresa o equipo que realiza el desarrollo. Actualmente existen empresas que se dedican únicamente a la codificación, puede llegar directamente un requerimiento o incluso el diseño. Sin embargo a pesar de que los requerimientos originales ya fueron analizados, especificados y diseñados, el “equipo” de codificación debe analizar y entender el diseño que le ha sido proporcionado, para poder llevar a cabo su diseño interno de desarrollo.

Se debe tener muy claro que la trazabilidad debe iniciar con los requerimientos, pero no implica que esta siempre sea originada en la etapa de análisis. Si bien, quien llevó a cabo el levantamiento y análisis de los requerimientos no inició la traza de los mismos, en diseño se puede recibir el caso de uso como requerimiento para mapearlo a sus diagramas y/o componentes.

Piattini (2003) expone un meta-modelo para la trazabilidad de requisitos, si bien, no se expondrá dicho modelo si se mencionarán los puntos más importantes del mismo.

5.2. CARACTERÍSTICAS DE LA TRAZABILIDAD DE REQUERIMIENTOS

La trazabilidad de los requerimientos de un sistema, dependen en gran medida del análisis. Es muy importante resaltar que es en esta etapa donde se inicia la trazabilidad, no donde se define por completo. La razón se debe a que un requerimiento puede derivarse en muchos elementos hacia las siguientes etapas, y se debe tener identificado, cada elemento asociado a su requisito original, esto deriva en un crecimiento gradual de la trazabilidad. Es decir, las asociaciones a los requerimientos se vuelven incrementales hacia las siguientes etapas del desarrollo.

Algunas ventajas de una buena trazabilidad son:

- ⇒ Permite validar que la funcionalidad del sistema reúna las expectativas del cliente y que no se ha realizado la codificación de la funcionalidad superflua.
- ⇒ El análisis del impacto a los requerimientos originales posee una estimación más objetiva. Los enlaces permiten determinar las especificaciones que se verían afectadas.
- ⇒ Mejora la comunicación y cooperación entre los participantes del proyecto.
- ⇒ En caso de una solicitud de cambio, permite una fácil gestión, los participantes que deberían estar involucrados y/o informados pueden ser determinados fácilmente (aunque generalmente son todos).

⇒ Asegurar que la contribución de cada individuo sea considerada y registrada.

⇒ Mejorar la comprensión del sistema por parte del cliente y con ello su aceptación.

A pesar que las metodologías ágiles evitan la mayor cantidad de documentación posible, se reconoce la utilidad de esta práctica, ya que las estimaciones resultan favorables para la metodología.

Para iniciar una buena trazabilidad, se debe contar con ciertos lineamientos pre establecidos como son:

1. El alcance del proyecto
2. Los requerimientos bien especificados (aunque en la práctica suele no ser así)
3. Análisis claramente definido y aprobado por el cliente

Es posible que existan otros factores que infieran en la trazabilidad, pero básicamente con estos se puede iniciar el seguimiento a los requerimientos.

5.3. MATRIZ DE LA TRAZABILIDAD

Existen muchas formas de llevar un seguimiento a los requerimientos, entre ellos se encuentran modelos, registros o simplemente mediante una matriz.

La matriz de trazabilidad es un artefacto sencillo de elaborar y comprender en comparación con otros, sin embargo, pueden llegar a generarse tantas matrices como elementos asociados. Su finalidad es mapear los requerimientos contra cada elemento dentro del desarrollo de un sistema.

El diseño de una matriz de seguimiento, puede derivar en muchas otras más detalladas, que agilicen la administración de los requerimientos y su impacto en el sistema. A continuación se describen las matrices mas comunes en un proceso de desarrollo de software.

⇒ Requerimientos – casos de uso. Es la matriz inicial o básica es aquella que mapea los requerimientos con los casos de uso. Ver Tabla 1.

Tabla 1. Requerimientos – Casos de uso

Requerimientos	Casos de uso								
	Caso de Uso 01	Caso de Uso 02	Caso de Uso 03	Caso de Uso 04	Caso de Uso 05	Caso de Uso 06	Caso de Uso 07	Caso de Uso 08	Caso de Uso 09
Nombre del Requerimiento No. 01									
Nombre del Requerimiento No. 02									
Nombre del Requerimiento No. 03									
Nombre del Requerimiento No. 04									
Nombre del Requerimiento No. 05									
Nombre del Requerimiento No. 06									
Nombre del Requerimiento No. 07									
Nombre del Requerimiento No. 08									

⇒ Casos de uso – casos de prueba. Es una matriz derivada de la básica, ya que no depende directamente de los requerimientos, sino de los casos de uso. Esta matriz se debe diseñar inmediatamente después de la matriz requerimientos – casos de uso, con el fin de tener el plan de pruebas antes de la etapa de desarrollo (esto se debe a las pruebas que se realiza en la etapa de construcción). Ver Tabla 2.

Tabla 2. Casos de uso - Casos de prueba

Casos de uso	Casos de prueba CPR								
	CPR 01	CPR 02	CPR 03	CPR 04	CPR 05	CPR 06	CPR 07	CPR 08	CPR 09
Caso de Uso 01									
Caso de Uso 02									
Caso de Uso 03									
Caso de Uso 04									
Caso de Uso 05									

⇒ Casos de uso – Clases. Esta matriz, mapea los casos de uso, contra las clases a construir en una etapa de desarrollo. Ver Tabla 3.

Tabla 3. Casos de uso - Clases

Clases									
Casos de uso	Clase_01	Clase_02	Clase_03	Clase_04	Clase_05	Clase_06	Clase_07	Clase_08	Clase_09
Caso de Uso 01									
Caso de Uso 02									
Caso de Uso 03									
Caso de Uso 04									
Caso de Uso 05									
Caso de Uso 06									
Caso de Uso 07									
Caso de Uso 08									

⇒ Casos de uso – Funciones ó Clases – Funciones. En este caso, tener una matriz de casos de uso contra funciones parecería demasiado detallado, sin embargo, en proyectos muy grandes o complejos, llegar a este nivel de detalle es muy recomendable, ya que el impacto que puede tener un requerimiento, puede no ser certero, sino se tiene una visión completa del sistema. Ver Tablas 4 y 5 respectivamente.

Tabla 4. Casos de uso - funciones

Funciones FN									
Casos de uso	FN 01	FN 02	FN 03	FN 04	FN 05	FN 06	FN 07	FN 08	FN 09
Caso de Uso 01									
Caso de Uso 02									
Caso de Uso 03									

O bien, se puede tener una matriz de Clases contra Funciones, esto pudiera ser más favorable para la etapa de construcción, ya que no sólo permite administrar los requerimientos, sino que permite una administración mas ágil de la misma construcción; sin embargo implica triangular los requerimientos que a final de cuentas son nuestro eje. La construcción de esta matriz depende tanto del líder técnico, como del diseñador, ya que son ellos quienes deben administrar las necesidades presentadas en el caso de uso con sus elementos de construcción.

Tabla 5. Clases - Funciones

Funciones FN									
Clases	FN 01	FN 02	FN 03	FN 04	FN 05	FN 06	FN 07	FN 08	FN 09
Clase_ 01									
Clase_ 02									
Clase_ 03									
Clase_ 04									
Clase_ 05									
Clase_ 06									

⇒ En general se puede tener una matriz que contenga todos los criterios aquí mencionados o más. Sin embargo, es necesario hacer notar que una matriz de este tipo, para proyectos grandes puede resultar muy compleja, ver Tabla 6, de ahí que se deriven de acuerdo a su necesidad. Es cierto, que se puede contar con otras que permitan tener un panorama más detallado del sistema, sin embargo, esto depende de la dimensión del proyecto y la complejidad.

Tabla 6. Requerimientos - Todos elementos

Elementos															
Requerimiento	Caso de Uso 01	Caso de Uso 02	CPR 01	CPR 02	CPR 03	CPR 04	Clase_ 01	Clase_ 02	Clase_ 03	FN 01	FN 02	FN 03	FN 04	FN 05	FN 06
Requerimiento 01															
Requerimiento 02															
Requerimiento 03															

REFERENCIAS BIBLIOGRAFICAS

- ⇒ Kendall Kenneth, E. & Kendall Julie E. (1991). Análisis y diseño de sistemas. Prentice-Hall
- ⇒ Pressman, Roger S. Ingeniería de Software (1993). Un enfoque práctico. Editorial McGraw-Hill. Tercera edición
- ⇒ Weitzenfeld, Alfredo. (2005) Ingeniería de software orientada a objetos con UML, java e internet. Editorial Thomson.
- ⇒ Fontela, Carlos (2011). *UML. Modelado de software para profesionales*. Editorial Alfaomega. 1ra edición.
- ⇒ Grady Booch, James Rumbaugh, Ivar Jacobson (2005). *El lenguaje unificado de modelado. UML 2.0*. 2ª. Edición. Pearson Addison Wesley. España
- ⇒ Mario G. Piattini, Feliz O. García (2003). *Calidad en el desarrollo y mantenimiento del software*. 6ª Edición. Editorial Alfaomega. España

CONCLUSIÓN

.

Una buena administración en los requerimientos es un paso fundamental en el desarrollo de un sistema. Definir de forma clara las necesidades reales del cliente o usuario, reduce en buena medida los riesgos de un desarrollo fallido. Se debe recordar que es complicado entender la solicitud de un cliente, pero hacerla del conocimiento del equipo resulta aún más complejo.

Existen estándares que si bien no eliminan los errores en el desarrollo de un sistema, sirven de guía para una buena obtención de requerimientos y análisis para solucionar el problema y diseñar un sistema a la medida.

También existen aquellos sistemas que ya se encuentran desarrollados, pero tienen áreas de oportunidad o problemas en algún aspecto, para su óptimo funcionamiento

Es muy cierto que dentro de un análisis toda la información es importante, pero si no se tiene un buen enfoque de lo que se desea obtener, es muy fácil perderse entre un mundo de información.

En la siguiente figura se ilustran los puntos más importantes dentro de la administración de los requerimientos para un sistema de software.



Figura 1. Descripción general del análisis

Donde la información y/o documentación de entrada contiene un conjunto de información que facilitará el entendimiento y modelado de un nuevo sistema. Esta información debe contar con las siguientes condiciones:

- Los documentos de la negociación autorizado, es decir, el documento que define el alcance del proyecto, no tener bien definido el alcance de un sistema e iniciar el análisis sin fronteras, puede derivar en un proyecto que tenga tantas iteraciones como ideas vayan surgiendo al usuario, convirtiendo el proyecto en posible fracaso debido a que nunca se llega a

tener una satisfacción completa del cliente y tampoco se tiene una construcción definida de un sistema.

- El documento de definición de criterios de aceptación autorizado. Es importante tener claro que se debe construir, pero también se debe tener claro, cómo se va a evaluar el producto final, no olvidemos que no existen sistemas perfectos, de allí tener un criterio de aceptación para el producto final, ayuda a solventar los de errores en los sistemas.
- Los requerimientos del sistema, obtenidos directamente del cliente o el usuario. Aunque suelen utilizarse de forma indistinta estos términos no refieren a lo mismo, ya que el cliente es quien negocia, decide y costea el sistema, mientras el usuario es, como su nombre lo define, utiliza el sistema. Es posible que el cliente sea el usuario, pero es poco común en empresas grandes. En estos casos, lo más usual es que el directivo costee el proyecto y defina los requerimientos en alto nivel y el usuario sea quien defina el detalle de los requerimientos (ya que es el usuario quien conoce la operación).

La siguiente etapa es el análisis, donde se descomponen todos los requerimientos y se ordenan de acuerdo a sus características y prioridades. La información y/o documentación generada dentro del análisis es la siguiente:

- Modelo de negocio o modelo conceptual. Es el modelo donde se tienen identificados todos los elementos del negocio y la comunicación entre estos. Este modelo se encuentra fundamentado en el estándar establecido por UML.
- Modelo de requerimientos (normalmente modelo de casos de uso). Es el modelo que identifica todos los requerimientos funcionales y sus asociaciones y dependencias entre si. Este modelo es muy útil, ya que identifica las acciones que debe realizar el sistema y las posibles acciones dependientes de forma interna.
- Especificación de requerimientos de software (normalmente especificación de casos de uso y las reglas del negocio que el sistema debe validar). Esta especificación es descrita bajo el lenguaje unificado de modelado, con el fin de darle una orientación a objetos, sin embargo, para algunas metodologías ágiles, es posible que esta especificación no sea utilizada, para este caso,

tener bien documentado el requerimiento solicitado puede ser suficiente (para sistemas pequeños).

- Diagramas de actividad de alto nivel. Diagrama que describe el flujo entre el actor y el sistema de una forma gráfica, resulta muy útil para el diseñador, ya que facilita el entendimiento del sistema.

Finalmente se tiene la información de salida del análisis cuyo objetivo es brindar la información necesaria a la etapa de diseño de las necesidades y características requeridas por el cliente, para facilitar la implementación de una solución que vaya de acuerdo a las necesidades planteadas. Esta información generalmente se presenta en documentos o modelos (artefactos) como son:

- Diagramas de casos de uso aprobado
- Diagramas de actividad detallado
- Diagramas de secuencia
- Diagramas de colaboración
- Especificación de requerimientos aprobada
- Matriz de trazabilidad.

Si es cierto que la mayor cantidad de investigaciones, estudios y estrategias para el buen desarrollo de un sistema nacen en el desarrollo del software, sin embargo hay aspectos que se pueden extender a cualquier proceso que desarrolle sistemas.

La matriz de trazabilidad es muy útil en el desarrollo de software, sin embargo, no se debe limitar a estos, puede ser una gran sorpresa ver esta matriz implementada en otros sistemas, como pudieran ser los sistemas de agua, para los arquitectos e ingenieros civiles, supongamos que se construirá un condominio donde se solicita que el sistema de agua, tenga ciertas características, como son el tipo de material, sus dimensiones e incluso la distancia entre una toma y otra, estos son requerimientos de los cuales pudiera depender el número de personas involucradas en este proceso, los costos del material. Utilizar una matriz en estos casos resultaría muy útil, ya que si alguno de los requerimientos solicitado llegase a cambiar, se tiene un panorama del impacto que esto puede generar en la construcción, incluso en la etapa del desarrollo en la que se encuentren.

Recuerde la definición de calidad enunciada por Pressman (1993), quien la puntualiza como “la concordancia con los requisitos funcionales y de rendimiento explícitamente establecidos, con los estándares de desarrollo explícitamente

documentados y con las características que se espera de todo software desarrollado profesionalmente”. Un sistema, es un conjunto de muchos elementos interactuando entre sí para cumplir un objetivo común, no necesariamente se debe referir a un software. Si bien, es cierto que muchas de las prácticas son desarrolladas aquí, también es cierto que muchas se pueden extender hacia otras áreas del desarrollo de sistemas.

Al final de todo proceso, debe satisfacer las necesidades del cliente establecidas en los requerimientos debe ser la prioridad y el punto culminante. Por ello la necesidad de tener una clara administración de los mismos y una aceptación del cliente que se ha entendido y definido de forma correcta su necesidad con la finalidad de gestionar de forma correcta una posible modificación en ambos sentidos.

La finalidad de investigar, obtener y analizar las mejores prácticas para la administración de requerimientos es identificar y recuperar los puntos de control más importantes dentro de un buen análisis. Aunque esto no garantiza en su totalidad que se construya un sistema que llene sus expectativas, el panorama establecido a lo largo de este estudio, si pone en evidencia situaciones y características fundamentales que permiten disminuir el riesgo de error, ya que lo definido en análisis es lo transmitido a diseño y esperado en pruebas. Desafortunadamente lo que ocurra en etapas posteriores al análisis se encuentra fuera de los límites en esta tesis, pero no fuera de vista.

El análisis es un punto fundamental en el desarrollo de un sistema, pero otros factores pueden afectar el éxito del proyecto. Algunos de estos son estimaciones en tiempo o esfuerzo mal calculadas, desarrollos sin estándares de calidad o entrega de productos sin aseguramiento por un área de pruebas; descuido de los acuerdos establecidos con el cliente y una mala administración de los recursos para el proyecto. Cada uno de estos puntos puede tener su propio estudio, desde la definición hasta el control y cierre del proyecto, ya sea por etapa o por características. Y aunque existen diversos estudios sobre estos puntos, ninguno se considera un método efectivo al 100%, ya que todo método o estándar se debe adaptar a las características particulares de cada proyecto en sí mismo.

Sería bueno, para un estudio posterior, analizar y documentar todos los factores que afectan el éxito en el desarrollo de sistemas y las consecuencias de no tomar precauciones ya establecidas en otros estudios.

ANEXO 1. DIAGRAMAS UML

Diagrama	Clases	
Fase	Análisis	Modelo: Dominio de negocio, clases conceptuales y relacionales
	Diseño	Modelo: Clases de software en nivel de detalle general
	Desarrollo	Modelo: Clases de software. Puede ser utilizado para documentación post-mortem. Es principalmente utilizado por ingeniería inversa en un alto nivel de detalle
	Pruebas	Modelo: Clases de software. Puede ser utilizado como documentación de referencia y suele ser en un nivel de detalle muy general
class Diagrama de clases		
Diagrama	Secuencia	
Fase	Análisis	Modelo: Secuencia detallada de casos de uso o historia de usuario en el sistema. Modelo: Secuencia extendido de diagramas de comunicación, esto si de desea puntualizar el tiempo. Se enfoca en mostrar mensajes entre entidades.
	Diseño	Modelo: Paso de mensajes entre objetos del sistema durante la ejecución de un escenario específico. Generalmente este nivel de detalle ya es más alto.
	Construcción	Modelo: Secuencia de escenarios en el software, es básicamente para documentación y suele ser obtenido por una ingeniería inversa. Para dar un mayor grado de información se contempla un alto nivel de detalle.
sd Diagrama de secuencia		

Diagrama	Estados	
Fase	Análisis	Modelo: Estados del sistema o de partes del mismo, el nivel de detalle puede ser muy general. Modelo: Cambios de estado de una entidad particular y eventos que provocan las transiciones. Nivel de detalle medio.
	Diseño	Modelo: Cambios de estado de un objeto software y eventos que provocan las transiciones dentro de los objetos. El nivel de detalle suele ser alto.
	Construcción	Modelo: Cambios de estado de un objeto software y eventos que provocan las transiciones dentro de los objetos. El nivel de detalle suele ser alto.
	Pruebas	Modelo: Cambios de estados y eventos de objetos sometidos a pruebas. Su uso y nivel de complejidad dependen directamente de si se elaboro o no este modelo en diseño y construcción.

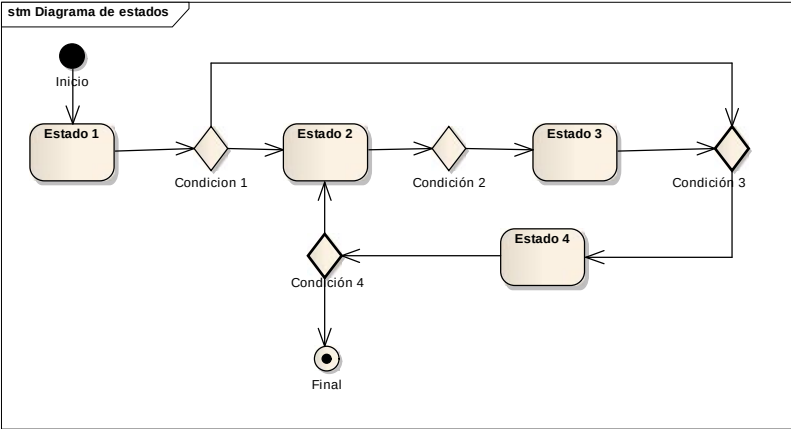


Diagrama	Actividades	
Fase	Análisis	Modelo: Especificación de casos de uso y escenarios. Puede ser utilizado como una herramienta de comunicación, sin embargo el nivel de detalle para esto debe ser muy alto. Modelado: Flujo del negocio en un aspecto general, es decir entre entidades macro.
	Pruebas	Modelado: Flujos de pruebas de caja blanca. Suele ser utilizado por ingenierías inversas, para documentar paso a paso un escenario obtenido. Modelado: Flujos de prueba de caja negra. Describe uno a uno los pasos a seguir para realizar una prueba dentro del sistema.

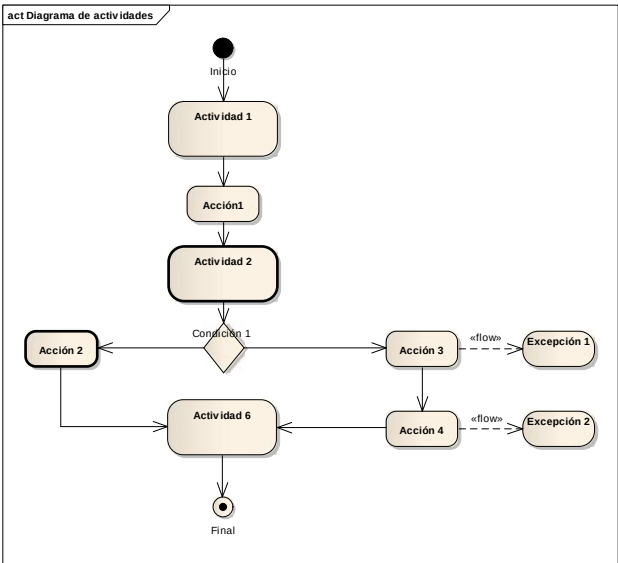


Diagrama	Comunicación	
Fase	Análisis	Modelo: Comportamiento de las entidades en un escenario, se enfoca a mostrar mensajes de comunicación entre las entidades
	Diseño	Modelo: Relaciones entre objetos. Este modelo requiere de los diagramas de secuencia. El nivel de detalle es el deseado.
	Construcción	Modelo: Relaciones entre objetos. Este modelo requiere de los diagramas de secuencia. El nivel de detalle suele ser alto.
	Pruebas	Modelo: Comunicación mediante mensajes entre objetos que están siendo probados. Su uso y nivel de detalle depende de diseño.

sd Diagrama de comunicación

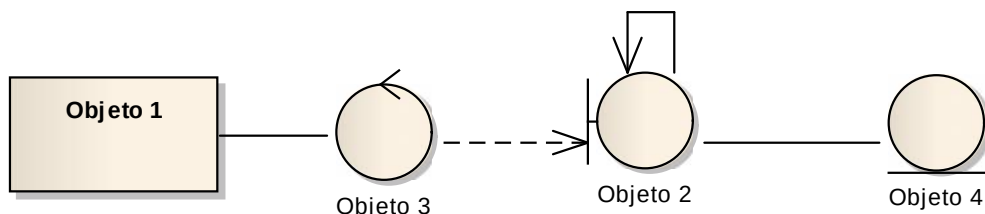


Diagrama	Paquetes	
Fase	Análisis	Modelo: Agrupaciones de casos de uso, esto ocurre generalmente cuando se definen módulos desde esta etapa. Al ser un agrupamiento, el nivel de detalle es muy general. Modelo: Agrupaciones de clases conceptuales. Suele utilizarse para aislar conceptos específicos del negocio debido a condiciones especiales y con un nivel de detalle muy general.
	Diseño	Modelo: Sub sistemas o módulos relacionados o dependientes entre ellos. El contenido en cada sub sistema o módulo suele tener características en común que permite agrupaciones. Suele utilizarse solo para visualizar dependencias y/o relaciones, y su nivel de detalle es general.
	Construcción	Modelo: Agrupaciones de clases software. Es una buena herramienta para analizar el acoplamiento entre objetos. El nivel de detalle suele ser de acuerdo a las necesidades.

pkg Diagrama de paquetes

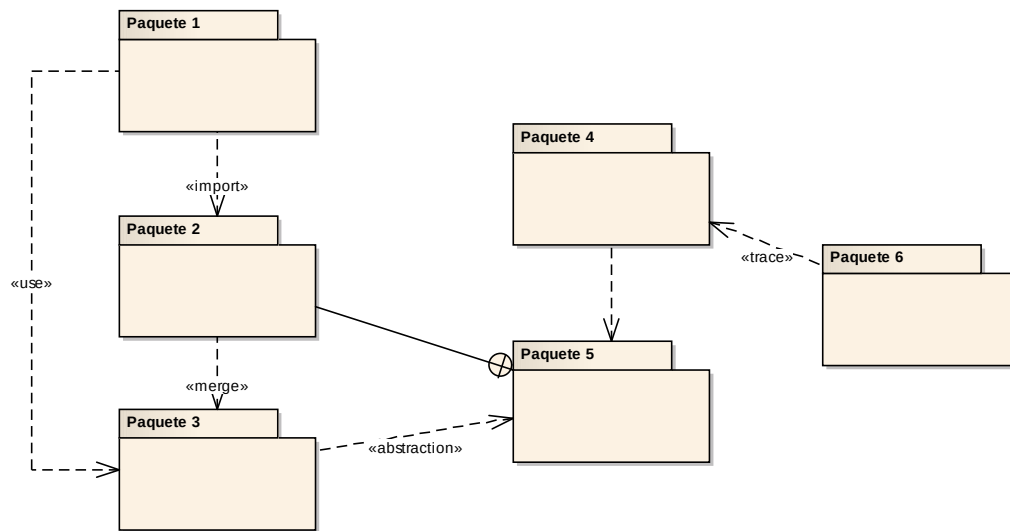


Diagrama	Objetos (estático)	
Fase	Análisis	Modelo: Relación entre entidades de un escenario.
	Diseño	Modelo: Vínculos entre objetos en un escenario determinado. El nivel de detalle es general o el deseado.
	Construcción	Modelo: Vínculos y estados entre objetos en un momento determinado de la ejecución en la aplicación. El nivel de detalle suele ser el deseado, pero se sugiere un alto.

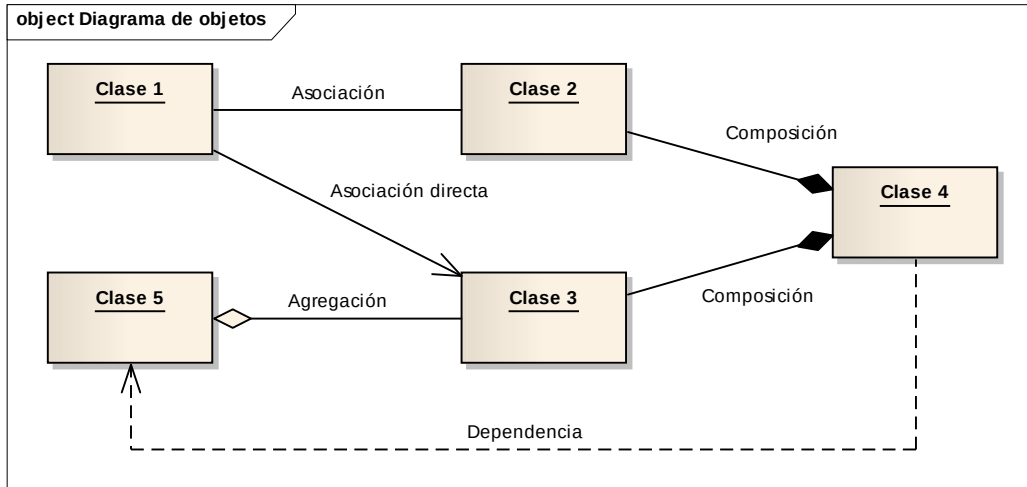


Diagrama	Despliegue	
Fase	Diseño	Modelo: Nodos de hardware o entornos de ejecución, relaciones y artefactos de como se espera desplegarlos. Se toma en cuenta el tipo de nodos y artefactos. El nivel de detalle suele ser alto.
	Construcción	Modelo: Nodos de hardware o entornos de ejecución, relaciones y artefactos de como se espera desplegarlos. Se toma en cuenta el tipo de nodos y artefactos. El nivel de detalle suele ser general.
	Despliegue	Modelo: Despliegue de los artefactos, nodos de hardware, entornos de ejecución y relaciones en tiempo real. El nivel de detalle se enfoca en nodos y artefactos concretos.

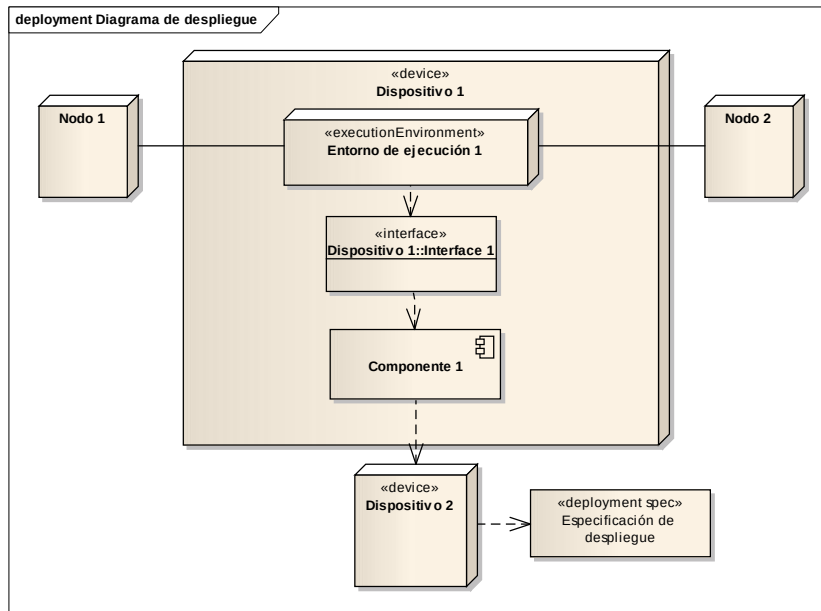


Diagrama	Componentes	
Fase	Diseño	Modelo: Relación entre componentes de software y sus interfaces.
	Construcción	Modelo: Relación entre componentes de software y sus interfaces.

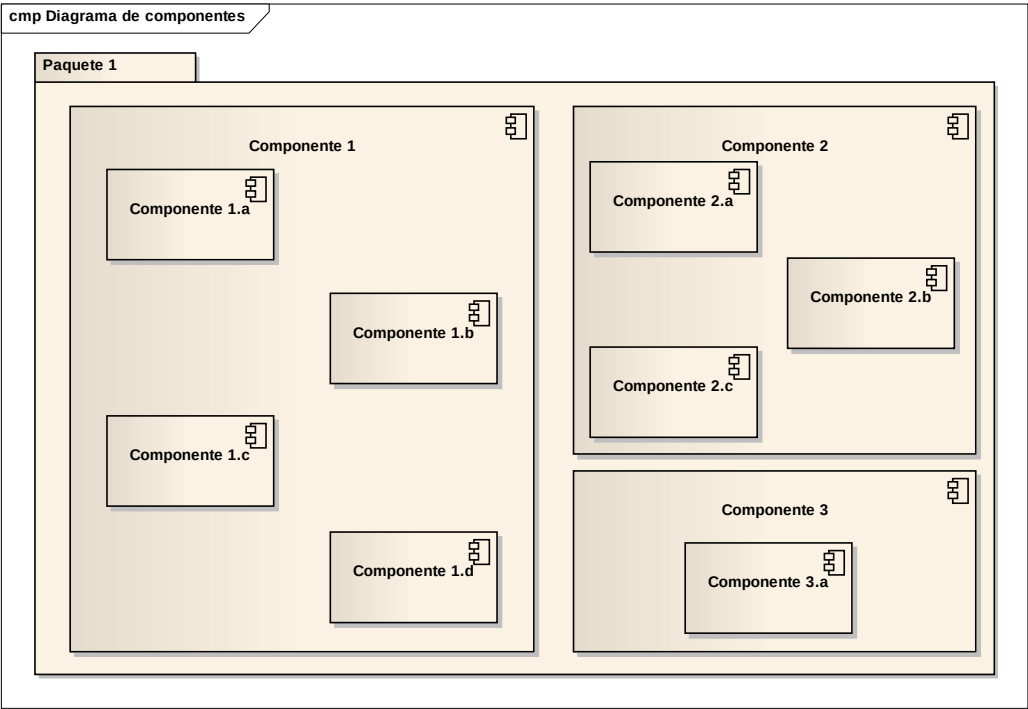


Diagrama	Casos de uso	
Fase	Análisis	Modelo: Requerimientos del sistema, relaciones entre requisitos y entre requisitos con actores o entre actores. Para que sea una buena herramienta de comunicación el nivel de detalle debe ser el mismo en todo el modelo, respecto a la funcionalidad.

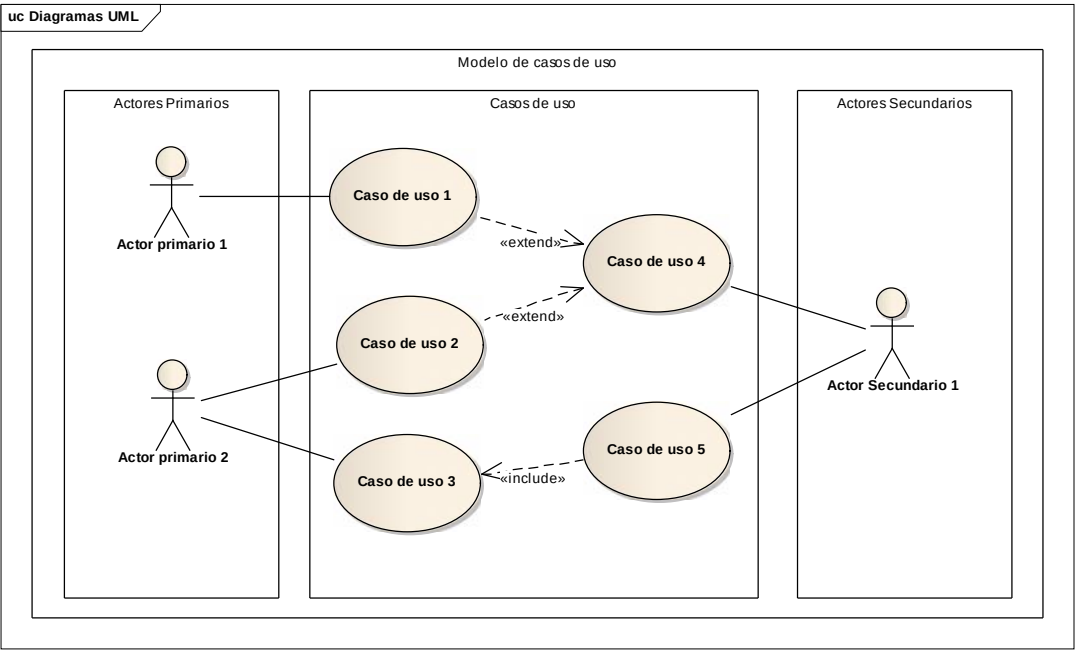


Diagrama	Tiempos	
Fase	Diseño	Modelo: Restricciones temporales que dependen de los diagramas de secuencia. Su uso es poco habitual y por eso mismo, su nivel de abstracción se sugiere a detalle.
	Construcción	
	Pruebas	Modelo: Restricciones temporales. El uso y nivel de detalle de este diagrama, depende directamente de los diagramas de secuencia

sd Diagrama de Tiempos

TimeLine1

Estado 0

Estado 1

Estado 2

Mensaje

Mensaje

0

10

20

30

40

50

60

70

80

90

100

ANEXO 2. ARTEFACTOS PARA EVALUACIÓN

Tabla 1. Checklist para analista

Instrucciones:

La siguiente lista de verificación tiene como objetivo evaluar la complejidad de un requerimiento basándose en los requerimientos obtenidos, para ello se debe contestar a cada uno de los enunciados con una "X" en la columna que considere pertinente.

Pregunta	SI	NO
1. ¿El requerimiento representa alguna nueva entidad dentro del modelo de dominio?		
2. Si el requerimiento representa una nueva entidad ¿se encuentra descrito dentro del artefacto de negociación?		
3. ¿El requerimiento interviene con alguna entidad definida en el modelo de dominio?		
4. De omitir este requerimiento, ¿el sistema dejaría de funcionar de acuerdo a lo descrito por el flujo definido por el usuario?		
5. De omitir este requerimiento, ¿podría ser afectada alguna entidad del modelo de dominio?		
6. ¿El requerimiento fue definido por el cliente?		
7. ¿El requerimiento fue sugerido por el analista?		
8. ¿El requerimiento se encuentra descrito como una característica que se desea para el sistema?		
9. ¿El requerimiento se encuentra descrito con el nombre bien definido de algún elemento (sustantivo)?		
10. ¿El requerimiento se encuentra descrito como una acción que se realiza comúnmente (verbo en infinitivo)?		
11. ¿El requerimiento describe más de una necesidad del sistema?		
12. Se describe ¿Quién o qué lo usa el requerimiento?		
13. Se describe ¿qué elementos intervienen con requerimiento?		
14. Se describe ¿cómo funciona el requerimiento dentro del flujo del negocio?		
15. Se describe ¿cuándo se usa el requerimiento dentro del flujo del negocio?		
16. Se describe ¿dónde se usa el requerimiento, dentro de las entidades del modelo de dominio?		
17. Se describe ¿por qué es necesario para el sistema?		
18. ¿El requerimiento es especificado por más de un usuario?		
19. ¿El requerimiento contradice o se opone a otros requerimientos?		
20. ¿El requerimiento tiene coherencia dentro del modelo de dominio?		
21. ¿El requerimiento es inconfundible con otros requerimientos?		
22. ¿El requerimiento puede ser identificado dentro de otros requerimientos?		
23. ¿El requerimiento puede ser ubicado dentro del sistema?		

Tabla 2. Checklist para usuario

Instrucciones:

La siguiente lista de verificación tiene como objetivo evaluar la complejidad de un requerimiento basándose en los requerimientos obtenidos, para ello se debe contestar a cada uno de los enunciados con una "X" en la columna que considere pertinente.

Pregunta	SI	NO
1. ¿El nombre del documento describe claramente los requerimientos a los que se encuentra asociado?		
2. ¿El documento describe claramente los tipos o niveles de usuarios que actúan con en el requerimiento?		
3. ¿Es posible confundir el requerimiento especificado con algún otro?		
4. ¿La descripción textual del documento refiere claramente las características del requerimiento?		
5. ¿Las características descritas cubren las necesidades especificadas inicialmente para el requerimiento?		
6. ¿Existe una asociación coherente entre el documento y la naturaleza del requerimiento solicitado?		
7. ¿La descripción, satisface condiciones de la etapa del negocio a la que pertenece el requerimiento?		
8. ¿Se utiliza un lenguaje conocido por los involucrados?		
9. ¿Los términos desconocidos o complejos utilizados en estos documentos se encuentran definidos en éste u otro artefacto?		
10. ¿La especificación, contiene las condiciones necesarias para realizar pruebas al sistema recorriendo sus condiciones o validando la existencia del requerimiento?		
11. ¿Se describe paso a paso el flujo para cubrir el requerimiento?		
12. ¿El documento describe lo sugerido por el consultor?		
13. ¿El documento describe una característica solicitada por el cliente?		
14. ¿El documento fue especificado por el cliente y el consultor?		
15. ¿El documento describe alguna entidad del negocio?		
16. ¿El documento una característica del negocio?		
17. ¿El documento describe un proceso del negocio?		
18. ¿El nivel de detalle para la especificación en el documento es uniforme para todos los requerimientos de este tipo?		
19. ¿El documento justifica la necesidad del requerimiento solicitado?		
20. ¿El documento describe de forma clara en que momento hace referencia a otro artefacto?		
21. ¿Se describe de manera explícita todos los elementos con los que se encuentra relacionado el requerimiento?		
22. ¿Se describe, textualmente, en que momento surge la dependencia con otros requerimientos?		
23. ¿Una modificación a este documento, impacta a más de un requerimiento?		
24. ¿Se describe claramente el tipo de dependencia que tiene con otros requerimientos?		
25. ¿Se han especificado todos los requerimientos para el sistema?		

Tabla 3. Checklist para diseño

Instrucciones:

La siguiente lista de verificación tiene como objetivo evaluar la complejidad de un requerimiento basándose en los requerimientos obtenidos, para ello se debe contestar a cada uno de los enunciados con una "X" en la columna que considere pertinente.

Pregunta	SI	NO
1. ¿Se contemplan todos los actores del sistema?		
2. ¿El nombre de cada actor del sistema es único e inconfundible?		
3. ¿El nombre de cada caso de uso es único e inconfundible?		
4. ¿Existe un caso de uso por funcionalidad?		
5. ¿Se visualizan correctamente las dependencias entre los casos de uso (inclusión o extensión)?		
6. ¿Se encuentran bien especificados los límites del sistema?		
7. ¿Todos los casos de uso se encuentran asociados al menos a un actor u otro caso de uso?		
8. ¿Todos los actores se encuentran asociados al menos a un caso de uso u otro actor?		
9. ¿La información de cada clase se encuentra en su sección correspondiente?		
10. ¿El nombre de la clase es único e inconfundible?		
11. ¿Se encuentran bien definidos las características, transacciones y permisos dentro de cada clase?		
12. ¿Se encuentran bien definidas las transacciones de cada clase?		
13. ¿Se encuentran bien definidos los permisos de cada clase?		
14. ¿Se visualizan correctamente las relaciones entre las clases (asociaciones, herencias, agregaciones y dependencias o instancias)?		
15. ¿Se muestra una cardinalidad específica para cada relación?		
16. ¿El nombre de cada estado es único e inconfundible?		
17. ¿Todos los estados poseen al menos una transición que le permite comunicarse con uno o más estados?		
18. ¿Se identifica claramente las transacciones de entrada y salida de cada estado?		
19. ¿En las transiciones se encuentran claramente identificadas las condiciones de las que depende el cambio?		
20. ¿El nombre del diagrama es intuitivo de la actividad a describir?		
21. ¿Se encuentran claramente identificados los nodos (inicio y fin)?		
22. ¿El nombre de cada estado de acción es único e inconfundible?		
23. ¿Se encuentran claramente definidas las transiciones en el flujo?		
24. ¿Todos los estados acción, poseen una transición de entrada y una de salida?		
25. ¿Se encuentran identificadas las actividades que se desarrollan de forma simultánea entre barras de sincronización?		
26. ¿Los nodos de decisión muestran todas las transiciones de acuerdo a la condición que deben cumplir?		
27. ¿Los carriles agrupan de forma ordenada las responsabilidades en el flujo?		
28. ¿El flujo del diagrama es lógico y coherente?		

Pregunta	SI	NO
29. ¿El nombre del diagrama describe de forma intuitiva el evento que desencadena las acciones en secuencia para el escenario específico?		
30. ¿Se encuentran claramente identificados los objetos que deben interactuar en la secuencia de acuerdo al caso de uso?		
31. ¿La secuencia de comunicación entre los objetos se realiza de acuerdo al flujo indicado en el caso de uso?		
32. ¿Todos los objetos en el diagrama poseen al menos un mensaje de comunicación con otros?		
33. ¿Se describe de forma clara el periodo de activación de cada objeto?		
34. ¿Se encuentran claramente identificados los objetos que deben interactuar?		
35. ¿La secuencia entre mensajes se encuentra especificada mediante un número y una flecha dirigida?		
36. ¿Todos los objetos poseen al menos un enlace de comunicación con otro objeto?		
37. ¿Se encuentran claramente identificados los componentes del sistema?		
38. ¿Se encuentran claramente identificadas las relaciones entre componentes (dependencias)?		
39. ¿Las clases contenidas en el componente se encuentran bien definidas?		

GLOSARIO

Abstracción.

- 1) Del latín *abstractio*, el concepto abstracción está vinculado al verbo abstraer (separar las propiedades de un objeto a través de una operación mental, dejar de prestar atención al mundo sensible para centrarse en un pensamiento). La abstracción, por lo tanto, es alguna de estas acciones o sus efectos.
- 2) Para la filosofía, la abstracción es una actividad cerebral que permite aislar, a nivel conceptual, una cierta cualidad de algo con la intención de entregarse a una reflexión sobre la misma sin considerar el resto de las propiedades del objeto en cuestión.

Administración.

- 1) El concepto de administración hace referencia al funcionamiento, la estructura y el rendimiento de las organizaciones. El término proviene del latín *ad-ministrare* ("servir") o *ad manus trahere* ("manejar" o "gestionar").
- 2) La administración puede ser entendida como la disciplina que se encarga de realizar una gestión de los recursos (ya sean materiales o humanos) en base a criterios científicos y orientada a satisfacer un objetivo concreto.

Análisis.

- 1) A nivel general, puede decirse que un análisis consiste en identificar los componentes de un todo, separarlos y examinarlos para lograr acceder a sus principios más elementales.

Calidad.

- 1) La obtención de un producto o servicio acorde con el coste, las especificaciones y expectativas que se tienen (Piattini).
- 2) El primer paso que se hace necesario llevar a cabo para poder determinar la esencia del concepto calidad que ahora nos ocupa es establecer su origen etimológico. En este sentido, tenemos que subrayar que el mismo se encuentra en la palabra latina *qualitas*, la cual a su vez procede del griego y más en concreto del término *ποιότης*.

Ciclo de vida

- 1) Es una sucesión de estados o fases por los cuales pasa un software a lo largo de su "vida".

Codificación

- 1) Transformación de la formulación de un mensaje a través de las reglas o normas de un código o lenguaje predeterminado.

Código

- 1) El verbo codificar viene del latín compuesto de *codex* (libro, registro, código) y el verbo *facere* (hacer). Se refiere a formular de manera reglada el cuerpo de un mensaje y un cuerpo de leyes.
- 2) El término código tiene diferentes usos y acepciones. Puede tratarse de una combinación de símbolos que, en el marco de un sistema ya establecido, cuente con un cierto valor.
- 3) En el caso de la informática, se conoce como código fuente al texto desarrollado en un lenguaje de programación y que debe ser compilado o interpretado para poder ejecutarse en un ordenador o computadora.

Compilar

- 1) Allegar o reunir, en un solo cuerpo de obra, partes, extractos o materias de otros varios libros, documentos o lenguaje máquina a partir de otro programa de ordenador

Criterio

- 1) El termino criterio tiene su origen en un vocablo griego que significa “juzgar”. El criterio es el juicio o discernimiento de una persona.

Defecto

- 1) Del latín *defectus*, un defecto es una imperfección en alguien o algo. El diccionario de la Real Academia Española (RAE) define al término como la carencia de alguna cualidad propia de algo. El concepto se utiliza como sinónimo de error, fallo o desperfecto. Los defectos pueden ser perceptibles por los sentidos (como una camisa a la que le falta un botón), advertirse en el funcionamiento de alguna cosa (un coche con problemas de freno) o estar vinculados a algo más simbólico o subjetivo (los defectos morales de una persona).

Definición

- 1) Es una proposición mediante la cual se trata de exponer de manera unívoca y con precisión la comprensión de un concepto o término o dicción o –si consta de dos o más palabras– de una expresión o locución. Se alude a determinar, por escrito u oralmente, de modo claro y exacto, las cualidades esenciales del tema implicado.
- 2) El vocablo definición deriva del caso genitivo latino *dēfīnīōnis*: del fin (nominativo *dēfīnīō*). Es decir: denota marcación de un límite del significado.

Desarrollo

- 1) Es un proceso continuo, ordenado en fases, a lo largo del tiempo, que se construye con la acción del sujeto al interactuar con su medio adaptándose gradualmente.
- 2) El desarrollo es la condición de evolución que siempre tiene una connotación positiva ya que implica un crecimiento o paso hacia etapas o estadios superiores.

Eficacia

- 1) Del latín *efficācia*, la eficacia es la capacidad de alcanzar el efecto que espera o se desea tras la realización de una acción. No debe confundirse este concepto con el de eficiencia (del latín *efficientia*), que se refiere al uso racional de los medios para alcanzar un objetivo predeterminado (es decir, cumplir un objetivo con el mínimo de recursos disponibles y tiempo).

Eficiencia

- 1) La noción de eficiencia tiene su origen en el término latino *efficientia* y refiere a la habilidad de contar con algo o alguien para obtener un resultado. El concepto también suele ser equiparado con el de fortaleza o el de acción.
- 2) Se trata de la capacidad de alcanzar un objetivo fijado con anterioridad en el menor tiempo posible y con el mínimo uso posible de los recursos, lo que supone una optimización.

Ejecución

- 1) Del latín *exsecutō*, el término ejecución permite nombrar a la acción y efecto de ejecutar. Este verbo tiene varios significados: poner por obra algo, desempeñar algo con facilidad, tocar una pieza musical, ajusticiar, reclamar una deuda por un procedimiento ejecutivo o, en informática, realizar las operaciones que son especificadas por un programa.

Elemento

- 1) En otros sentidos, se conoce como elemento a la parte integrante de algo, a las piezas que forman una estructura

Entidad

- 1) Del latín medieval *entitas*, entidad es toda colectividad que puede considerarse como una unidad.

Error

- 1) Un error es algo equivocado o desacertado. Puede ser una acción, un concepto o una cosa que no se realizó de manera correcta.

Especificación

- 1) Determinación, explicación o detalle de las características o cualidades de una cosa
- 2) Descripción de las características de un producto, elaborada por su fabricante por razones legales, de inventario, o para beneficio del comprador.

Estándar

- 1) Un estándar es un conjunto de reglas que deben cumplir los productos, procedimientos o investigaciones que afirmen ser compatibles con el mismo producto.
- 2) Los estándares son acuerdos (normas) documentados que contienen especificaciones técnicas u otros criterios precisos para ser usados consistentemente como reglas, guías, o definiciones de características, para asegurar que los materiales productos, procesos y servicios se ajusten a su propósito.

Etapas

- 1) El concepto de etapa tiene su origen en el vocablo francés *étape* y puede hacer referencia tanto a una porción del camino de un trayecto específico, como al sitio en el cual se hace una pausa para descansar en el marco de un traslado o a una fase en el desarrollo de una determinada actividad o acción.
- 2) En algunos campos, como la tecnología, las etapas son entendidas como cada una de las fases de un procedimiento técnico; para la historia, consisten en cada momento histórico con sus particularidades distintivas. En algunos libros se encuentra presente como sinónimo del concepto de período.

Evaluación.

- 1) Proceso por el que se estima, se aprecia, se calcula el valor de una cosa. (Piattini, 2003)
- 2) Su etimología se remonta al francés *évaluer* y que permite indicar, valorar, establecer, apreciar o calcular la importancia de una determinada cosa o asunto.

Fiabilidad

- 1) En el campo de la psicología, la educación y la investigación social, la fiabilidad (también llamada técnicamente confiabilidad) es una propiedad psicométrica que hace referencia a la ausencia de errores de medida, o lo que es lo mismo, al grado de consistencia y estabilidad de las puntuaciones obtenidas a lo largo de sucesivos procesos de medición con un mismo instrumento.
- 2) El término fiabilidad es descrito en el diccionario de la RAE como "probabilidad de buen funcionamiento de algo".

Flujo

- 1) El concepto de flujo (vocablo derivado del latín *fluxus*) da nombre al acto y la consecuencia de fluir (entendido como sinónimo de brotar, correr o circular).

Frecuencia absoluta

- 1) Es el número de veces que aparece un determinado valor en un estudio estadístico. Se representa por f_i .
La suma de las frecuencias absolutas es igual al número total de datos, que se representa por N .

$$f_1 + f_2 + f_3 + \dots + f_n = N$$

Para indicar resumidamente estas sumas se utiliza la letra griega Σ (sigma mayúscula) que se lee suma o sumatoria.

$$\sum_{i=1}^{i=N} f_i = N$$

La frecuencia relativa

- 1) Es el cociente entre la frecuencia absoluta de un determinado valor y el número total de datos. Se puede expresar en tantos por ciento y se representa por n_i .

$$n_i = \frac{f_i}{N}$$

La suma de las frecuencias relativas es igual a 1.

Frecuencia acumulada

- 1) La frecuencia acumulada es la suma de las frecuencias absolutas de todos los valores inferiores o iguales al valor considerado.

Funcionalidad

- 1) El concepto de función tiene su origen en el término latino *functio*. La palabra puede ser utilizada en diversos ámbitos y con distintos significados.
- 2) Este verbo hace referencia a ejecutar las funciones que le son propias a algo o alguien o a aquello que marcha o resulta bien.

Gestión

- 1) Del latín *gestio*, el concepto de gestión hace referencia a la acción y a la consecuencia de administrar o gestionar algo. Al respecto, hay que decir que gestionar es llevar a cabo diligencias que hacen posible la realización de una operación comercial o de un anhelo cualquiera.
- 2) Administrar, por otra parte, abarca las ideas de gobernar, disponer, dirigir, ordenar u organizar una determinada cosa o situación.

Grafico para control de atributos

- 1) Los gráficos de control por atributos toman como referencia una variable discreta. Se caracterizan porque representan la evolución de características no medibles ni cuantificables

Gráfico de Control c.

- 1) Se grafica directamente el número de fallas o defectos y se utiliza para controlar y analizar directamente la variación en el número de fallas o defectos que aparecen en una unidad de producción constante (Ej. no. de fisuras en un lote de lamina). Tiene el mismo objetivo que el Gráfico u pero el tamaño de la muestra o unidad de producción debe ser constante.

Gráfico de Control p.

- 1) Muy utilizado para controlar y analizar el Porcentaje de productos defectuosos en la línea de producción y la variación entre una y otra muestra, cuyo tamaño puede variar.

Gráfico de Control pn.

- 1) Muy utilizado para controlar y analizar directamente el número de productos defectuosos en la línea de producción. Es el mismo objetivo que el Gráfico p con la diferencia de que el tamaño de la muestra debe ser constante.

Gráfico de Control u

- 1) Se utiliza para controlar y analizar la variación en el número de defectos que aparecen por unidad de producción (Ej. no. de fisuras en 1 mt. de lámina, No. de errores por auto arreglado, No. de errores en cada formato W) y el tamaño de dicha muestra o unidad de producción puede variar

Herramienta

- 1) Del latín *ferramenta*, una herramienta es un instrumento que permite realizar ciertos trabajos. Estos objetos fueron diseñados para facilitar la realización de una tarea

Ingeniería

- 1) Si queremos conocer el origen etimológico de la palabra ingeniería que ahora nos ocupa lo que tenemos que hacer es marcharnos, metafóricamente hablando, hasta el latín pues descubriremos que dicho término emana del vocablo latino *ingenium*. Un término este que puede traducirse como “producir”.
- 2) La ingeniería es el estudio y la aplicación de las distintas ramas de la tecnología. El profesional en este ámbito recibe el nombre de ingeniero.

Ingeniería inversa

- 1) Proceso de obtención de información técnica a partir de un producto accesible al público, con el fin de determinar de qué está hecho, que lo hace funcionar y como fue fabricado
- 2) La ingeniería inversa es el proceso de descubrir los principios tecnológicos de un dispositivo, objeto o sistema, a través de razonamiento abductivo de su estructura, función y operación.

Interface (interfaz)

- 1) Interfaz es un término que procede del vocablo inglés *interface* (“superficie de contacto”).
- 2) En informática, esta noción se utiliza para nombrar a la conexión física y funcional entre dos sistemas o dispositivos.

Iteración

- 1) Iteración es un vocablo que tiene su origen en el término latino *iteratio*. Se trata de una palabra que describe el acto y consecuencia de iterar, un verbo que se emplea como sinónimo de reiterar o repetir (entendidos como volver a desarrollar una acción o pronunciar de nuevo lo que ya se había dicho).

Mantenimiento

- 1) En términos generales por mantenimiento se designa al conjunto de acciones que tienen como objetivo mantener un artículo o restaurarlo a un estado en el cual el mismo pueda desplegar la función requerida o las que venía desplegando hasta el momento en que se dañó, en caso que haya sufrido alguna rotura que hizo que necesite del pertinente mantenimiento y arreglo.

Método

- 1) Método es una palabra que proviene del término griego *methodos* (“camino” o “vía”) y que se refiere al medio utilizado para llegar a un fin. Su significado original señala el camino que conduce a un lugar.
- 2) Proceso o camino sistemático establecido para realizar una tarea o trabajo con el fin de alcanzar un objetivo predeterminado.
- 3) Un procedimiento que se usa para realizar una tarea específica en la clase o módulo.

Metodología

- 1) Metodología es un vocablo generado a partir de tres palabras de origen griego: *metà* (“más allá”), *odòs* (“camino”) y *logos* (“estudio”). Por lo tanto, puede entenderse a la metodología como el conjunto de procedimientos que determinan una investigación de tipo científico o marcan el rumbo de una exposición doctrinal.

Métrica.

- 1) El área que trata de medir la usabilidad bajo un determinado criterio o parámetro, utilizando un método, un procedimiento o una técnica determinada o cualquier combinación de ellas (Piattini).

- 2) En términos generales, por métrica, se designará a todo aquello que corresponda o esté vinculado al metro o a su sistema de medida.
- 3) Es una metodología de planificación, desarrollo y mantenimiento de sistemas de información.

Migración

- 1) Se denomina migración al proceso mediante el cual los programas y la información de una computadora o sistema se traspasan a otro.
- 2) Se denomina migración al proceso mediante el que un individuo se desplaza de una zona geográfica hacia otra.

Modelo

- 1) Un modelo es una representación de un objeto, sistema o idea, de forma diferente al de la entidad misma. El propósito de los modelos es ayudarnos a explicar, entender o mejorar un sistema. Un modelo de un objeto puede ser una réplica exacta de éste o una abstracción de las propiedades dominantes del objeto (Universidad nacional de Colombia).
- 2) Un modelo es un bosquejo que representa un conjunto real con cierto grado de precisión y en la forma más completa posible, pero sin pretender aportar una réplica de lo que existe en la realidad.

Modificación

- 1) Christopher Alexander definió el patrón en 1979 como una regla de tres partes, la cual expresa una relación entre un cierto contexto, un problema y una solución. Es en general un problema que ocurre y una y otra vez en nuestro entorno, describiendo el núcleo de la solución a ese problema, de tal manera que esa solución pueda ser usada mas de un millón de veces sin hacerlo dos veces de la misma forma (Piattini, 2003)
- 2) Del latín *modificatio*, modificación es la acción y efecto de modificar. Este verbo, cuyo origen etimológico nos remite al latín *modificāre*, hace mención a cambiar o transformar algo, dar un nuevo modo de existencia a una sustancia material o a limitar algo a cierto estado de manera en que se distinga de otras cosas.

Portabilidad

- 1) La portabilidad, en informática, se refiere a la capacidad de un programa o sistema de ejecutarse en diferentes plataformas o arquitecturas con mínimas modificaciones.
- 2) La portabilidad, en cuanto al hardware, es la facilidad con que se puede transportar un dispositivo electrónico (computadora, consola de juego, etc.) y está básicamente relacionado al peso y las dimensiones del equipo. A menor peso y dimensiones, más portable es el dispositivo.

Problema

- 1) la hora de analizar a fondo el término problema es necesario que antes estipulemos el origen etimológico del mismo. En este sentido, podemos decir que aquel se encuentra en el latín y más concretamente en la palabra *problēma*. No obstante, también hay que determinar que la misma a su vez procede del término griego *πρόβλημα*.
- 2) Es un determinado asunto o una cuestión que requiere de una solución. A nivel social, se trata de alguna situación en concreto que, en el momento en que se logra solucionar, aporta beneficios a la sociedad (como lograr disminuir la tasa de pobreza de un país o reconstruir edificios arrasados por un terremoto).

Proceso

- 1) La noción de proceso halla su raíz en el término de origen latino *processus*. Según informa el diccionario de la Real Academia Española (RAE), este concepto describe la acción de avanzar o ir para adelante, al paso del tiempo y al conjunto de etapas sucesivas advertidas en un fenómeno natural o necesario para concretar una operación artificial.
- 2) En la informática, un proceso es un conjunto de procedimientos o funciones que tienen uno o más objetivos.

Prototipo

- 1) Un prototipo es un modelo (representación, demostración o simulación) fácilmente ampliable y modificable de un sistema planificado, probablemente incluyendo su interfaz y su funcionalidad de entradas y salidas.

Prueba

- 1) Una prueba también puede ser una evaluación o un examen que se hace para que algo o alguien demuestre su nivel de satisfacción hacia otro.
- 2) Una prueba de concepto es una implementación, a menudo resumido o incompleto, de un método, realizado con el propósito de verificar que el concepto o teoría en cuestión es susceptible de ser explotada de una manera útil.

Re ingeniería

- 1) Rediseñar procesos como ser el de manufactura, desarrollo de producto, ventas, despacho de pedidos y servicio, entre otros, para darle una nueva cara a las compañías.
- 2) “Reingeniería es la revisión fundamental y el rediseño radical de procesos para alcanzar mejoras espectaculares en medidas críticas y contemporáneas de rendimiento, tales como costos, calidad, servicio y rapidez”. (Hammer y Champy)

Requerimiento

- 1) En ingeniería del software y el desarrollo de sistemas, un requerimiento es una necesidad documentada sobre el contenido, forma o funcionalidad de un producto o servicio.
- 2) Los requerimientos son declaraciones que identifican atributos, capacidades, características y/o cualidades que necesita cumplir un sistema (o un sistema de software) para que tenga valor y utilidad para el usuario. En otras palabras, los requerimientos muestran qué elementos y funciones son necesarias para un proyecto. (alegsa)

Riesgo

- 1) Riesgo proviene del italiano *risico* o *rischio* que, a su vez, tiene origen en el árabe clásico *rizq* (“lo que depara la providencia”). El término hace referencia a la proximidad o contingencia de un posible daño.

Sistema

- 1) Del latín *systema*, un sistema es módulo ordenado de elementos que se encuentran interrelacionados y que interactúan entre sí. El concepto se utiliza tanto para definir a un conjunto de conceptos como a objetos reales dotados de organización.
- 2) Conjunto de procesos o elementos interrelacionados con un medio para formar una totalidad encauzada hacia un objetivo común.

Trazabilidad

- 1) Para la International Organization of Standardization (cuya sigla es ISO), la trazabilidad es la propiedad que dispone el resultado de un valor estándar, que puede vincularse con referencias específicas mediante una seguidilla continuada de comparaciones.
- 2) La trazabilidad es una herramienta para conocer todos los elementos que intervienen en la elaboración de un producto (materias primas, aditivos, envases, etc.) y todas las fases por las que pasa dicho producto (adquisición, recolección, producción, elaboración, almacenaje, distribución, etc.).

Usabilidad

- 1) El concepto proviene del inglés *usability* y hace referencia a la facilidad con que un usuario puede utilizar una herramienta fabricada por otras personas con el fin de alcanzar un cierto objetivo.
- 2) En otras palabras, la noción tiene relación con la eficacia percibida de un objeto y la posibilidad de aprovechar todo su potencial.

Validación.

- 1) ¿Estamos construyendo el producto correcto? Se ocupa de controlar si el producto satisface los requerimientos del usuario (Bohem)
- 2) El adjetivo válido, por otra parte, hace referencia a aquello que vale legalmente o que es firme y subsistente. revisión al que se somete un programa informático para comprobar que cumple con sus especificaciones

Verificación.

- 1) Según Bohem (1979) ¿Estamos construyendo correctamente el producto? Implica controlar que el producto conforma su especificación inicial.
- 2) La verificación confirma que los productos reflejan los requisitos especificados para cada caso concreto, garantizando que “desarrolló el producto correctamente”. (PTC)